Experiment plan: turning the two findings into laws

Version 48, frozen on 23 October 2026. It amends block C's placement before any of its judged runs, after version 47's pilot showed that the card's interrupts on the consumer's own vCPU starve a consumer at real-time priority on this testbed, and runs block C's own pilot first (D48-1 to D48-4). It changes no prediction.

Grey text in brackets is a plain-word explanation. Green tags such as **[D48-1]** mark what changed since version 47, and grey tags such as [D47-1], [D46-1], [D45-1], [D44-1], [D43-1], [D42-1], [D41-1], [D40-1], [D39-1], [D38-1], [D37-1], [D36-1], [D35-1], [D34-1], [D33-1], [D32-1], [D31-1], [D30-1], [D29-1], [D28-1], [D27-1], [D26-1], [D25-1], [D24-1], [D23-1], [D22-1], [D21-1], [D20-1], [D19-1], [D17-1], [D16-1], [D15-1], [D14-1], [D13-1], [D12-1], [D11-1], [D10-1], [D9-1], [D8-1], [D7-1], [D6-1], [D5-2] and [D3-5] what changed in earlier versions; the tables below say what each change is and why. Prices are Azure's list prices as read again on 16 September 2026, and they change.

In one paragraph

The paper has three parts, and this plan serves all of them: **the mechanism** of what is happening, **that it is happening**, and **that the industry is not dealing with it**. [D3-1]

Both findings rest on old ideas: late clock readings were shown in 2012, and millisecond rounding is in a 1970 HP manual. What could make them laws is prediction. We write down numbers *before* running, on machines and tools we have never touched, and then check them.

Mode 1 (late notes, the mechanism): we test one specific law, *plateau, cliff, floor.* The machine's scheduler settings predict the trip length at which negative results suddenly stop, and how sharply they stop.

Mode 2 (rounding and throwing away, the industry): a tool's clock, its rounding and its filter should predict, before it runs, what it will report. Its documentation and issue tracker show whether anyone has dealt with it.

Version 3 added what the pilot on the first Azure machines showed: a calibration of the added delay in every session, a study of why that delay does not lengthen the trip one-for-one, three tick kernels, and the tools block. Version 4 splits the work into 47 small campaigns, each answering one question in a few hours. It sets the number of repeats from a simulation of each campaign's own test, drops what earlier campaigns already answered, and lets campaigns run at the same time on separate machine pairs without mixing machines inside any test. [D4-1]

Version 5 writes down how the campaigns are run. A program judges every run the moment it ends, by rules fixed here, and stops a campaign when the instrument is in doubt. A watch reads every machine pair every five minutes, and every file is fingerprinted from the machine that wrote it to the analysis. No prediction, campaign, number of rounds or decision rule changes. [D5-2]

Version 6 corrects what the first runs of version 5 found, and stage 0 starts again. The calibration must now be *known* within 0.3 ms at every step, where version 5 asked every run to lie within 0.1 ms of it, which no machine could meet. The broker's own packet capture checks the added delay, because ping could not decide 0.05 ms. The Redis client acknowledges messages in batches, as the paper's own settings say, and M0 studies bursty traffic, where the departure it explains appears. The other changes protect runs from faults and keep everything measured. [D6-1]

Version 8 says which slices are a test of the law on a given pair, and puts one brake above the noise it brakes. A message cannot arrive sooner than the client's own zero-delay trip, and the plateau the law predicts lives at trips below the slice: the first pair's calibration put that floor at 1.964 ms for Kafka, which leaves A1's two smallest slices with no plateau to stand on. Those slices are reported as out of reach, with the trip that excludes them, and the prediction is judged on the slices the pair can reach. The got-it brake now also has to clear the scatter the session's own zero-delay runs showed. [D8-1]

Version 48 amends block C before any of its judged runs, and changes no prediction. [D48-1] Block C's placement: the card's interrupts on the core's other thread, and the consumer's other threads there too. [D48-2] Block C's own pilot, and what is stated before its judged runs. [D48-3] Block C after block S. [D48-4] What was read before this version.

Version 47 tests a general method in new blocks, and changes no earlier prediction. [D47-1] The method, in four parts. [D47-2] The kernel-timed latency from the client host's captures, by the broker's ids, and its clock against the card. [D47-3] The check, certain and estimated. [D47-4] The three latencies. [D47-5] Poisson sends, and the load in blocks within runs. [D47-6] The brokers pinned. [D47-7] Seven blocks. [D47-8] Predictions on the timestamps, the ids and the check. [D47-9] Predictions on the latencies and the remedy. [D47-10] The judging. [D47-11] Pilot round a, and what is stated before any judged run. [D47-12] What was read before this version, and the order of the runs.

Version 46 amends block U before any of its judged runs, and changes no margin. [D46-1] The producer at real-time priority in every setup, and a lateness rule. [D46-2] The clients' collectors off, and logged. [D46-3] A priming run on each broker before each campaign. [D46-4] Each loaded setup's duty frozen from the pilot. [D46-5] The turnaround from the previous request for messages, less the connection's other requests. [D46-6] The split, with each connection pair's floor and the broker's bound. [D46-7] U-c's share from tail means, with an interval. [D46-8] U-e on the split's terms, and U-f within runs whose load runs in blocks. [D46-9] What every run records, and what is described. [D46-10] The block as amended. [D46-11] Pilot round d, and what is stated before any judged run. [D46-12] What was read before this version, and the order of the runs.

Version 45 tests the fix and the consumer's turnaround where load and rate bite together, and changes no prediction. [D45-1] The block U on the second x86 pair: 500 and 2,000 messages a second, idle, at 80% load, and at 80% load at real-time priority; 50 messages a second idle and at 88% load. [D45-2] What is measured: the consumer's turnaround, the thread's saving and the thread-free latency, the broker's own capture, the card's receive timestamps, and the split. [D45-3] U's predictions. [D45-4] U's decision rules, and what is described. [D45-5] The margins, and the rounds. [D45-6] How it is judged, and what the kit gains. [D45-7] The pilot, and the rule for the load. [D45-8] What was read before this version, and what it informed. [D45-9] The order of the runs.

Version 44 answers what the paper's ninth version left open, and changes no prediction. [D44-1] The block K: each acknowledgment's own arrival, read by the kernel on the producer's socket, beside the delivery's, on the first x86 pair. [D44-2] What is measured, and how the clients' journals are read. [D44-3] K's predictions. [D44-4] The block R: the scheduler's run-to-parity rule on and off. [D44-5] The block P: a delay added on purpose, which the kernel-timed latency must carry one for one. [D44-6] The block L: the kernel-timed latency at 50, 500 and 5,000 messages a second. [D44-7] The block I: idle processors spinning instead of halting. [D44-8] The margins, and why. [D44-9] How it is judged. [D44-10] The pilot, and what it changed in the instruments. [D44-11] What was read before this version, and the order of the runs.

Version 43 tests the full fix as a tool would read it, and changes no prediction. [D43-1] The block F2: the kernel's own timestamps read by the producer and the consumer on their sockets, beside the captures, on the first x86 pair. [D43-2] What is measured, and how the clients read it. [D43-3] Its predictions. [D43-4] Its margins, and why. [D43-5] How it is judged. [D43-6] The pilot, and what it changed in the instruments. [D43-7] The order of the runs.

Version 42 tests a full fix, and changes no prediction. [D42-1] The block F1: both of a message's timestamps taken in the client machine's kernel, against the clients' own, on the first x86 pair. [D42-2] What is measured, and the card's own timestamps beside it, described and not judged. [D42-3] Its predictions. [D42-4] Its margins, and why. [D42-5] How it is judged. [D42-6] The pilot, and what it changed in the instruments. [D42-7] The order of the runs.

Version 41 runs the last tool of the second October reading, and changes no prediction. [D41-1] Google's Cloud Pub/Sub load test framework runs T1 to T4 and S1, four rounds, from the second x86 pair's driver against Google's own Pub/Sub, in a project made for it. [D41-2] How it is run, and what of it is read. [D41-3] Its reference. [D41-4] Its predictions, from its source. [D41-5] S1 on it. [D41-6] The pilot, and what it changed in the instruments. [D41-7] The end of the second October reading's list. [D41-8] The order of the runs.

Version 40 runs two tools built from source, and changes no prediction. [D40-1] Quiver and the Pravega benchmark run T1 to T4, four rounds each, on the second x86 pair: Quiver against the Artemis broker version 39 runs, the Pravega benchmark against Pravega's own server on the broker machine. [D40-2] How each is run, and what of it is read. [D40-3] Their references. [D40-4] Their predictions, from each one's source. [D40-5] S1 on both. [D40-6] The pilot, and what it changed in the instruments. [D40-7] The one tool of the second October reading still unrun. [D40-8] The order of the runs, and the end of the list.

Version 39 runs three tools whose brokers the testbed did not run, and changes no prediction. [D39-1] Artemis's perf client, RocketMQ's benchmark and BookKeeper's benchmark run T1 to T4, four rounds each, on the second x86 pair, each broker on its broker machine. [D39-2] How each is run, and what of it is read. [D39-3] Their references. [D39-4] Their predictions, from each one's source. [D39-5] S1 on Artemis's perf client. [D39-6] The pilot of 11 October, and what it changed in the instruments. [D39-7] The tools of the second October reading still unrun. [D39-8] The order of the runs, and the end of the list.

Version 38 runs four tools of the second October reading, and changes no prediction. [D38-1] mqtt-benchmark, Sarama's kafka-producer-performance, SBK and kgo-repeater run T1 to T4, four rounds each, on the first x86 pair. [D38-2] How each is run, and what of it is read. [D38-3] Their references. [D38-4] Their predictions, from each one's source. [D38-5] S1 on three of them, and its predictions. [D38-6] The pilot of 11 October, and what it changed in the instruments. [D38-7] The tools of the second October reading this version does not run. [D38-8] The order of the runs, and the end of the list.

Version 37 runs PerfKit Benchmarker's messaging benchmark and S1, and changes no prediction. [D37-1] PerfKit runs T1 to T4, four rounds, on the first x86 pair, against an Azure Service Bus queue made for it. [D37-2] Its predictions. [D37-3] S1, the step test: each tool once as it is and once while the wall clock steps back under it. [D37-4] S1's predictions, and how they are judged. [D37-5] The pilot of 11 October, and what it changed in the instruments. [D37-6] The tools of the second October reading this version does not run. [D37-7] The order of the runs, and the end of the list.

Version 36 runs the Yahoo Streaming Benchmark and changes no prediction. [D36-1] YSB runs T1 to T4, four rounds, on the second x86 pair, beside version 35's block. [D36-2] Its servers: the Kafka 3.9.1 broker, because its generator speaks Kafka 0.8.2's protocol, and the broker's Redis. [D36-3] Its reference is itself at no added delay. [D36-4] How it is run, and how it is read: as its own notebook reads it. [D36-5] Its predictions. [D36-6] Its pilot. [D36-7] Why PerfKit Benchmarker is not run in this version. [D36-8] The order of the runs.

Version 35 runs four more message tools and K1, and changes no prediction. [D35-1] Xinfra Monitor, Strimzi Canary, Trogdor and Stream PerfTest run T1 to T4, four rounds each, on the first x86 pair. [D35-2] Their predictions, from each one's source, under T-P7. [D35-3] How each is run, and which of its figures is read. [D35-4] Stream PerfTest runs as a producer and a consumer, the arrangement its source stamps messages for. [D35-5] The canary runs against a Kafka 3.9.1 broker of its own, because its client cannot commit offsets to Kafka 4. [D35-6] K1: Kafka's end-to-end tool, traced, beside an observer of when each record became readable. [D35-7] K1's predictions, and how they are judged. [D35-8] The pilot of 11 October, and what it changed in the instruments. [D35-9] The two tools of that reading this version does not run, and why. [D35-10] The order of the runs and the end of the list.

Version 34 corrects how T2 is judged and changes no prediction. [D34-1] A count short of what a tool was asked to send rules out “times against one clock” as well as the behaviours that keep every value. [D34-2] T2 asks first, from the offset itself, whether the tool’s two timestamps come from two clocks. [D34-3] Every T2 run of both blocks is judged again under both, beside the verdict it was given, and which tools time against one clock is read from D34-2. [D34-4] What stands as a limit of the judgement, and is not corrected.

Version 33 runs the five tools version 15 left unrun and changes no prediction. [D33-1] The OpenMessaging Benchmark, Apache Pulsar’s performance client, emqtt-bench, YCSB and nats bench run T1 to T4, four rounds each, on the first x86 pair, under the predictions fixed for them in versions 3 to 12. [D33-2] Why version 15 left them out, and why the reason it gave did not hold. [D33-3] The two servers they need beside those already on the broker, Mosquitto and Pulsar, installed on it as RabbitMQ and NATS are. [D33-4] How each is run, and which of its figures is read. [D33-5] How each keeps time, read from its invocation and its source before any of its runs. [D33-6] A reference of our own for each. [D33-7] Which clauses of their predictions these runs can reach. [D33-8] The pilot that checked the instruments. [D33-9] The order of the runs and the end of the list.

Version 32 finishes what was started and changes no prediction. [D32-1] The got-it brake records rather than stops in every campaign from here on, as it did for A5’s last runs; each prediction it touches is judged with and without the runs it would have stopped. [D32-2] A campaign that stopped before its runs were all done is finished from its own queue, under its own calibration, on the kernel it ran on. [D32-3] The kit’s two faults that cut runs are corrected. [D32-4] Every run’s machine, kernel and settings are registered, and every fault met is recorded.

Version 31 fixes how M0 is read and changes no prediction. [D31-1] The departure is signed and read on the recorded runs, and an explanation accounts for half of it only by pushing the same way. [D31-2] M-H2’s replay runs each recorded run’s messages through the receive loop’s own rules and costs, never through its outcomes. [D31-3] Both clients switch Nagle’s algorithm off, so only the send window can hold a request for an acknowledgement; M-H4 is run as written, and not for Kafka.

Version 30 corrects the programs that judge and changes no prediction. [D30-1] P3a and P3b compare the lowest and the highest load whose plateau is at least 2%, as D14-2 says; every A3 verdict until now compared 50% load with 88%. [D30-2] P7 is held to P1’s band at each core count, and to nothing else. [D30-3] A campaign that cannot reach what a rule reads is reported out of reach, not as a test that went against the law. [D30-4] A block of several campaigns is resampled campaign by campaign. [D30-5] P6 has a program, and the recordings in half of A1’s and A3’s runs are read against it. [D30-6] How A9 is read is fixed where the plan left it open, and a second reading of A9-2, message by message, stands beside the one version 28 froze. [D30-7] M0’s reading comes in version 31. Every verdict D30-1 to D30-4 touch is judged again, with the reading as it was beside it.

Version 29 builds M0’s recording and runs M0, and changes no prediction. [D29-1] M0 runs on the first x86 pair at 10 rounds, recording in half its runs the packets at both ends, the receive loop’s cycle and every `python3` thread’s scheduling events. [D29-2] A run replaying the football match is held to the plan’s own count and rate in its window, and a Redis setup to its own acknowledgement batch. [D29-3] The corrections of D27-5 come in version 30, with the program that reads A9.

Version 28 adds A9 and changes no prediction. [D28-1] A9 records, on all three pairs, the waits of the one thread that stamps each acknowledgement, event by event, and reads each negative reading against them; the clients name the threads that stamp. How it is read is fixed before its runs. [D28-2] M0, with the recording it never had, and the corrections of D27-5 come in version 29, frozen before M0 runs.

Version 27 adds the runs that could still improve the results before the testbed is taken down, and records what the check of all the data found in the judging programs; it changes no prediction. [D27-1] A2's Redis session on our HZ=1000 build runs its second day again, whole. [D27-2] Each backend gets a second bridge day on the stock kernel. [D27-3] A1 gains a slice of 3.75 ms and one Kafka campaign at 3 and 3.75 ms. [D27-4] A third x86 A3 Kafka campaign runs at 75 and 88% load only, on the second x86 pair. [D27-5] Five places where the judging programs depart from this plan's text are recorded, to be corrected in version 28 with every verdict they touch judged again. [D27-6] Two studies of the mechanism, a trace of the got-it helper's own waits and M0, are frozen in version 28 before they run.

Version 26 records the corrections of 25 September and changes no prediction. [D26-1] A5's session at 2 CPUs finishes its last thirteen runs with the got-it brake recording rather than stopping, after two stops on moves the added delay could not have made; P7 at 2 CPUs is judged on the runs taken under the rule and again on all forty. [D26-2] T2 holds a tool's count against what the tool was asked to send. [D26-3] The Go tools' T2 moves the driver's own clock, puts it back after every run and checks that it is back. [D26-4] The tools block runs its four rounds, each in a folder of its own. [D26-5] Corrections that decide no verdict are recorded.

Version 25 records the corrections of 24 September and changes no prediction. [D25-1] T2's four behaviours are held against the control run, as D23-1 says; the program held them against T1's zero step. [D25-2] T2 allows for each tool's own run-to-run noise, measured from its T1 staircase, judges the one-clock hypothesis as a fifth beside the four, and predicts at the offset measured at the clock. [D25-3] A reading must come from a run that did the work: k6 made no request in any of its fifteen runs, and they are run again. [D25-4] The tools' servers are set up on the broker and checked from the driver, the way the tools reach them, before every stage; the thirty runs that met no server are run again. [D25-5] T1's slope is recorded for every tool and held to the number of times its interval crosses the delayed direction. [D25-6] The Arm pair's A3 is read at its measured loads. [D25-7] Green tags mark only this version's changes. [D25-8] Corrections that decide no verdict are recorded.

Version 24 gives the Arm pair A3's Kafka campaign and changes no prediction. [D24-1] A3's Kafka campaign runs on the Arm pair at 24 rounds, from P3a's simulation at that pair's own spread and levels: confirmed in 88% of campaigns under the law and in 0% of campaigns where it is false. Load has been asked of one processor family only, and the second part of this paper is whether the effect appears on machines it was never seen on. [D24-2] A3's Redis campaign is not run there: the Arm pair's spread pilot reads a plateau of 0.0115 and a floor its own reader marks as below resolution, which is the same shallow curve that puts P3a and P4 out of reach on Redis on the x86 pair. [D24-3] P3b is judged on the same runs and is recorded as underpowered before any of them exist: 76% at the ceiling of 40 rounds where the rule asks 80%, so a non-confirmation there is a limit of this instrument and not evidence against the prediction.

Version 23 adds an addendum and one run to each T2 session, and changes no prediction. [D23-1] A T2 session runs three times rather than two: once at no offset and once at each of its two offsets, with the offset actually achieved measured at the clock and written beside every run. Where the clock moved and the tool's figures did not, the tool times against one clock and no value below zero can arise in it — a verdict rather than the undecided this produced before, and the one the artifact's own premise predicts. [D23-2] rdkafka_performance, the only tool arranged across two clocks, has that arrangement established by a shakedown before anything of it is measured: its consumer read none of its producer's messages over a whole staircase, and a tool that measures nothing must not be reported as a tool that measured zero. The addendum at the end sets out both, with how each of the eleven tools keeps time.

Version 22 reads one gate across the staircase instead of step by step and changes no prediction. [D22-1] A gross departure of the "got it" note is the added delay reaching it, and that is a shift which grows with the delay, so it is read as the slope of the shift against the delay and fails the gate above a quarter. Step by step, the same quarter could not tell leakage from drift: a constant drift fails at every step below four times its size and passes above, so the second x86 pair's Redis ended a sound session on +0.342 ms at 1 ms added while its Kafka passed on −0.315 at 4 ms and −0.282 at 8. Their slopes are −0.010 and −0.038. A shift larger than the whole delay added is still gross at any one step on its own, and the slope is written out beside the gate.

Version 21 moves A2's Kafka half to the first x86 pair and changes no prediction. [D21-1] The six Kafka sessions and the Kafka bridge run on the first x86 pair; the second keeps Redis, A5's two remaining sessions and the Redis bridge. Nothing is pooled across pairs or backends in any judgement here, so each backend's comparison of the three kernels stays whole on the machine that ran it, with each kernel on two days as [D4-7] asks. The first pair has been idle-bound behind the tools and the second was carrying about forty-five hours alone; this has the last measurement land about nineteen hours earlier, and the Kafka session the second pair already ran becomes a replicate of that cell on a second machine. [D21-2] A prediction is judged on the runs the integrity rule passed, and the answer names the ones it left out. The reader that feeds every judge kept anything with a trip and a negative rate; a stopped run is one whose own instrument was in doubt and a repeated one was run again, and `run_integrity.py` has always refused to compare a run against such neighbours. Five runs of 3,179 are affected: A3's P3a and P3b reproduce to four decimals without them and A8's Python-to-Java ratio moves from 3.12 to 3.25, not confirmed either way.

Version 20 widens two brakes and changes no prediction. Both are widened because they stopped work that was not wrong, and neither changes what is written down about any run. [D20-1] The calibration's precision bound follows the tick: the larger of 0.30 ms and 0.15 of it, where it was a flat 0.30 set on a 1 ms tick. A2 boots kernels whose tick is 4 and 10 ms, and a calibration cannot be known to a fraction of the quantum its trips are quantised by. Kafka on the second x86 pair came in at 0.410 at HZ=250 and 0.387 at HZ=100 against Redis's 0.128 to 0.237 on the same boots, and four rounds did not move it. The bound each session used is written beside its gate, and the check is no longer called `known_within_0_3_ms`, which would now be a name that lies. [D20-2] The got-it brake's floor goes from 0.10 ms to 0.25. A8's x86 campaign stopped at 86 of 144 on a move of 0.134 ms where its own pooled scatter allowed 0.132; the floor is what stops that allowance collapsing onto a very tight scatter, and at 0.10 it sat below the drift sound campaigns show — 0.124 ms on the Arm pair, 0.134 here. It is still a brake: a campaign that moves 0.4 ms at a 0.5 ms delay still stops. [D20-3] A campaign is gated on the calibration it is placed from rather than on every one in the file; until today a Redis campaign was refused for the state of Kafka's fit, which cost three sessions and about seven and a half hours of one pair.

Version 19 corrects a program and no prediction. P8 says that Java shows the same cliff *and priority effect* as Python, and that Python's rate is at most 1.5 times Java's. Three clauses; the judge tested two. It printed the rule it was applying, and the rule it printed named the cliff and the ratio — while the function's own documentation said it tested the priority effect. Nothing about that is visible in a verdict: a judge that omits a clause a campaign passes returns exactly what a complete one would. [D19-1] Go-first's cut of Java's plateau is now computed at the trips both clients ran, gated at the five times this plan has asked for since version 8, and recorded beside the verdict. A8's Arm campaign has been judged again under all three clauses and returns the same answer — not confirmed, on the ratio, at 4.33 where 1.5 is allowed — with a cut of 469.7. [D19-2] The ten tool campaigns run on the first x86 pair rather than five on each, which changes no rule: each tool still runs entirely on one pair, and that is what this plan requires of them. This is the third program in two days found to report on less than it claimed, after the two of 21 September, and the three share a shape: each was caught by asking a judge what it actually did rather than reading what it said.

Version 18 widens one brake, after that brake stopped a campaign. The order of events deserves suspicion, so here is the argument that does not depend on it: version 17 allows a run three times the scatter of its own setup's earlier runs, and lets the brake judge as soon as two of them exist. A standard deviation from two points has one degree of freedom. It is not an estimate of spread, and version 17 guarantees that the first braked run of every setup is measured against one. A2's first session, ten setups over 28 counted runs, gave within-setup scatters of 0.053, 0.055, 0.082, 0.082, 0.116, 0.127, 0.177, 0.185, 0.224 and 0.227 ms — one common quantity seen through two or three points each, pooling to 0.133 — and it was the 0.053 that stopped the campaign, on a run 0.275 ms from its setup's centre. Three times the pooled figure is 0.398. [D18-1] The scatter is now pooled over the campaign's own setups, from each run's distance to its own setup's centre; the centre a run is held against does not change. [D18-2] The brake waits until that pool has five degrees of freedom, and until then a run is recorded rather than braked. [D18-3] The pooled scatter and its degrees of freedom are written beside every run, so the allowance can be recomputed by anyone who doubts it. The brake is not switched off: a run a whole millisecond from its setup's centre still stops this campaign. And nothing is cut — a run the brake cannot yet judge still counts, with its got-it beside it.

Version 17 corrects one brake and no prediction, and it is the same brake version 16 corrected the day before. Version 16 settled *which* runs it may judge. This settles *what it judges them against.* The brake held each run's "got it" median against the session's calibration — a different sitting, hours earlier — and allowed it a quarter of the delay that run added. Those are two different quantities: what separates a campaign from its calibration is drift, which does not grow with the delay, and the allowance does. A3's Kafka campaign drifted 0.022 ms from its calibration over 216 judged comparisons and A8's drifted 0.124 over 14, on noise floors of 0.170 and 0.199 — near-equal floors, a six-fold difference in drift — and the brake bit at exactly one rung: the shortest delay of the client that needs the least delay, where its allowance falls to 0.221 ms. [D17-1] A run is now held against its own campaign's earlier runs of the same setup, which shares its client, its note's place, its slice and its delay, so no drift enters the comparison; until a setup has two such runs its got-it is recorded and not braked. [D17-2] The allowance keeps its shape and its quarter-share, now measured against those runs. [D17-3] The distance from the session's calibration is still computed and written beside every run, and no longer stops anything; P5(c) stays where this plan has always put it, in the session's own calibration, which varies the added delay with everything else fixed and is the experiment P5(c) describes. The brake is not loosened: checked against A8's own runs, a campaign that broke mid-run would still stop.

Version 16 corrects one brake and no prediction. P5(c) asks whether the added delay moved the "got it" time, and the brake that enforces it compared every run against the session's calibration. A8 takes that note two ways, and taking it inline forces the producer to one message in flight where the calibration ran with sixty-four, so the comparison asked what A8's own treatment did rather than what the delay did. [D16-1] A run whose note is taken a way the calibration never took it is now recorded rather than braked: its got-it median and where the note was taken travel with it, and P5(c) is asked of those runs by comparing the ones that share a note's place across the delays they ran at. [D16-2] And a campaign counts, from its own queue before anything runs, how many of its runs the brake can judge and how many it cannot, because on the Arm pair that was found on the third run and three hours in. [D16-3]

Version 15 changes the tools block and no prediction of the law. T1 and T2 were run against made-up tools before any real one, to find out what they could answer. T1 can answer: it reports the chopping class at 0.9 ms where every other class reports 0.1 ms. T2, as frozen, cannot. Its smaller offset makes no negative at all on a 3 ms trip, and its larger one makes none that a whole-millisecond clock can hold. [D15-1] The offsets are now taken from the session's own median trip, and what each behaviour would have given is worked out through the step T1 measured for that tool, because that is what the tool's rule for odd values actually sees. [D15-2] The verdict names what survives being ruled out rather than what is nearest: on a tool whose clock is coarser than the gap between two behaviours the nearest is decided by the rounding, and a run leaving two standing has not answered. [D15-3] The identification is read where sends are spread evenly; sends paced in step with the tick are T-P2's evidence and not an identification. [D15-4] Each figure a tool prints now carries the step its own printing can express, because two of these tools print an exact average beside percentiles chopped to whole milliseconds, and one step for the tool would lose the thing T-P1 and T-P2 predict. [D15-5]

Version 14 is the first written after a campaign was run and read. A3's Kafka campaign confirms that load raises the plateau, and leaves P3a unresolved – not for want of rounds but because at 50% load the negative rate is 0.15%, and no cliff can be located in a curve that shallow. Neither more rounds nor more messages narrow it, and both were checked against that campaign's own runs. [D14-3] P3a is now judged where the ordinary plateau is at least 2%, the guard P4 has carried since version 4; simulated at A3's measured levels that lifts it from 9% power to 98%, with false confirmations at 1%. [D14-2] A load below the bar is reported as out of reach, as an unreachable slice already is. [D14-3] The rounds rule is simulated at the plateau and floor the pilot measures rather than at 0.30 and 0.01. [D14-1] The world those simulations draw on now counts negatives out of a run's messages and moves the cliff between runs, because one spread multiplied onto a rate can represent neither. [D14-4]

Version 13 corrects the instrument and no prediction. The rule judging P8 read the middle of the cliff from $s + 0.4h$; A8 runs $s + 0.5h$, as P8 has said since version 8, so the rule asked for a trip its own campaign never runs and no result could have satisfied it. It confirmed in 0% of simulated campaigns at every number of rounds, which the rounds rule reported as forty rounds and underpowered. Corrected, P8 asks 6. [D13-1] That rule now separates one that never confirms from one merely weak. [D13-2] P8's ratio is taken only where both clients ran, and says whether the campaign tested P8 at all. [D13-3] Both clients now measure their clock and refuse one coarser than 25 μs. [D13-4]

Version 11 finishes what A5's sessions need. The plan gives each core count its own session with its own calibration, because the delay's effect on the trip has to be measured on the machine as the campaign will run it. The baseline is the other half of that same instrument: it is the zero-delay trip every target trip is placed from, and it moves with the core count for the same reasons the calibration does. The plan named the calibration and said nothing about the baseline, so a session at two CPUs would have placed its trips from a floor measured at eight. Each A5 session now measures its own baseline, at the load its campaign runs, before its calibration. [D11-1]

Version 10 changes no rule. It records that one was misapplied and puts it right. Rounds are set by simulating a campaign at the run-to-run spread its pair's spread pilot measured, and the plan has said since version 4 that the figure is the one measured *on that pair for that backend*. In practice one figure per pair was computed, the median over every setup the pilot ran with both backends mixed together, and that is what set the rounds for A1, A4 and A5's first simulation. Pooling two variances into one is allowed only where the two populations may be assumed to share a variance, and this instrument's own pilots refute that assumption: on the second x86 pair the spread is 0.213 for Kafka and 0.701 for Redis, a factor of 3.3 in the scatter and about 11 in the variance. Their pooled median, 0.457, estimates neither. The harm is not only waste. At the pooled figure A5 would run 8 rounds and its log would claim 86% power; simulated at the spread Redis actually has, the same campaign reaches about 64%, below the 80% the rule demands. Pooling spends too many runs on the quiet backend and certifies the noisy one as powered when it is not. [D10-1]

Version 9 splits P3 in two and states its first half the way a claim that something did *not* change has to be stated. P3 asked that load raise the plateau *and* leave the cliff where it is, and a prediction is confirmed here only where every part of it is. Simulated at the first pair's own spread, the cliff half held in 97% of campaigns under the law and the plateau half in 20%, so P3 held in 18% — and the plateau half held in 20% of campaigns where the law is false as well, because P3's falsifier moves the cliff and not the plateau. Joined together, one half could only subtract from the other. They are now P3a and P3b, each with its own falsifier. P3a is also restated: the whole interval of the cliff's movement must lie inside the band, not merely the number in the middle of it, which a scattered estimate can put inside by luck. [D9-1]

Version 7 corrects the instrument that judges the delay. Version 6's first shakedown turned a sound pair away on a reading taken by ping, and ping is not the path our messages take: measured side by side, it reads about half a millisecond high, with a tail several times longer, and it shows a difference between the two paths that TCP does not. Every judgement now rests on the broker's own capture, which times the delay where it is added; ping and the paths are recorded and decide nothing. No prediction, and no rule that decides one, changes. [D7-1]

Pocket dictionary

Run	one measurement, a few minutes of traffic	**Round**	one pass through every setup of a campaign once
Setup	one combination of settings being tested	**Session**	one machine pair, booted one way, at one core count, on one day
Campaign	runs that answer one question in one sitting, with the machines stopped before and after	**Machine pair**	a driver machine and a broker machine used together
Time slice (s)	how long the scheduler lets a helper run before switching; 2.8 ms on the first Azure machine	**Tick** (h)	the kernel's heartbeat; 1 ms if HZ=1000, 4 ms if HZ=250, 10 ms if HZ=100
Trip (D)	the real time from sending to arrival	**Plateau, cliff, floor**	the three parts of the predicted curve (Section 2)
Anchor slice	the 3 ms slice, included in every slice campaign so that drift between days can be measured and removed	**Simulated power**	the share of made-up campaigns, generated from the law, in which the decision rule confirms the prediction
Go-first priority	a setting that makes a helper jump the queue *(real-time priority)*	**Receiver-only delay**	extra delay added only on the receiving program's connection
Clock offset	making one program's clock read a known amount early or late *(libfaketime)*	**Precision class**	low = rounds to milliseconds, medium = microseconds, high = nanoseconds
Retention	the share of results a tool keeps	**p50, p99**	the median, and the value 99% of results fall below
Pre-register	write the predictions down and freeze them before running	**Power**	the chance of detecting an effect if it is really there
Calibration	measuring how much a known change moves our reading	**Equivalence test**	a test that two things are the same within a stated margin *(two one-sided tests)*
Manipulation check	confirming that the setting we changed really changed	**Bridge session**	a short repeat that ties new results to earlier ones
Packet capture	a record of network packets with their arrival times	**Counterbalanced**	arranged so that each kernel comes first, middle and last on different days
Lane	one machine pair with its own queues and logs; two lanes never share a machine	**Verdict**	what a run's checks decide as it ends: it counts, it is repeated, or it stops its campaign
Stop rule	a reason for a campaign to stop itself and wait for a person	**Shakedown**	the instrument checks a new machine pair passes before its first campaign
Fingerprint	a SHA-256 code computed from a file; any change to the file changes it	**Ledger**	the list of a campaign's runs with what happened to each, failures included
Void session	a session broken by a fault: kept and reported, never analysed	**Batched acknowledgement**	the receiving client confirming many messages in one round trip, instead of one round trip per message
95% interval	a range built so that 95 of 100 such ranges would hold the true value	**Bursty traffic**	messages sent in clusters, as in the football match, instead of evenly
Ping *(ICMP)*	a message type made for testing reachability, which machines may handle differently from ordinary traffic	**sockperf**	a tool that measures round trips with TCP or UDP, the traffic applications use

What changed since version 47, and why

Pilot round a of version 47 ran block C's three arms as that version placed them, and showed that the placement cannot hold on this testbed. This version moves the card's interrupts, and the consumer's other threads, off the vCPU where the consumer polls, and runs block C's own pilot before any of its judged runs. It changes no prediction and no margin (`docs/results/general_method/DESIGN48.md`). No judged run of block C had begun.

No.	Change	Reason	Prompted by results?
D48-1	Block C's placement. In all three arms the card's interrupts go to the chosen core's other hardware thread (`irq_cpus: sibling`), with irqbalance and receive packet steering off as before. In the two arms that keep the core, the consumer's main thread, the one that polls, stays alone on the core's first thread; every other thread of the consumer moves onto the core's other thread as soon as it appears (`threads_apart`, `scripts/thread_apart.py`), kafka-python's heartbeat thread among them; every other process of the run stays off the core; the load runs on the six other vCPUs, and in the sibling-loaded arm on the core's other thread as well. The baseline arm keeps its consumer unpinned, with the interrupts placed as in the other two. C-a, its margin and the arms' loads (70%, 53% and 61% of the machine) do not change.	One card carries both clients' traffic and three captures, so version 47's "receiver's interrupts on the isolated vCPU" put every network interrupt of the machine there: 186,702 to 430,424 a run, every NET_RX softirq with them. The consumer at real-time priority alone on that vCPU then ran at the throttle's 0.95, counting the receive path's work, which this kernel charges to the task it interrupts. In one pilot run its producer fell 388 ms behind; in the other the consumer read nothing, its heartbeat thread never ran, and the broker removed it from its group after 10 s. On the core's other thread, the receive path's work is the same in both load states, so it cancels in the residual C-a reads within each run.	**Yes**: pilot round a's block C runs (D48-4).

No.	Change	Reason	Prompted by results?
D48-2	Block C's pilot, after this version is public and before any judged run of block C: one run of each arm at the starting duty of 75 (`gpilot_c48_design.json`). Before the judged runs the record states each arm's frozen duty, corrected as D47-11 corrects it, and C-a's power by version 45's rule; and, in each arm, that at least 99.9% of each card interrupt's count and at least 99% of the NET_RX softirqs ran on the core's other thread, receive steering was off and irqbalance stopped during the run and restored after; in the arms that keep the core, that the main thread was seen only on its own vCPU and every other consumer thread ended on the core's other thread, both clients at SCHED_FIFO 80, and every other process of the run off the core; that the consumer's main thread used at most 0.70 of its vCPU over the run's middle half; that the broker's log shows no member removed from the group and no rebalance after the first send; and that the integrity and lateness rules hold. An arm that still fails one of these once its duty is found is left out, with the reason, and C-a is then out of reach.	The new placement has not run on the testbed.	No.
D48-3	The order: block C's pilot and judged runs come after blocks V, N, P, K and S on the x86 pair.	Version 47 ran block C before block S; block C now waits for its own pilot.	No.
D48-4	What was read before this version: pilot round a's block C runs, for their interrupt counts, their consumers' scheduler counters and their brokers' logs (`docs/results/general_method/README.md`); and the placement, read against how the kernel runs real-time tasks and its receive path.	These readings are the results this version follows; no judged run of block C had begun.	**Yes**: they are the results this version follows.

What changed since version 46, and why

Version 46's block U showed the mechanism: under load the consumer's thread is slow to ask again for messages, a message that reaches the broker while no request waits there sits until the next request comes, and the thread's delay enters even the kernel-timed latency. Read after its verdicts, the same runs showed that a client can tell which of its messages found no request waiting from its own timestamps, best from one timestamp per packet. This version tests a general method built on that, in the form the author set on 23 October 2026, in new blocks of its own; it changes no earlier prediction (`docs/results/general_method/DESIGN47.md`).

No.	Change	Reason	Prompted by results?
D47-1	The method under test, in four parts. Kernel timestamps on the client machine for every packet, from the sockets where the language allows and otherwise from a packet capture on the client host, with no change to the client; messages matched by the broker's own ids (Kafka's offsets, Redis's entry ids, NATS JetStream's stream sequence; AMQP's message-id property where a classic queue gives no id both ends see). A check any tool can run from those timestamps: was a request waiting at the broker when each message arrived, the share that found none reported beside the latency. Three latencies, each named: what the broker and network deliver to a ready receiver (F), what reaches this client's kernel (W), what reaches the application (D). A remedy that depends on the protocol: Redis, NATS and brokers that push with credit can keep requests waiting; Kafka can only shorten the gap with a fetch thread that does nothing else, and its round trip is real latency.	Block U showed the mechanism in its own clients; a method a benchmark can use must work beneath any client, say which of its messages carry the client's delay, and name what it measures.	**Yes**: block U's verdicts (`a1a94091`) and its reading after them (D47-12).
D47-2	W from the client host's captures, each message named by the broker's id (`scripts/client_wire.py`): the capture time of the packet carrying its record's last byte as it left, and the time the last byte of its record in the delivery could first be read. The sockets' W_{so} is described beside it for the Python clients. D from the clients' own files, joined to the wire by the broker's id. The timing claim rests on the card: per packet, Q, the kernel's receive timestamp on the client host's capture less the card's hardware timestamp of the same packet, on the system's clock.	Read after block U's verdicts: matching by the broker's ids paired exactly the messages the payload's event ids pair, on eight runs; the sockets' and the capture's receive timestamps are the same kernel value, so their agreement tests the matching, not the clock. Per packet, block U's kernel timestamp trailed the card's by 18–20 μs at the 99th percentile on Redis and idle Kafka and 82–99 μs on loaded Kafka, never earlier.	**Yes**: as D47-1.

No.	Change	Reason	Prompted by results?
D47-3	The check, to a resolution of 50 μs. The broker's record, from its own capture matched by the broker's id: the delivering request (for push with credit, the acknowledgment that freed the message's credit; for NATS, the oldest pull request still outstanding, as the server serves them first in, first out) reached the broker more than 50 μs after the message's acknowledgment left it (none), before the message arrived (waiting), or in between (close). The certain reading: the request left the consumer more than 50 μs after the producer had the acknowledgment. The estimate: the same with the pair of connections' two crossings $\hat{C}$ added back, the least acknowledgment round trip on the producer's connection plus the least round trip on the consumer's connection of a request that cannot have waited (one that asks for no messages, or whose answer carried a message stored before it left) less the least W over the pair, less an allowance per broker set from the pilot's broker captures; each least value from at least 100 values, or no estimate; a probe's reply cannot have waited (the NATS consumer sends PING on its receive connection every 100 ms, the RabbitMQ consumer a synchronous method on a second channel). The certain reading of waiting, in counting form: the message certainly found a request waiting where, on its delivering connection, the requests that had left by B before it left, less the answers to them that had arrived by B after its acknowledgment arrived, number at least one; for push with credit, where the deliveries received by B after the acknowledgment less those acknowledged by acknowledgments sent by B before it left are fewer than the prefetch. $B = 10$ ms bounds every one-way delay of the delivering connection, checked in every run (no retransmission or zero window, the largest probe round trip below B, the largest per-packet Q below half of B, no capture silence longer than B while the measured messages are sent, from the warm-up's end to the plan's last send); $B = 2$ ms is described beside it. The check is read first from the client host's captures.	Read after block U's verdicts: the certain reading made no false call in 3.02 million messages; where the remedy works, no request can be shown not to have waited, so only a causal reading of waiting can confirm it from the client's side; the estimate agreed with the broker's record on 96.9–99.2% of messages on Kafka at 2,000 a second from the per-packet captures and on only 86.6–98.2% from the sockets' per-read timestamps; taking the floor from every request failed at low rates, where every request waits; many messages sat within a few microseconds of their request.	**Yes**: as D47-1.

No.	Change	Reason	Prompted by results?
D47-4	F, as ready as the protocol allows: where requests or credit can wait (Redis, NATS, RabbitMQ), $E_3 = W - S$, with S the time the message sat stored at the broker before its request came; on Kafka, $E_2 = W - \min(T_c, S)$, the latency had the client asked again at once, its round trip kept. Each from the broker's capture and from the clients' timestamps alone with the estimate. T_c as version 46 defines it, from the client host's capture.	A ready receiver is one with a request already waiting where the protocol allows it; Kafka's protocol does not, so its readiest receiver asks again at once.	No.
D47-5	Every run: 2,000 messages a second with Poisson sends, 70 s with a 20-s warm-up; the load at 70% switched on and off in 5-s blocks, a message sent within 1 s after a switch left out of either state, at a duty frozen per setup from the pilot; the producer at real-time priority; the clients' collectors off where the client has one; every capture block U took; the run's facts and each broker's image digest and version; each campaign primed on its broker; version 46's integrity rule with its lateness rule and each new client's settings. Per load state, a quantity's tail mean is its mean over that state's messages above the state's 99th percentile of W; a run's on/off ratio is the on state's over the off state's.	Block U's sends were evenly spaced, so a comparison needs its own baseline here; within one run the connections, and so their path delays, are the same while the load is on and off.	**Yes**: block U's design and its reading after the verdicts.
D47-6	The brokers, pinned: Redis 7.4.11 by image digest; RabbitMQ 4.3.7 by image digest; NATS 2.15.1, by the release's checksums, with JetStream on memory storage; Kafka 4.1.1. Redis and NATS keep 256 requests waiting (NATS's `max_waiting` 1,024).	The images were floating tags and no run recorded a broker's version; at 2,000 a second 64 requests cover 32 ms where block U held one consumer for 997 ms; NATS's file store syncs every two minutes, where Kafka here writes to the page cache and Redis runs without persistence.	**Yes**: the reading of the draft against the brokers' and clients' sources (D47-12).

No.	Change	Reason	Prompted by results?
D47-7	Seven blocks, four rounds of each setup in random order within each round, seed 20261104: V, Redis with our waiting consumer at depth 1 (each replacement sent before the reply is read), at depth 256 with one reply per read, and at depth 256 with the library's own reads; N, NATS with our pull consumer at depth 1 and at depth 256, and nats.go's `Consume()` with its own defaults; P, RabbitMQ at prefetch 1 and 1,000; K, Kafka with kafka-python 2.3.2, Java kafka-clients 3.9.0 (classic), franz-go 1.19.5 and librdkafka 2.15.1 at normal priority, and librdkafka with only its broker thread at real-time priority; C, kafka-python at real-time priority with irqbalance off, the receiver's interrupts on one vCPU and receive packet steering off, not isolated, alone on that vCPU's core with its sibling idle, and with its sibling loaded; S, scope probes, Redis at depth 256 with the producer at normal priority and under a loopback UDP load pinned to the receiver queue's vCPU, described only; A, on the Arm pair, V at depth 1 and 256 and librdkafka at normal priority and with its broker thread at real-time priority. 72 runs on the x86 pair and 16 on the Arm pair (`g<block>_design.json`).	Each block tests one part of the method beneath the clients a benchmark uses: the remedy where the protocol allows it, Kafka's own limit, what real-time priority leaves, and where the method's assumptions end.	**Yes**: as D47-1 and D47-6.

No.	Change	Reason	Prompted by results?
D47-8	Predictions on the timestamps, the ids and the check. M1-a: in every run with the card captured, Q's 99th percentile at most 150 μs, and no Q below minus the card's clock window. M1-b: matching by the broker's ids pairs every message the payload's event ids pair, none differently. M1-c (V and N at depth 256): with one reply per read, W_{so} within 10 μs of the capture's W at the 99th percentile; with the library's own reads under load, at least 100 μs above it there. M2-a: in no run does the certain reading call a message unwaited that the broker's record shows waiting or close. M2-b: in every run where the estimate has a value, its share of messages that found none within 6 points of the broker's. M2-c (V and N at depth 256, nats.go, P at 1,000): the broker's record shows a request or credit waiting for at least 99.9% of messages, and the certain reading of waiting certifies at least 99.9% of them, out of reach in a run whose check of B fails; the estimate's share reported where it has a value.	Each part of the method's first two points is judged where it can fail: the clock against the card, the ids against the payload, the per-read socket against the per-packet capture, the check against the broker's own record; the remedy's own condition is checked before its effect is judged. M2-b's margin is block U's largest error read after its verdicts, rounded up.	**Yes**: as D47-2 and D47-3.

No.	Change	Reason	Prompted by results?
D47-9	Predictions on the latencies and the remedy. M3-a: where M2-c holds, over the messages the broker's record shows found a request or credit waiting, W's tail on/off ratio lies within 1/1.2 and 1.2 by two one-sided tests across the setup's runs at $\alpha = 0.05$, while D's on/off ratio is at least 1.5; over all messages, described. M3-b: at depth 1, at prefetch 1 and for each in-poll Kafka client at normal priority, W's tail on/off ratio at least 1.5, and where it is, removing the client's part of the request gap (E_2, with the credit turnaround for P) takes out at least half of its rise, a run-level 95% interval from 2,000 draws beside it. V-a, N-a, P-a: M3-a on Redis at depth 256, on NATS at depth 256 and with nats.go, on RabbitMQ at 1,000. KC-a: each in-poll Kafka client's turnaround tail mean at least 1.5 times higher in the on-blocks. KC-b: as M3-b, on Kafka. KC-c: librdkafka's rise of the turnaround's tail mean at most half of each in-poll client's. KC-d: librdkafka with only its broker thread at real-time priority, W's tail held still as M3-a reads it over all messages, while D's on/off ratio is at least 1.5. KC-h: in that setup, the turnaround's tail on/off ratio at most 1.2, and in both load states at least half of the messages found no fetch waiting, by the broker's record; the share of time no fetch waited at the broker and $\lambda g/(\lambda g + e^{-\lambda g})$ described. C-a: the residual (W's on-block tail mean less its off-block one, within runs) with the consumer alone on its core and its sibling idle at most a third of the baseline arm's; the sibling-loaded arm described. A: V-a, M2-c (Redis) and KC-d hold on the Arm pair. S described only.	The method's third and fourth points, each where it can fail: the kernel-timed latency held still where the protocol keeps a request waiting while the application's grows; the request gap carrying the rise where it does not; Kafka's dedicated fetch thread holding, and its round trip leaving messages waiting even so. The formula is described, not judged: it assumes a constant gap and no broker handling, and librdkafka sends its next fetch after parsing the answer.	**Yes**: as D47-6.
D47-10	The judging. A cell is one setup, or one comparison of two setups, in one block. A prediction holds where every cell it reaches holds; a cell missing a value it needs is out of reach, reported and not judged; a clause that fails fails its cell. A setup's value is the median over its passing runs, except where a line says a mean. Read by `scripts/general_method.py`.	Version 46's rules, carried to the new blocks.	No.

No.	Change	Reason	Prompted by results?
D47-11	Pilot round a, after this version is public and before any judged run, instruments only and never judged: one run of each setup on each pair at a starting duty of 75 (`gpilot_x86_design.json`, `gpilot_arm_design.json`). It checks the integrity and lateness rules; that every message is found in the broker's and the card's captures by its id with no packet dropped; each broker's digest and version as pinned; that the Go and Java consumers build on the driver and their thread names; that Redis serves a queued request in the same pass, NATS keeps 256 requests waiting and RabbitMQ a credit at 1,000; and the whole-core arms' masks, interrupt placement and irqbalance's state. Before any judged run the record states and the push makes public: each setup's frozen duty, corrected by the on-blocks' miss where they miss 70% by more than 3 points; the allowance per broker from the pilot's broker captures; and the rounds, simulated from the pilot by version 45's rule, a clause whose pilot effect does not clear its margin resting on the floor of four with its power stated. A setup whose client cannot run as designed is left out, with the reason, before any judged run; no margin changes.	The new clients, brokers and placements have not run on the testbed.	No.
D47-12	What was read before this version: block U's 64 judged runs and its pilot round d, read again after the verdicts by `scripts/under_load_posthoc.py` (`docs/results/under_load/README.md`, "Read again after the verdicts"), every number in D47-2, D47-3 and D47-8 among them; the client captures of eight of its runs, read by `scripts/client_wire.py`; and the draft of this version, read against the brokers' and clients' sources, a reading that changed D47-5, D47-6, D47-7 and D47-9. The order of the runs: on the x86 pair from its own work list, pilot round a's campaigns, then, once D47-11's statements are public, blocks V, N, P, K, C and S, each campaign copied home and checked when it ends; on the Arm pair, its pilot, then block A.	These readings are the results this version follows; no run it covers had begun.	**Yes**: they are the results this version follows.

What changed since version 45, and why

Version 45's pilot ran three rounds and passed its instrument checks. A check of the design against the pilot's own runs, read after the pilot and before any judged run, then found that several quantities the predictions read would not mean what their names say: under load the producer, not only the consumer, fell behind; the clients paused for their own collectors; the first run after a broker started was unlike the rest; the real-time setups ran a heavier load; Redis's acknowledgments fell outside the turnaround; each TCP connection kept its own network delay; and U-c's share was a difference of percentiles. This version amends block U before any of its judged runs, under its own pilot round, and changes no margin (`docs/results/under_load/DESIGN46.md`).

No.	Change	Reason	Prompted by results?
D46-1	The producer runs at real-time priority (SCHED_FIFO 80) in every setup, so only the consumer's priority varies between the loaded setups, and the real-time setup is the consumer-alone control. The integrity rule gains a lateness rule: a message's lateness is its send time less its planned time, and over the messages sent after the warm-up, its spread between its 1st and 99th percentiles at most 10 ms; a run that fails it is repeated.	Under 70% load at normal priority, Redis's producer at 2,000 messages a second sent a median of 24 ms late (272 ms at the 99th percentile, 84% of messages over 1 ms late), in runs that passed because the rule checked only the mean rate. Its sends came in bursts, and the consumer made about 36,000 requests against 88,000 idle: the load changed what the broker received, not only the consumer. Block L's 88% Redis runs, which set U-c's Redis margins, were 0.5 and 0.9 s late at the median.	**Yes**: version 45's pilot, read before any judged run (D46-12).
D46-2	Both clients turn their collector off once connected (`gc.freeze(); gc.disable()`) and log any collection that still happens; two runs at 2,000 messages a second idle keep it on and log every collection (`ug2000_design.json`), unjudged and not read by the block's reader.	In every setup the clients paused 17–84 ms at about 24, 34, 46 and 60 s after the first send, each pause longer than the last; leaving out the 0.25–0.53% of messages delivered after a turnaround over 10 ms lowered W_{so}'s 99th percentile by 4–16% and moved U-d's Redis ratio from 0.53 to 0.48. The pattern fits CPython's full collections over the ∼140,000 per-message objects the clients keep.	**Yes**: as D46-1.
D46-3	Each campaign starts with an unjudged, unread 60-s priming run on each broker (`PRIME_S=60`).	In the run during which Kafka created its `__consumer_offsets` topic, the first after the broker started, 1.42% of messages were acknowledged after more than 1 ms, against 0.10–0.36% in every other Kafka run at 2,000; that run alone moved U-c's Kafka-2,000 share from 0.83 to 1.00. Every session recreates both brokers.	**Yes**: as D46-1.
D46-4	Each loaded setup's stress-ng duty is frozen from pilot round d, the duty its run used with the second correction, and judged runs run open-loop at it; the integrity rule still holds the measured background within 3 points of the design's load. The paper compares the consumer's two priorities at equal measured background, and says so.	The second correction set the real-time setups 4–5 points higher (76–77 against 72 at 2,000); its law assumed one point of duty moves the measured background by one, where the pilot's moved it 0.60–1.04, and real-time Kafka at 2,000 landed half a point above the floor.	**Yes**: as D46-1.

No.	Change	Reason	Prompted by results?
D46-5	The turnaround of a request for messages k runs from $t_{\text{ask}}(k)$, the receive timestamp of the answer to the previous request for messages on its connection, less $X(k)$, the time within the gap that the connection's other requests spent waiting on the broker, counted once where they overlap: $T_c(k) = t_{\text{req}}(k) - t_{\text{ask}}(k) - X(k)$, zero where negative. The saving's bounds and $F = W_{\text{so}} - s_{\min}$ take this T_c; version 45's turnaround, from the answer to whatever request came last, is described beside it.	On Redis the consumer acknowledges every 200 messages with a blocking XACK on its connection, and version 45 timed the next turnaround from that acknowledgment's answer: the messages that arrived during the cycle were 2.5–3% of all under load but 85–95% of F's top 1%, and the split put their wait in the broker's term.	**Yes**: as D46-1.
D46-6	The split, on one identity: $W_{\text{so}} = \mathit{floor} + \mathit{excess} + \mathit{hand} + b_{\min} + \mathit{ack} + \mathit{wait} + Q$. With $S = \max(0, F_{\text{in}} - A_{\text{out}})$, the time the message lay stored and unasked-for, $b_{\min} = \min(T_c, S)$ is the broker's bound on the saving; $\mathit{hand} = R - S$ the broker's handling; $\mathit{ack} = \min(X, S - b_{\min})$ what the consumer's other requests kept it waiting; $\mathit{wait} = S - b_{\min} - \mathit{ack}$ the rest. The floor is the message's connection pair's (the producer's connection that carried it, the consumer's that delivered it): the 1st percentile of $W_{\text{so}} - R - Q$ over the pair's messages in the run, or the least of them where the pair carried fewer than 100; *excess* is the path's term above it. So $G = W_{\text{so}} - b_{\min} - \mathit{floor}$ splits exactly into $\mathit{excess} + \mathit{hand} + \mathit{ack} + \mathit{wait} + Q$; version 45's split is described run by run.	Each TCP connection keeps its own network delay: within one Redis run the 16 producer connections sat near 330, 500 or 590 μs, each level only 10–20 μs wide, and each Kafka run, on one connection each way, drew one level, from 328 to 655 μs. $s_{\min}$ subtracts the request's and the acknowledgment's network crossings, so the broker's bound is always at least as tight; $s_{\min}$ was exact for 2 of 80,174 flagged messages.	**Yes**: as D46-1.
D46-7	U-c's share is $1 - (F_{\text{load}} - F_{\text{idle}})/(W_{\text{so,load}} - W_{\text{so,idle}})$ with each run's means of W_{so} and of F over its own messages above its 99th percentile of W_{so}, and a setup's value the mean over its passing runs, so that $W_{\text{so}} = F + s_{\min}$ holds in the setup's values. It holds where the share is at least one half; the paper says the turnaround is the only path where the lower end of its 95% interval is at least 0.9, from 2,000 draws of each setup's runs with replacement, seed 20261022. Described beside it: the share with version 45's turnaround and with the broker's bound. Where U-b holds nowhere, U-c's verdict is "not judged". U-a, U-b and U-d and their margins are unchanged.	A difference of 99th percentiles is not a share of a rise: it came out above 1 on one round and had no interval, and its 0.9 decision fired in 25–36% of simulated blocks per cell. U-c keeps $s_{\min}$, the bound it was fixed with, since the broker's tighter bound would raise the share after the pilot was seen.	**Yes**: as D46-1.

No.	Change	Reason	Prompted by results?
D46-8	U-e takes the rise of G from idle to 70% load at normal priority, as the mean over the messages above each setup's pooled 99th percentile of G, split exactly into *excess*, *hand*, *ack*, *wait* and Q, and names the term that holds at least half of it or gives each term's rise with its sign and the floor's. At 50 messages a second the loaded setup runs its load in 5-s blocks within each run, on then off (`U50-<broker>-l88-blocks`, at a duty of 88), and U-f takes, within those runs, the rise of the mean of G from the messages sent while the load was off to those sent while it was on, a message sent within 1 s after a switch left out, and names the term the same way; the comparison of the idle runs with the on-blocks is described beside it.	With each run drawing its own paths, a difference of means between runs at 50 a second cannot be read: block L's Redis runs at 50 had W_{so} medians of 445, 473, 456 and 854 μs idle and 706, 443, 462 and 848 μs under load, so L-b's rise is one more run drawing a slow path, and with no load effect its 10% test fails 64% of the time on Redis. Within one run the connections, and so the floors, are the same while the load is on and off.	**Yes**: as D46-1.
D46-9	Every run records where the network interrupts run and how they moved (`/proc/interrupts`, `/proc/softirqs` and `softnet_stat` before and after, each card interrupt's affinity, irqbalance's state), the kernel's preemption, timer and accounting settings, the real-time throttle, each client thread's policy, cgroup and autogroup, the command line, and stress-ng's version and live command lines (`scripts/run_facts.py`); the card is chosen by its MAC address. Described per run: the producer's lateness, the share of produce requests the broker held more than 1 ms before acknowledging, the duty, each client's collections, and the consumer thread's own counters read just before each request (`/proc/thread-self/schedstat`): per turnaround, the time it waited for a core, the time it ran, and the rest. Described per cell: the wait real-time priority takes from the turnaround (normal less real-time) and what it leaves (real-time less idle); each setup with its failed attempts; the 99th percentile of $\lvert W_{\mathrm{so}} - W \rvert$.	None of these was recorded, and a referee will ask for each; the counters split the turnaround into the mechanism's parts directly. The card was chosen by its place in a list.	**Yes**: as D46-1.

No.	Change	Reason	Prompted by results?
D46-10	The block as amended, on the second x86 pair, the base slice at 3 ms, no added delay, 70 s runs with a 20 s warm-up, in three campaigns each primed: at 500 and 2,000 messages a second, idle, 70% load with the consumer at normal priority, and 70% load with the consumer at real-time priority; at 50, idle and 88% load in blocks. Every run takes version 45's instruments and D46-1, D46-2 and D46-9's. Sixteen setups, four rounds, 64 runs (`u2000`, `u500` and `u50_design.json`), each judged by the integrity rule as it ends, only passing runs read, by `scripts/under_load.py`. The four rounds stand until D46-11's pilot restates them.	The amended block asks the same questions as version 45's on quantities that mean what their names say.	No.

No.	Change	Reason	Prompted by results?
D46-11	Pilot round d, after this version is public and before any judged run, instruments only and never judged: one run of each of the sixteen setups, each campaign primed, the loaded setups at 500 and 2,000 with the second correction, so that each finds its duty (`up2000d`, `up500d` and `up50d_design.json`); and D46-2's two runs. It checks the integrity rule with the lateness rule; that every message is found in the broker's and the card's captures with no packet dropped; that the consumer's counters record time waiting for a core under load; that no client with its collector off collected; that the blocks hold 88% within 3 points while on and 0% while off; and that the run facts are written. Before any judged run the record states and the push makes public: each loaded setup's frozen duty; the rounds, by D45-5's rule simulated from this pilot (`scripts/under_load_power.py --pilot-table`), U-a's real-time clause and U-d simulated beside, a clause whose pilot effect does not clear its margin resting on the floor of four with its power stated; the pilot's values of U-a's real-time clause and of U-d; and whether the pauses were the collector's. If the lateness rule fails at 70% at a rate, every loaded setup there takes the highest load at which a second pilot round holds the load and lateness rules; if the blocks miss 88%, their duty is corrected by the miss and checked again; if a client cannot run with its collector off, it logs its collections instead; each stated before any judged run.	Version 45's power was simulated at 80–86% load: at 70% four rounds confirm U-c in 94% of made-up blocks at the pilot's effects but in 61% with the excess halved, where version 45 said 83%, and U-d sat on its margin at 2,000 a second (0.50–0.53).	**Yes**: as D46-1.
D46-12	What was read before this version: version 45's pilot, all three rounds, from its home copies, with the repository's own readers and these checks, after the pilot and before any judged run, every number in D46-1 to D46-11 among them; and block L's runs, as `l_runs.csv` reports them. The order of the runs, on the second x86 pair from its own work list: round d's campaigns at 2,000, 500 and 50 messages a second and D46-2's two runs, each copied home; then, once D46-11's statements are public, the block's three campaigns in the same order, each copied home and checked when it ends.	These readings are the results this version follows; no judged run had begun.	**Yes**: they are the results this version follows.

What changed since version 44, and why

Version 44's rate ladder judged the kernel-timed latency under load at 50 messages a second only, because an 88% load could not be held beside the clients at 500 and 2,000 (D44-10). Read after its verdicts, its runs showed how a pull consumer lets its own thread's time back into that latency: through the next request it sends, after its thread's turnaround, which a tool can time on its own socket. Version 44's load pilots, never judged, showed that turnaround's tail growing to milliseconds under load. This version tests the fix and the turnaround where load and rate bite together, and places what remains at the broker and at the card, under predictions fixed before any of its runs (`docs/results/under_load/DESIGN.md`).

No.	Change	Reason	Prompted by results?
D45-1	The block **U** on the second x86 pair, where block L and its pilots ran: Kafka and Redis, the base slice at 3 ms, no added delay, 70 s with a 20 s warm-up, in three campaigns. At 500 and 2,000 messages a second: idle; 80% load at normal priority; 80% load at real-time priority (SCHED_FIFO 80 for both processes that read the clock). At 50 messages a second: idle and 88% load at normal priority. Every run takes the socket timestamps at the device queue, both clients' journals, the receiver's capture and both directions of the sender's, the card's receive capture and the broker's own capture, every capture with a 32 MiB buffer. Sixteen setups, four rounds, 64 runs (`u2000`, `u500` and `u50_design.json`), each judged by the integrity rule as it ends, only passing runs read.	Version 44 judged the fix under load at 50 messages a second only (D44-10). A pull consumer's thread reaches the latency through its next request at rates where requests wait, and at 50 the one rise left untraced (L-b, Redis) is where the card and the broker can place it. A packet dropped from a capture leaves every later byte of its stream unread.	**Yes**: by the post hoc checks and the unjudged pilots of D45-8.

No.	Change	Reason	Prompted by results?
D45-2	What is measured, per message, on the driver's one clock beside version 44's, except where the broker's or the card's is named. Per consumer request for messages k (Kafka's fetch; Redis's XREAD or XREADGROUP): $t_{\text{req}}(k)$, the transmit timestamp of its last byte; $t_{\text{ans}}(k)$, the receive timestamp of the read that delivered its answer's last byte; $t_{\text{before}}(k)$, the receive timestamp of the answer to the request just before it on that connection, whatever it asked (Redis's acknowledgment batches go on the same connection); and the turnaround $T_c(k) = t_{\text{req}}(k) - t_{\text{before}}(k)$, the consumer's own time from an answer to its next request for messages, with no broker time or network crossing in it, zero for a request sent before that answer came in, and read over the requests sent while the run's measured messages were in flight. Per message, the thread's saving lies between $s_{\min} = \min(T_c, \max(0, t_{\text{req}} - t_{\text{rx,ack}}))$ and $s_{\max} = \min(T_c, \max(0, t_{\text{req}} - t_{\text{tx,so}}))$, both T_c when $t_{\text{rx,ack}} \le t_{\text{before}}$, and the thread-free latency is $F = W_{\text{so}} - s_{\min}$, both W_{so} and F over the messages the journals time. From the broker's own capture, on its clock: P_{in} and F_{in}, the arrivals of the last byte of the produce request and of the delivering request; A_{out}, the departure of the acknowledgment's last byte; R_{out}, the departure of the byte that ends the message's own id in its answer, at the later of Kafka's two carryings in one record, which the socket and the card time; the broker's residence $R = R_{\text{out}} - P_{\text{in}}$ and the broker-side saving range $[\min(T_c, \max(0, F_{\text{in}} - A_{\text{out}})), \min(T_c, \max(0, F_{\text{in}} - P_{\text{in}}))]$. From the card: $Q = t_{\text{rx,so}} - t_{\text{card}}$, the consumer kernel's receive processing, the one term read across two clocks, the card's put on the system's through its samples, how tightly recorded per run. The split: $F = (W_{\text{so}} - R - Q) + (R - s_{\min}) + Q$. Where no run of the block has card timestamps, every run's split carries Q as zero, folded into the first term, and the rows say so; otherwise a run without them has no split, stays out of the pool, and is listed.	The turnaround is the thread's own time, read on one socket. The saving is bounded, because no client clock sees when the broker could hand the message out. The broker's capture and the card's timestamps place what remains, and the two machines' clocks never meet.	No.

No.	Change	Reason	Prompted by results?
D45-3	U's predictions, a setup's value the median over its passing runs of each run's statistic; a cell, one broker at one rate, is out of reach where a setup it needs has no passing run or no value of the statistic, and the prediction is judged on the cells it reaches; a clause that fails fails its cell, even where the cell's other clause cannot be read. At 500 and 2,000 messages a second, on each broker: **U-a**: T_c's 99th percentile at 80% load and normal priority is at least 1.5 times its value idle, and at 80% load its value at real-time priority is at most half its value at normal priority. **U-b**: W_{so}'s 99th percentile at 80% load and normal priority is at least 1.5 times its value idle; where it holds, the paper says that a pull consumer's kernel-timed latency does not hold still under load at that rate, on that broker. **U-c**: at each rate and broker where U-b holds, the share of U-b's rise that the turnaround removes, $1 - (F_{\mathrm{load}} - F_{\mathrm{idle}})/(W_{\mathrm{so,load}} - W_{\mathrm{so,idle}})$ with setup values of the 99th percentile, is at least one half; where it is at least 0.9, the paper says that the turnaround carries the load into W_{so} at that rate, on that broker, and elsewhere it gives the share and U-e's split; where U-b does not hold, the cell is not judged. **U-d**: at 80% load, W_{so}'s 99th percentile at real-time priority is at most half its value at normal priority.	The mechanism: a woken client thread waits for a core under load, and at real-time priority it does not; a pull consumer's thread reaches the latency through the request it sends next.	No.

No.	Change	Reason	Prompted by results?
D45-4	U's decision rules. **U-e**: at 500 and 2,000 messages a second, on each broker, the rise of F from idle to 80% load at normal priority, as the mean over the messages above each setup's pooled 99th percentile of F, splits exactly into the split's three terms; the paper names the term that holds at least half of it, or gives the split. **U-f**: at 50 messages a second, on each broker, the rise from idle to 88% load of the mean of F over each setup's pooled messages, where F is W_{so}, splits the same way, and the paper names the term that holds at least half of it. The pool is a setup's runs whose split reaches at least 99.5% of their timed messages, and the messages the split reaches. Described, not judged: the medians of all the above, the split's terms per run among them, the flagged share, the share of reads that drained more than one packet, $\lvert W_{\mathrm{so}} - W \rvert$ against the captures, the saving's bounds and widths on the client's side and the broker's, and what block U's extra instruments cost: its idle medians of W_{so} and of the end-to-end latency at 500 and 2,000 messages a second less block L's at the same rates.	What remains after the turnaround lies in the network with the producer's sending path, in the broker, or in the consumer kernel's receive processing, and only the broker's capture and the card can tell them apart. A run whose split falls short would pool only the messages before a hole in a capture.	No.
D45-5	The margins and the rounds. U-a's 1.5 and one half, U-b's 1.5 and U-d's one half repeat K-c's effect sizes (D44-3). U-c's one half asks the turnaround to be the main path, and 0.9 is the bar for the paper to call it the only one. U-e's and U-f's one half names a term only where it holds most of the rise, and the pool's 99.5% is F-c's share of messages a capture may miss. Four rounds, simulated from block L's and version 44's pilots' per-run values (`scripts/under_load_power.py`): U-a's growth clause, U-b and U-c, each in all four cells at once as the judge requires, confirm in at least 94% of made-up blocks at the pilots' effects, and in at least 83% with the load's excess over idle halved, against the rule's 80%. U-a's real-time clause and U-d have no pilot and rest on the floor of four.	The rounds rule of D4-2, with its floor of four.	**Yes**: U-c's margins and the rounds use the readings of D45-8.

No.	Change	Reason	Prompted by results?
D45-6	How it is judged: `scripts/under_load.py read` reads the block's passing runs, with the journals through `scripts/so_journal.py`, the broker's capture and the card's, and writes a row per run (`u_runs.csv`) and each setup's pooled split (`u_tails.csv`); `scripts/under_load.py judge` applies D45-3 and D45-4 and writes `u_verdicts.json`, with block L's `l_runs.csv` for the cost described. The kit gains one switch, `broker_capture`: the broker captures its own port, both directions, at full length and to the nanosecond with a 32 MiB buffer; the capture is stopped by its own id and waited for, and it and tcpdump's count of drops are copied into the run's folder. The card's capture takes the same buffer wherever a setup's other captures do (`cloud/azure/campaign.sh`).	Every number the paper takes from the block is computed by a program fixed here.	No.
D45-7	The pilot, after this version is public and before any judged run, instruments only and never judged: one round of the sixteen setups (`up2000`, `up500` and `up50_design.json`). It checks that every run passes the integrity rule or that its failure is read for its instrument; that every message is found in the broker's capture and in the card's, and that no capture drops a packet; that the card's device and clock are recorded; and that 80% load holds at both upper rates and at real-time priority. If 80% does not hold, every load setup at 500 and 2,000 takes the highest load the pilot held within the rule's 3 points at both rates, and the record says so before any judged run; if the card gives no timestamps, Q folds into the split's first term.	80% is the least load version 44's pilots held beside the clients at 2,000 a second, and block U adds the producer's journal and the card's capture on the driver; the card's instrument has not run on this pair.	No.

No.	Change	Reason	Prompted by results?
D45-8	What was read before this version, every range rounded outward. Block L's judged runs, as the paper reports them. On 20 and 21 October, after block L's verdicts: the turnaround and two corrections on block L's idle runs at 500 and 2,000 messages a second, their acknowledgments rebuilt from the producer's capture. At 2,000 the turnaround's median was 0.42–0.51 ms on Kafka and 0.15–0.17 ms on Redis, and removing it brought W_{so}'s 99th percentile from 1.79 to 1.30–1.35 ms on Kafka and from 1.69 to 1.32–1.34 ms on Redis (medians over runs). On 22 October, version 44's load pilots, never judged, seven of whose eight runs failed the load check (`lp2000`, 80.0–82.0% held when 88% was asked; `lp500188`, 83.5–85.8%): the turnaround's 99th percentile was 3.53–5.33 ms, against 0.20–0.81 ms idle; W_{so}'s 99th percentile was 2.5–4.1 times its idle value at 2,000, against the pilot's own idle runs, and 4.9–5.9 times at 500, against block L's idle medians; and removing the turnaround took out 68–101% of that rise against the same references (Kafka 88–101%, Redis 68–82%). These turnarounds were counted as an earlier script counted them, over every command on the consumer's connection and over the whole run; the registered reader counts the requests for messages sent while the measured messages were in flight, which moves the values under load by about 1% and the idle ones by up to about 40 μs.	These readings set U-c's margins, the 80% load and the rounds; no other prediction uses them.	**Yes**: they are the results this version follows.
D45-9	The order of the runs, on the second x86 pair from its own work list: the pilot's three campaigns at 2,000, 500 and 50 messages a second; then, once the pilot's instruments are read, the block's three campaigns in the same order, each copied home and checked when it ends.	Each campaign is copied home before the next, so the driver's disk holds one campaign's captures at a time.	No.

What changed since version 43, and why

Version 43 tested the full fix in the form a benchmark tool would read it, and all five of its predictions held. Two referees of the paper's ninth version then asked, independently, for the same measurements: the acknowledgment's own arrival, timed by the kernel, to settle the one rival the paper leaves open; a positive control and a rate ladder for the kernel-timed latency; and direct tests of the timescale. This version runs them, under predictions fixed before any of their runs.

No.	Change	Reason	Prompted by results?
D44-1	The block **K** on the first x86 pair: version 43's eight setups (Kafka and Redis; no background load or 88%; normal or real-time priority, SCHED_FIFO 80, for both processes that read the clock), the base slice at 3 ms, no added delay, 50 messages a second for 130 s with the 30 s warm-up left out, the socket timestamps taken at the device queue as in F2, with every read and write the clients make on their broker connections journaled, both directions of the producer's connection captured, and the receiver's capture as before; and two more setups, Redis at real-time priority with one send worker instead of sixteen, idle and at 88%. Ten setups, six shuffled rounds, 60 runs (`k_design.json`).	Two referees asked, independently, for the measurement the paper says would settle its one open rival: whether the broker ever puts the delivery on the wire before the acknowledgment. F2 built the instrument and did not log the acknowledgment's arrival.	**No**, the referees' reports of 15 October; the one-worker setups follow D44-11.
D44-2	What is measured, per message, on the driver's one clock, beside version 43's: $t_{\mathrm{rx,ack}}$, the kernel's receive timestamp of the producer's read that completed the message's acknowledgment (Kafka: the produce response whose correlation id is the produce request's that carried it; Redis: the reply to that XADD, a connection answering in order); and t_{req}, the kernel's transmit timestamp of the consumer's request that delivered it (Kafka: the fetch request whose correlation id its response carries; Redis: the XREAD command whose reply carried it). From them: $S_k = t_{\mathrm{rx,so}} - t_{\mathrm{rx,ack}}$, the order timed by the kernel; $\delta_{\mathrm{ack}} = t_{\mathrm{ack}} - t_{\mathrm{rx,ack}}$, the late reading itself; $\delta_{\mathrm{recv}} = t_{\mathrm{recv}} - t_{\mathrm{rx,so}}$; and, on one clock, $S = S_k + \delta_{\mathrm{recv}} - \delta_{\mathrm{ack}}$ exactly. A message carries the pull flag when $t_{\mathrm{req}} > t_{\mathrm{tx,so}}$. The journal, `scripts/so_timestamps.py -so-journal`, writes each connection's bytes each way, one row per read with the stream offset after it and its receive timestamp, and every transmit timestamp with the offset of its send call's last byte; `scripts/so_journal.py` reads the brokers' wire formats after the run. The client libraries are unchanged.	No client thread's wait is in S_k; the identity names exactly what puts a value of S below zero, message by message.	**No.**

No.	Change	Reason	Prompted by results?
D44-3	K's predictions, a setup's value the median over its passing runs of each run's statistic unless a prediction pools. **K-v**: in every setup the median over runs of each run's median of $\lvert t_{\mathrm{rx,ack}} -$ the capture's arrival timestamp of that acknowledgment$\rvert$ is at most 20 μs. **K-a**: in every Kafka setup with at least 20 negative values of S, at least 90% of the messages with $S < 0$ have $S_k \geq 0$, pooled. **K-b**: in Redis's two real-time setups with sixteen workers, at least half of the messages with $S < 0$ have $S_k < 0$, pooled; for every Redis setup $\Pr[S_k < 0]$ and the share of negative S with $S_k < 0$ are described, and the paper attributes to reply order exactly the share of each broker's negative S that has $S_k < 0$. **K-c**: on both brokers (sixteen workers on Redis), δ_{ack}'s 99th percentile at 88% load and normal priority is at least 1.5 times its value idle, and at 88% its value at real-time priority at most half its value at normal priority. **K-d**: in each broker's setup at 88% and normal priority, the messages with $S_k \geq 0$, pooled and cut into quintiles by S_k, give a rate of $S < 0$ that does not rise from the lowest quintile to the highest, and that lies within a factor of two of $\Pr[\delta_{\mathrm{ack}} - \delta_{\mathrm{recv}} > S_k]$, from the setup's own distribution, in every quintile with at least 20 negatives; the paper's $p\,G(S_k)$ beside it, described. **K-e**, a decision: if the one-worker Redis setup's rate of $S < 0$ is at most half the sixteen-worker setup's at the same load, the interpreter lock sets at least half of Redis's real-time rate; otherwise it does not.	K-v holds the journal to the capture; K-a and K-b name each broker's share of reply order; K-c and K-d test the mechanism message by message; K-e settles the interpreter lock.	**No**, except K-b's prediction for Redis at real-time priority (D44-11).
D44-4	The block **R** on the first x86 pair after K: Kafka and Redis at 88% load and normal priority, the scheduler's `RUN_TO_PARITY` feature on (the kernel's default) or off (`NO_RUN_TO_PARITY`, set through debugfs before each run, checked, and put back after it), journals as in K; four setups, six rounds, 24 runs (`r_design.json`). **R-a**: on both brokers the rate of $S < 0$ with the feature off is at most two-thirds of the rate with it on. **R-b**: the same for δ_{ack}'s 99th percentile.	The rule is what should make a woken thread wait out the rest of a slice (Supplement S6.3); turning it off tests the timescale as directly as priority tests the waiting probability.	**No.**

No.	Change	Reason	Prompted by results?
D44-5	The block **P** on the second x86 pair: Kafka and Redis, no load, normal priority, a delay of 0, 1 or 2 ms added on the broker's path to the receiver only, socket timestamps and both captures; six setups, four rounds, 24 runs (`p_design.json`). **P-a**: on each broker W_{so}'s median rises by between 0.9 and 1.1 ms per ms added, the least-squares slope through the three setup medians. **P-b**: the publish latency A's median moves by at most 10% between any two delays. **P-c**: D's median rises by between 0.9 and 1.1 ms per ms.	A latency that held still under load is what a sound instrument shows and also what a broken one shows; the delay tests that it moves when the path does.	**No.**
D44-6	The block **L** on the second x86 pair: Kafka and Redis, no load or 88%, normal priority, in three campaigns at 50, 500 and 5,000 messages a second, each for 70 s with a 20 s warm-up, socket timestamps, the consumer's journal and both captures; four setups a rung, four rounds, 48 runs (`l50`, `l500` and `l5000_design.json`); the top rung 2,000 if the pilot finds 5,000 out of the clients' reach. **L-a**: at every rung, in every setup, the median over runs of each run's median of $\lvert W_{so} - W \rvert$ is at most 20 μs. **L-b**: at every rung, on each broker, W_{so}'s median at 88% lies within 10% of its median idle; where it does not, the paper scopes the fix to the rungs where it does. **L-c**: at 50 messages a second at most 1% of messages carry the pull flag in every setup; at every rung the flagged share, W_{so} for flagged and unflagged messages, the share of consumer reads that drained more than one packet, and the rate of $S < 0$ are described.	At benchmark rates a pull consumer's request cadence, receive processing deferred to a kernel thread, and reads that drain several packets could put client waits back into the kernel-timed latency.	**No.**
D44-7	The block **I** on the second x86 pair, last: Kafka and Redis at 88% load and normal priority, on the stock kernel's default boot and then on a boot with `idle=poll` on the kernel command line, selected once by the pair's own list with grub-reboot and verified from `/proc/cmdline` before any run; journals as in K; two setups a boot, four rounds, 16 runs (`i_default` and `i_poll_design.json`). A decision: if `idle=poll` at least halves the share of Redis's negative S between -250 and -150 μs, that mode is the wake of an idle processor; otherwise it is not. δ_{ack}'s median and its share above 100 μs are described for both boots.	A referee proposed the wake of an idle processor for that mode; a processor that spins needs no waking.	**No**, except its load (D44-11).

No.	Change	Reason	Prompted by results?
D44-8	The margins. K-v and L-a repeat G-a. K-a's 90% is the share a "late reading" must explain to keep its name; K-b's half is a deliberately weak bar for a mechanism two sources point to; K-c's 1.5 and one half repeat the effects F1 and F2 required of D; K-d's factor of two widens the registered campaign's 1.5, quintiles of one setup holding fewer messages than a condition; R's two-thirds asks for an effect a reader would notice; P's ±10% around one for one is the precision F2's setups gave W, 3–8% at the median; L-b repeats F-a.	Each margin set from the precision it must clear and the effect it must not hide.	**No.**
D44-9	How it is judged: `scripts/order_scope.py read` reads each block's passing runs, with the journals through `scripts/so_journal.py`, and writes a row per run and, for K, the quintiles of D44-3; `scripts/order_scope.py judge` applies D44-3 to D44-7 and writes the verdicts. The kit gains the setups' new switches (`so_journal`, `send_capture_both`, `send_workers`, `sched_features`, `idle_poll`), the scheduler feature in `scripts/sched_settings.py`, the `idle=poll` boot in `cloud/azure/idle.sh` and the list's boot type `idlepoll`.	One program, written and tested, every one of its checks mutated once, before any run of the block was read.	**No.**

No.	Change	Reason	Prompted by results?
D44-10	The pilot, 15 October 2026, instruments only and never judged. **K and R**, first x86 pair, one round of their fourteen setups (`k_pilot_design.json`, seed 20261016): all fourteen runs passed the integrity rule at their first attempt; in every run every message's acknowledgment, and the consumer request that delivered it, was found in the clients' journals, none missing; and in every run that captured both directions, each acknowledgment's receive timestamp in the producer's journal was the capture's own. **L**, second x86 pair, one idle run per broker at 50, 500 and 5,000 messages a second (`lp50`, `lp500` and `lp5000_design.json`): the sockets' kernel-timed latency came within 1.2–2.6 μs of the captures' at the median at every rung. At 500 a second Kafka's idle run failed the load check three times, at 7.1–7.4% against 0%, because the integrity rule counted the clients' own processor time as background load. **Change 1**: the sampler writes the clients' share of the machine beside its utilisation, and the load check reads the machine's load less that share (`scripts/util_sampler.py`, `scripts/run_integrity.py`); at 50 a second that share is under a point. At 5,000 a second Redis's producer sent 3,536 and 3,579 messages a second, so, as D44-6 provides, **the top rung is 2,000**. Further pilots then tried the 88% load at the upper rungs (`lp2000` and `lp500l88_design.json`). Beside the clients it held 80.0–81.9% at 2,000 a second on both brokers, and at 500 a second 83.5–83.9% on Kafka and 85.7% on Redis, at or past the rule's three-point edge, while the idle runs at 2,000 passed, their background load 0.01% and 0.12% once the clients' share was taken out. **Change 2**: the 500 and 2,000 rungs run their two idle setups only, eight runs each, so L-b is judged at 50 messages a second alone, and block L has 32 runs. **I**, second x86 pair: the `idle=poll` boot came up, `/proc/cmdline` held the token, and one Redis run at 88% passed (`ip_design.json`). **Change 3**: the reader takes each broker's port from the journal's own connections, since the kit's Kafka listens on 19092; the pilot's first reading of Kafka's packets found none on the port it had assumed. No prediction changes. The pilot's runs are in	Instruments only; the predictions above do not change, and a rung the machine cannot hold a load at is out of reach, not failed.	**No**: by the pilot's checks of the instruments and of what the machine can hold, before any judged run.

No.	Change	Reason	Prompted by results?
D44-11	What was read before this version: on 15 October, before `DESIGN.md` was written, F2's own runs. With no background load, 10.9% of Redis's S was below zero at normal priority and 17.9% at real-time priority, every value within 70 μs of zero; at 88% load and normal priority 1,479 of Redis's 4,824 negative values lay between -250 and -150 μs; Kafka's lay below zero only under load at normal priority, most below -500 μs. These are why K-b is a prediction for Redis at real-time priority and why block I runs at 88%; nothing else used them. `DESIGN.md` was fingerprinted and pushed before any pilot (commit 176ddbe7). The order: each pair's own list runs its queues in the shuffled order each seed fixes, the first x86 pair K then R, the second P, L's three rungs, then I's two boots, and ends by copying the runs off and stopping itself.	A prediction informed by data is labelled as such.	**Yes, in part**: K-b's Redis prediction and block I's load.

What changed since version 42, and why

Version 42 tested a full fix and all five of its predictions held: timed between the kernel's own timestamps, read from packet captures, a message's latency held still across background load and priority, while the latency timed from the publish call rose 4.3 to 4.6-fold in its tail at normal priority. A benchmark tool would not run a capture. It would ask the kernel for the same timestamps on its own sockets, through the `SO_TIMESTAMPING` socket option, and the paper says that form was not tested. This version tests it, beside the captures, under predictions fixed before any of its runs.

No.	Change	Reason	Prompted by results?
D43-1	The block **F2** on the first x86 pair, as F1: both brokers, the base slice at 3 ms, no added delay, 50 messages a second for 130 s with the 30 s warm-up left out; no background load or 88%; normal or real-time priority (SCHED_FIFO 80) for both processes that read the clock. Eight setups, six shuffled rounds, 48 runs, every run judged by the integrity rule as it ends and only passing runs read, its folders `runs/azure/collected/matched/f2`. Both directions are captured, as F1 captured them, as the reference; the network card's own timestamps are not.	The fix has to be shown in the form a tool would ship, and against the instrument that showed it, message by message.	**Yes.** Prompted by F1's result and by the paper's sentence that this form was not tested; none of F2's runs exists as this is frozen.

No.	Change	Reason	Prompted by results?
D43-2	What is measured, per message, on the driver's one clock: t_{pub}, t_{tx}, t_{rx} and t_{recv} as in F1, and two more. $t_{\text{tx,so}}$ is the kernel's transmit timestamp of the producer's send call that carried the end of the message's event id, read by the producer from its socket's error queue, keyed by the byte offset of the call's last byte, so that a message is matched to the first timestamp at or after its id's last byte. $t_{\text{rx,so}}$ is the kernel's receive timestamp returned with the consumer's read that delivered the end of the id: for a stream socket, that of the last packet the read consumed. From them $W_{\text{so}} = t_{\text{rx,so}} - t_{\text{tx,so}}$, beside F1's $W = t_{\text{rx}} - t_{\text{tx}}$ from the captures. A small module, `scripts/so_timestamps.py`, replaces the socket class in each client's process before the client library opens its connections; on each connection to the broker's port it turns the option on at the first send or read, once the connection is up, drains the error queue only when `poll()` reports something waiting, and finds each event id in the bytes as `scripts/full_fix.py` finds it in the captures. The client libraries are unchanged.	The kernel puts the same receive timestamp on a packet for the capture and for the socket, and takes a transmit timestamp on the socket without any program thread; so W_{so} should be W, read without a capture.	**No.**
D43-3	The predictions, a setup's value being the median over its passing runs of each run's statistic. **G-a**: in every setup, the median over its runs of each run's median of $\lvert W_{\text{so}} - W \rvert$ is at most 20 μs, and of each run's 99th percentile of it at most 100 μs. **G-b**: as F-a for W_{so}: for each broker and each priority, its median at 88% load lies within 10% of its median idle, and for each broker and each load, its median at real-time priority within 10% of its median at normal priority. **G-c**: the same comparisons for its 99th percentile, within 25%. **G-d**: no message has W_{so} at or below zero, and the messages missing either socket timestamp are counted and are under 0.5% of each run's. **G-e**: as F-d with W_{so}: at normal priority, D's 99th percentile at 88% load is at least 1.5 times its value idle on both brokers, and W_{so}'s mean moves by less than a fifth of the rise in D's mean.	G-a says the socket option reads what the captures read; G-b to G-e repeat F1's tests of the fix with W_{so}.	**No.** Written before the pilot (`DESIGN.md`, fingerprinted before any F2 run).

No.	Change	Reason	Prompted by results?
D43-4	The margins. G-b to G-e repeat F1's. For G-a, the receive timestamps should be identical, the capture and the consumer's socket reading the same timestamp the kernel put on the packet, unless one read consumes more than one packet, which at 50 messages a second should be rare; the transmit timestamps differ by the path between where the socket's timestamp is taken and the capture's tap on eth0, microseconds. 20 μs is about 3% of F1's kernel-timed latency, wide enough for that path and narrow enough that an instrument that disagreed would show; 100 μs in the tail leaves room for the rare read that consumes two packets and stays far below the milliseconds the clients' waits add.	A margin set from the path it must clear and an effect it must not hide.	**No.**
D43-5	How it is judged: `scripts/so_fix.py` reads each passing run's captures as `scripts/full_fix.py` does, and its two socket files, `producer_so.csv` and `consumer_so.csv`, and judges G-a to G-e, with F1's own functions for G-b, G-c and G-e. Pauses are marked as F1 marks them, in D and in W_{so}, and the runs they held are listed beside the verdicts. Each run's row also records its instrument checks and, described and not judged, where the two instruments' timestamps differ and how often the receive timestamps are the very same.	One program, written and tested before any F2 run was read.	**No.**

No.	Change	Reason	Prompted by results?
D43-6	The pilot, on 14 October, after the predictions above and before any judged run, in two steps, its runs not judged. The first step ran one idle run on each broker (`f2_smoke_design.json`). On Redis the run passed the integrity rule, and the consumer's socket returned a receive timestamp with every one of its 6,500 messages, but the producer's socket returned no transmit timestamp at all. Probes on the driver, kept with the runs, showed why: the kernel gives software transmit timestamps on the loopback device but none on the path to the broker, while the timestamp taken as a packet enters the device queue arrives for every send, twice; the first is kept. So, as `DESIGN.md` said it would, the transmit timestamp is taken at the queue (`SOF_TIMESTAMPING_TX_SCHED`). On Kafka all three attempts of the first step failed before a message was sent. kafka-python sets a timeout on its socket while it sends, and on such a socket Python waits for the socket to become readable before it reads the error queue, so the module's drain held each request past the client's own timeout. The module now drains the queue only when `poll()` reports something waiting, and a test holds it to that. The second step (`f2_pilot_design.json`) ran all eight setups with both changes. All eight passed the integrity rule, the four idle runs at a measured load of 0.52 to 1.28% against a limit of 3 points. In every run neither capture dropped a packet, each of the 4,988 messages the consumer stamped after the warm-up was found in both captures and had both socket timestamps, and no socket latency came out at or below zero. Both steps' runs, their logs and the probes are kept in `runs/azure/collected/matched`: `f2_smoke`, `f2_pilot` and `snapshot_20261013T234847Z`.	Instruments only; the predictions above do not change.	**No.**
D43-7	The order. The first x86 pair's own list runs the block's queue, the six rounds in the shuffled order its seed fixes; the list ends by copying the runs off the pair, and the pair stopping itself.	As every block's.	**No.**

What changed since version 41, and why

Version 41 finished the tools block. Every remedy the paper names leaves something: timing from the publish call keeps the receiving thread's wait for a core, and real-time priority shrinks the waits without ending them. On 13 October 2026, after the fact, R1's packet captures (`docs/results/recv_wait`, the runs of 3 October) were read again: a latency timed to the kernel's receive time, rather than to the consumer's own clock, no longer moved with the consumer's priority or the machine's load. R1 captured only the receiving side. This version tests the whole fix, both sides, under predictions fixed before any of its runs.

No.	Change	Reason	Prompted by results?
D42-1	The block **F1** on the first x86 pair, R1's machines: both brokers, the base slice at 3 ms, no added delay, 50 messages a second for 130 s with the 30 s warm-up left out, as R1. Two factors on the driver: background load, idle or 88%, and priority, normal or real-time (SCHED_FIFO 80) for both processes that read the clock. Eight setups, six shuffled rounds, 48 runs, every run judged by the integrity rule as it ends and only passing runs read, its folders `runs/azure/collected/matched/f1`.	A full fix has to be shown, not argued: the latency it reports must not move when the host's threads are made to wait, while the latency timed by those threads does, in the same runs.	**Yes.** Prompted by R1's reading after the fact, which covers the receiving side only; none of F1's runs exists as this is frozen.
D42-2	What is measured, per message, on the driver's one clock: t_{pub}, the producer's clock at the publish call; t_{tx}, the kernel's capture time of the packet carrying the end of the message's event id as it leaves, in the root namespace on eth0, from the driver to the broker and not the commands sent over ssh (new, `sender_full.pcap`); t_{rx}, the same as it arrives, in the receiver's namespace (R1's `receiver_full.pcap`); and t_{recv}, the consumer's clock when it stamps the message. From them $D = t_{\text{recv}} - t_{\text{pub}}$, the latency timed from the publish call; $W = t_{\text{rx}} - t_{\text{tx}}$, the wire time, network and broker, the fix; and the host's two waits, $H_{\text{send}} = t_{\text{tx}} - t_{\text{pub}}$ and $H_{\text{recv}} = t_{\text{recv}} - t_{\text{rx}}$, so that $D = H_{\text{send}} + W + H_{\text{recv}}$ message by message. Idle is no load at all, not a duty of one percent. Described and not judged: the receiver's packets as the network card stamped them on its own clock, on the accelerated-networking device, with that clock sampled against the system's twice a second (`phc_offset.py`), so that each stamp is put on the system's clock; the card's arrival less the kernel's is the kernel's own delay, the part of the host the kernel's timestamp keeps.	The kernel stamps a packet when it passes, whether or not any program thread is running, so neither stamp waits for a core; and both are on the clock the clients read. The card stamps it earlier still, before the kernel's interrupt; it is described, not judged, because the card's clock must first be put on the system's.	**No.**

No.	Change	Reason	Prompted by results?
D42-3	The predictions, a setup's value being the median over its passing runs of each run's statistic. **F-a**: for each broker and each priority, W's median at 88% load lies within 10% of its median idle; and for each broker and each load, W's median at real-time priority lies within 10% of its median at normal priority. **F-b**: the same comparisons for W's 99th percentile, within 25%. **F-c**: no message has W at or below zero, and the messages either capture missed are counted and are under 0.5% of each run's. **F-d**: in the same runs, at normal priority, D's 99th percentile at 88% load is at least 1.5 times its value idle on both brokers, and W's mean moves by less than a fifth of the rise in D's mean. **F-e**: on Kafka at 88% load and normal priority, H_{send}'s mean is at least 0.2 ms larger than idle, its client's background thread sending each message after the publish call returns; on Redis, whose client writes from the calling thread, it moves by less than 0.1 ms.	F-a to F-c are the fix; F-d says the effect it removes is present in the runs that test it, and is the host's; F-e names where Kafka's waits sit.	**No.** Written before the pilot (`DESIGN.md`, fingerprinted before any F1 run).
D42-4	The margins, from R1's reading: within a setup, run medians of the kernel-timed latency spread over about 110 μs on Redis and 330 μs on Kafka, so that six runs fix a setup's median to about 20 and 60 μs; ten percent of the medians, about 70 and 140 μs, is wider than that and narrower than the host's effect at the median, about 170 μs on Redis and 0.4 to 1.0 ms on Kafka. In the tail the effect is milliseconds, so 25% of a sub-millisecond W separates the two easily.	A margin set from noise it must clear and an effect it must not hide.	**No.**

No.	Change	Reason	Prompted by results?
D42-5	How it is judged: `scripts/full_fix.py` reads each passing run's four times, finds each event id in the sending stream by the packet that carried the end of it out, whatever became of earlier packets, and in the receiving stream as `recv_wait.py` finds it, and judges F-a to F-e. Pauses are marked as R1 marked them, each run counting the messages whose D, and whose W, pass the quality report's limit of 150 ms, and the runs they held are listed beside the verdicts. Where a run captured the card's own stamps, the wire time to the card and the kernel's own delay are written beside, described and not judged. Each run's row also records its instrument checks: the load the integrity rule measured, the packets its captures dropped, and each span's smallest value.	One program, written and tested before any F1 run. While the pilot ran, before any of its spans was read, it gained the marking of pauses `DESIGN.md` asks for; after the pilot, the columns that record each run's instrument checks. Neither changes how F-a to F-e are computed.	**No.**

No.	Change	Reason	Prompted by results?
D42-6	The pilot, on 13 October, after the predictions above and before any judged run: one round of the eight setups, its runs not judged (`f1_pilot_design.json`). Its first attempt stopped at its first idle run, after three runs under load. The runner treats an unset variable as an error, and an idle run, which starts no load, left the variable that holds the load's process unset; it is now set empty first, and a test runs those lines as the runner does, under the same rule. In the same attempt the card capture found no device: it looked for the word SLAVE in the address column of `ip -br link` rather than in its flags. It reads the flags now, and a test holds it to the driver's own listing. The second attempt (`f1_pilot2_design.json`) ran all eight setups, and all eight passed the integrity rule, the four idle runs at a measured load of 0.45 to 1.02% against a limit of 3 points. In every run neither capture dropped a packet; each of the 4,988 messages the consumer stamped after the warm-up was found in both; and every message left the driver before it arrived, $t_{\mathrm{tx}} < t_{\mathrm{rx}}$. The card offers its own receive timestamps (`ethtool -T`: hardware receive and a raw hardware clock, on the accelerated-networking device), so, as `DESIGN.md` said it would, every run of the block captures them beside the kernel's, described and not judged. In the pilot each sample bracketed the card's clock within about 1.5 μs of the system's, and the card stamped every packet after it left the driver and before the kernel stamped it. The first attempt's three runs, the stopped run and both attempts' logs are kept with the second's (`runs/azure/collected/matched`: `f1_pilot`, `f1_pilot2` and `snapshot_20261013T124310Z`).	Instruments only; the predictions above do not change.	**No.**
D42-7	The order. The first x86 pair's own list runs the block's queue, the six rounds in the shuffled order its seed fixes; the list ends by copying the runs off the pair, and the pair stopping itself.	As every block's.	**No.**

What changed since version 40, and why

Version 40 left one tool of the second October reading unrun, because it drives Google Cloud's Pub/Sub and would need a project and billing of its own there *(D40-7)*. The author made that project. This version runs the tool: one whose controller throws, with a message that blames negative values, when half its latencies fall at or below a millisecond, and labels every bucket one bucket below the values in it.

No.	Change	Reason	Prompted by results?
D41-1	The last tool of the second October reading *(`data/tools_audit/batch_11.json`)* is run at the version whose code that reading read: **Google's Cloud Pub/Sub load test framework**, at cd7d07bb on its repository's master, whose latency, bucketing and percentile files are those of its last release tag, e7e3a6e8; with the Java client library its pom pins, google-cloud-pubsub 1.69.0. Built as its `run.py` builds it, `mvn package` and its `driver.jar` copied where its local and GCE modes look for the clients' jar *(run.py:21-22; Client.java:47)*, by Maven 3.9.9 on Ubuntu's Java 8, which its pom compiles for *(pom.xml:197-198)* and its GCE startup scripts install for its clients, and run on Java 11 *(D41-2)*. Its broker is Google's own Cloud Pub/Sub: its local and in-process controllers make their topic and subscription through Pub/Sub's REST API with Application Default Credentials *(LocalController.java:43-71; PubsubResourceController.java:52-100)*, so no emulator stands in without changing its code. The author made a project for it in the author's own Google Cloud account, `sbl-pubsub-load-test`, with a service account that may use Pub/Sub alone; its key lives on the driver only, never in the repository, and the project is deleted when the runs are home, as version 37's Service Bus queue was *(D37-1)*. It runs T1 to T4 under the rules every tool ran under, four rounds *(D4-2)*, in folders of its own (`runs/azure/tools_v41`, then `_round2` to `_round4`).	It is Google's own load test for Pub/Sub, published in Google's Pub/Sub repository, and its sentence in the paper rests on its source alone.	**No.** None of its runs under this plan has been read: the pilot *(D41-6)* ran it to check the instruments.

No.	Change	Reason	Prompted by results?
D41-2	How it is run *(`cloud/azure/tools_run.sh`)*: its driver, `java -jar target/driver.jar -project sbl-pubsub-load-test -local -test_parameters latency -burn_in_minutes 1 -test_minutes 2`, from the second x86 pair's driver. Its latency preset publishes 1-byte messages one at a time, batch size 1, at the rate the preset fixes, one a second *(StandardParameters.java:23-29)*; its burn-in and test, by default 5 and 10 minutes *(TestParameters.java:53-54)*, are shortened by its own flags *(ParameterOverrides.java:41-48)*. Its default client is Java, on Cloud Pub/Sub *(Driver.java:82-91)*. In local mode the driver starts its publisher and its subscriber as two processes on the same machine, each a `java` found on the path *(LocalComputeResourceController.java:57, 80-90)*, and reaches them on its loopback. It runs on the machine itself in every stage, as PerfKit does: its broker is a managed service the receiver's namespace was never for, and the namespace cannot reach the machine's name resolver *(D41-6)*. The rest of the wrap, a moved clock or go-first, goes to the subscriber alone, the process that subtracts, through a `java` first on the path written for each run, under `sudo` as PerfKit's, with its credentials passed on; it writes down every process it starts and how. Its driver and both clients run on Java 11, which runs the Java 8 its pom compiles for: under libfaketime Java 8's `Thread.sleep` never returns, whatever libfaketime is told, and the worker waits for its start and for the test's end by sleeping *(JavaLoadtestWorker.java:137-147, 154)*, while on Java 11 the sleep returns with the wall clock moved *(D41-6)*. An attempt whose controller never started, printing no "Will start burn-in", measured nothing: it is set aside beside the run and made again after a minute, at most twice *(D41-6)*. Its driver is stopped after 6 minutes, inside the runner's own limit, so that its exit status and what it wrote are kept. What is read: the ranges its controller prints for each client side at the 50th, 99th and 99.9th percentiles; the `output.csv` it writes, each side's midpoints and rate; the bucket counts in every check response its driver logs from each client *(Client.java:232-236)*, summed after the burn-in and in it apart, as its controller counts only the	A tool is run as its own documentation runs it, with only the load set to the block's where it can be set.	**As to the instruments only:** the machine itself, the attempts made again, Java 11 and the bucket counts read *(D41-6)*.

No.	Change	Reason	Prompted by results?
D41-3	Its reference, which T2 takes its offsets from *(D15-1)*. As PerfKit's *(D37-1)*, its traffic goes to a managed service the staircase cannot reach, the staircase delaying the broker machine's traffic toward the driver, so its **T1** is its zero step alone, and no client of ours was written for Pub/Sub, so it is its own reference, run once more at no offset. Its controller labels each bucket one bucket below the values in it *(D41-4)*, so its subscriber's true median lies in $[U, 1.5\,U)$, U the upper bound of the 50% range it prints, and the trip T2 takes its offsets from is $1.5\,U$, the top of that bucket: T2 moves the subscriber's clock back by $1.5\,U + 1$ and $1.5\,U + 3$ ms.	As D37-1 chose for a managed service; the trip read from what the tool prints, as D40-3 read Quiver's.	**No.**

No.	Change	Reason	Prompted by results?
D41-4	Its predictions, under T-P7, from its source. It is **low**: whole milliseconds of `System.currentTimeMillis()`, then buckets a factor of 1.5 wide. Its publisher takes the wall clock once per message, writes it into the message as `sendTime` and times its own publish from it *(AbstractPublisher.java:83, 93-95; CPSPublisherTask.java:75)*; its subscriber subtracts `sendTime` from its own wall clock on receipt *(CPSSubscriberTask.java:68-70)*, in its default GCE mode on another machine: **two clocks**, its interval entered as across two processes. Each value v goes into bucket $\lfloor \log_{1.5} v \rfloor$, floored at 0 *(MetricsHandler.java:72-73)*, so 0, 1 and every value below zero share bucket 0. Its controller reads bucket n as $[1.5^{n-1}, 1.5^{n})$ and bucket 0 as $[0, 1)$ *(LatencyTracker.java:28-30, 54-59)*, one bucket below the values in it, and when bucket 0 holds at least $\lfloor Np/100 \rfloor$ of the N values its percentile search runs past its loop and throws "Programming error: negative values appear in buckets." *(LatencyTracker.java:37-52)*, called with no try before the CSV is written and before the driver exits with status 0 *(Driver.java:153-169, 239-246)*. **T2-l**: at both offsets at least half its subscriber's values fall in bucket 0, so the controller throws that exception when it computes the subscriber's 50th percentile: it prints no subscriber percentile, writes no `output.csv`, and the driver does not exit with status 0. Its control, at no offset, prints the subscriber's percentiles and writes `output.csv`, and the driver exits with status 0. Version 34's rule therefore has no figure to hold against the control, and leaves its offset runs undecided *(D34-2)*: its clocks are not answered from the offset. **T4-l**: with the subscriber's clock one second ahead under libfaketime, every subscriber latency is raised by exactly 1,000 ms, and the 50% range its controller prints for the subscriber lies wholly below 1,000 ms, its upper bound $1.5^{17} = 985.3$ ms for any trip under 478 ms. **T3** is described, real-time priority going to the subscriber. T2-l and T4-l each hold if they hold in all four rounds *(scripts/block_v41.py)*.	As D40-4 fixed version 40's before their runs.	**No.** From the reading of its source, written down before the pilot.

No.	Change	Reason	Prompted by results?
D41-5	S1 *(D37-3, as made there)* on it, four rounds, in the same folders, each step 50 ms back, as PerfKit's, since its trip to the managed service is longer than 20 ms. **S1-l**: its stepped run ends as its control does, with `output.csv` written and the driver's status 0; its subscriber's printed 50% range is its control's or one bucket away from it; and no line it prints mentions a value below zero. A step catches only the messages in flight across it, too few to carry its median into bucket 0, and each value below zero the step makes is counted with the fastest. It is judged as D37-4 judges, in all four rounds *(scripts/s1_block.py)*.	S1 asks what a tool does when the clock it reads steps back under it.	**No.** Written down before the pilot.

No.	Change	Reason	Prompted by results?
D41-6	The pilots of 12 October 2026, on the second x86 pair, kept as `pilot_v41` to `pilot_v41d`. In the first, the tool built at cd7d07bb on Ubuntu's OpenJDK 1.8.0_504, its build's own tests passing; its zero step printed both sides' medians as [11.39, 17.09) ms, counted 120 messages a side, the preset's one a second for two minutes, and wrote its `output.csv`. Its driver waits 59 s before the burn-in, so a run takes some four and a half minutes. Its reference, run straight after, never started: its controller's first call to Pub/Sub's REST API timed out, the run before it having just deleted the framework's topic and subscription. Such an attempt is now set aside beside the run and made again after a minute, at most twice. Its T3, T4 and S1, run in the receiver's namespace, ended at once: "UnknownHostException: oauth2.googleapis.com", the namespace having no name resolver. It now runs on the machine itself in every stage, as PerfKit does *(D37-1)*. In the second, its reference gave $U = 17.09$ ms and T2's offsets 26.63 and 28.63 ms, which libfaketime was measured to give as 26.51 and 28.66 ms. Its control printed both sides and wrote its `output.csv`. Under each offset its subscriber, whose monotonic clock libfaketime moved as well as its wall clock, recorded nothing: its start, scheduled a minute ahead, never came, while its publisher published and timed some 200 messages; its driver printed neither side and was stopped at six minutes. Its T4, a second ahead, did the same. Its T3 ran in all four cells, each counting 120: its median printed as [11.39, 17.09) ms idle, [17.09, 25.63) ms at 88% load, and [11.39, 17.09) ms again at real-time priority. Its S1 stepped run made 1,714 steps, 85.7 s back in all, and printed its median as [11.39, 17.09) ms; the clock was put back within 0.01 ms of the PTP clock. Its S1 control ended part way: its controller's own check handler threw a `NullPointerException` *(Client.java:250)*, after which the controller gave up on its subscriber, "failed 4 health checks". Such an attempt is now set aside and made again as well. Its driver logs every check response from each client with its bucket counts *(Client.java:232-236)*, and those are now read too. In the third, libfaketime left the subscriber's monotonic clock alone (`-exclude-monotonic`), as libfaketime's notes advise for Java, and	A pilot tests instruments, never a prediction.	**Yes, as to the instruments:** the pilot.

No.	Change	Reason	Prompted by results?
D41-7	With this tool every tool of the second October reading that times a message has run. Pepper-Box and xk6-kafka time no message *(D39-7)*. When its runs are home and checked, its key, its service account and its project are deleted.	A tool is run under predictions fixed before its runs.	**No.**
D41-8	The order. The second x86 pair's own list runs it: the install, then for each round T1 to T4 and S1. The list ends by copying the runs off the pair, and the pair stopping itself.	As version 40's.	**No.**

What changed since version 39, and why

Version 39 left two tools of the second October reading unrun, because each builds from source on an older toolchain *(D39-7)*. This version builds and runs both: one that subtracts in unsigned integers, so that a value below zero comes out as about 1.8×10^{19} ms, and one whose statistics stop at the first value below zero and print nothing more.

No.	Change	Reason	Prompted by results?
D40-1	Two tools of the second October reading *(data/tools_audit/batch_11.json)* are run, each at the version whose code that reading read. **Quiver** at the commit it read, ba4e5766 on main, its last release, 0.3.0, being six years and 109 commits older; built as its README builds it, `make build` and `sudo make install`, against Qpid Proton C 0.40.0 from the Apache release, checked against the SHA-512 published beside it, since Ubuntu 22.04 ships Proton 0.22, without the two libraries Quiver's C arrow links, and Quiver's build ends by running that arrow *(Makefile:120-123, 185-187)*; its Java arrows built by Maven 3.9.9 on Java 17. Its broker is the Artemis instance version 39 runs on the broker machine *(tools.sh server36 up artemis)*, whose main acceptor speaks AMQP 1.0 beside Artemis's own protocol on 61616. **The Pravega benchmark** at its v0.13.0 tag, 3807aedd, whose Java sources are those of the master the reading read, 33b79e5f, and which builds against the released Pravega 0.13.0 where master names an unreleased snapshot *(gradle.properties:15)*; built by the Gradle its wrapper names, 4.10.2, fetched and checked against Gradle's published SHA-256 because the wrapper checks nothing, on Java 8, since that Gradle cannot run on 11. Its broker is Pravega 0.13.0's own standalone server on the broker machine *(tools.sh server36 up pravega)*, from the release archive, checked against its published SHA-512, on Java 11, its heap cut to 2 GB from the 4 GB its script asks for, and its data server telling clients the broker's address, as Pravega's own image does *(init_standalone.sh:19)*. Each runs T1 to T4 under the rules every tool ran under, four rounds *(D4-2)*, in folders of its own (`runs/azure/tools_v40`, then `_round2` to `_round4`).	Each is a tool the industry uses, and its sentence in the paper rests on its source alone.	**No.** None of their runs under this plan has been read: the pilot *(D40-6)* ran them to check the instruments.

No.	Change	Reason	Prompted by results?
D40-2	How each is run *(`cloud/azure/tools_run.sh`)*. **Quiver**: its receiving arrow first, then its sending arrow, each a `quiver-arrow` with the qpid-jms implementation, the one of its arrows that keeps a rate *(QuiverArrowJms.java:120-124)*; a queue of the run's own, which Artemis makes on first use; 3,000 messages of 512 bytes at 50 a second. `-duration 0`, since its default of 30 s would end a minute's run halfway, and `-timeout 120`, since its JMS arrow writes its transfers through a buffer it empties only when full or at the end *(QuiverArrowJms.java:180-182, 252, 288)* and its wrapper stops an arrow whose file has not grown for that long, ten seconds by default *(arrow.py:251-258)*: at 50 a second the sender's lines fill the buffer once in ten seconds. What is read: the summary the receiving arrow's wrapper writes, its average, quartiles and nines; the transfers it wrote, read again with their sign; and the sending arrow's count. **The Pravega benchmark**: its End to End mode, one writer and one reader in one JVM, a stream of the run's own in one segment, 512-byte events at 50 a second for 60 s, reporting each second. What is read: its final line, its window lines, and any `ArrayIndexOutOfBoundsException` it prints, with the index it names. **T2's clock**: Quiver's receiving arrow, and the JVM it starts, under libfaketime, as every tool T2 reaches through the C library; the Pravega benchmark by this machine's own clock, stepped back for the run and put back after it, as the Go tools' is *(D15-1)*, since libfaketime slows its JVM *(D40-6)*.	A tool is run as its own documentation runs it, with only the load set to the block's where it can be set.	**As to the instruments only:** the Pravega benchmark's T2 by the machine's clock *(D40-6)*.

No.	Change	Reason	Prompted by results?
D40-3	Their references, which T2 takes its offsets from *(D15-1)*. No client of ours speaks AMQP 1.0 or Pravega's protocol, so each tool is its own reference, run once more at no offset, as PerfKit, YSB and version 39's three were *(D36-3, D37-1, D39-3)*: Quiver's every difference from the transfers its receiver writes, and the Pravega benchmark's median from its final line. Quiver's is run under libfaketime at no offset, since T2 holds its receiver under libfaketime and libfaketime alone lengthens its trip, so that T2's offsets, the trip plus one and plus three milliseconds, are taken from the trip its receiver has under them.	As D39-3 chose for tools no client of ours speaks to.	**As to the instruments only:** Quiver's reference under libfaketime *(D40-6)*.
D40-4	Their predictions, under T-P7, from each one's source. Both are **low**: whole milliseconds of `System.currentTimeMillis()`. Each crosses the delayed direction once *(D25-5)*, Quiver from the broker to its receiver and the Pravega benchmark from Pravega's data server to its reader, so **T1** holds each to a slope of 0.5 to 1.5. **T2**: Quiver's receiving arrow subtracts the sender's stamp, carried in the message, from its own clock, which T2 moves under libfaketime on that arrow and the JVM it starts: **two clocks** *(D34-2)*. **T2-q**: at each offset the transfers its receiver writes hold differences below zero, and its summary's maximum and its average each exceed 10^{18} ms, since its wrapper subtracts in unsigned 64-bit integers *(arrow.py:264-265, 301)*, so that a value below zero becomes 2^{64} less its size, about 1.8×10^{19} ms, which the maximum takes and the average, over a share of the messages, carries; its control's transfers hold none below zero, and neither its maximum nor its average reaches 10^{18} ms: nothing in it wrapped. The Pravega benchmark takes both stamps with `System.currentTimeMillis()` in one JVM *(WriterWorker.java:207-208; ReaderWorker.java:135-141)*, which the offset moves together: **one clock**. **T3** is described. **T4**: what each reports. T2-q holds if it holds in all four rounds *(scripts/block_v40.py)*.	As D39-4 fixed version 39's before their runs.	**In part.** Each is from the reading of each one's source, written down before the pilot. The design held T2-q's control under 1,000 ms. In two of the third pilot's staircase runs nothing reached Quiver's receiver for up to 17 s, after which what had been held arrived at once *(D40-6)*, so a control can reach seconds with nothing below zero. The bound is moved to 10^{18} ms, which only a wrapped value passes, and the control must still hold nothing below zero.

No.	Change	Reason	Prompted by results?
D40-5	S1 *(D37-3, as made there)* on both, four rounds, in the same folders. **S1-q**: Quiver's stepped run's transfers hold at least one difference below zero, and its summary's maximum exceeds 10^{18} ms; its control's hold none, and its maximum does not reach 10^{18} ms. Its average is not predicted under S1: a step catches only the few messages in flight across it. **S1-p**: the Pravega benchmark's stepped run prints no final line and prints an `ArrayIndexOutOfBoundsException`: its one statistics task counts each latency into an array indexed by the latency *(PerfStats.java:123-126, 334-337)*, so the first value below zero throws there, uncaught; the task ends, no window after it and no total is printed, and the program still exits with status 0 *(PravegaPerfTest.java:171)*. Its control prints its final line and no such exception. Each is judged as D37-4 judges, in all four rounds *(scripts/s1_block.py)*.	S1 asks what a tool does when the clock it reads steps back under it.	**In part.** Both were written down before the pilot. S1-q's control was held under 1,000 ms there, and is held under 10^{18} ms for the reason D40-4 gives: S1's runs are in the receiver's namespace, where the stalls happened.

No.	Change	Reason	Prompted by results?
D40-6	The pilots of 12 October 2026, on the second x86 pair, kept as `pilot_v40` to `pilot_v40c`. In the first, both tools built: Qpid Proton C 0.40.0 from its release, matching its published SHA-512, then Quiver at ba4e5766 with its Java arrows, its build's own smoke test passing twice; and the Pravega benchmark at its v0.13.0 tag, with Gradle 4.10.2 matching Gradle's published SHA-256, on Java 8, the JCenter its build names now sending every request on to Maven Central. Pravega 0.13.0's standalone server said it was running within half a minute of being started. Quiver timed 3,000 of 3,000 messages at a median of 1 ms, none at or below zero; with its receiver's clock 5 ms behind, 2,998 of its transfers came out below zero, and its summary gave its median and its maximum as 2^{64} ms, about 1.8×10^{19}, and its average as 1.84×10^{19} ms. Under S1's steps its transfers held 17 differences of −19 and −20 ms, its maximum was 2^{64} ms and its median 1 ms, and its control held none below zero. The Pravega benchmark counted 2,862 events at a median of 1 ms and printed its final line; under S1 it printed one window and then no other and no final line, and its shutdown printed an `ArrayIndexOutOfBoundsException` naming −18, twice, from its main thread and from its shutdown hook. With its JVM's clock 5 ms behind under libfaketime it averaged 19.9 ms against 1.4, and four of its windows held a maximum of 914 to 1,009 ms, its reader's timeout being one second. The second pilot ran each again under libfaketime as T2 would, under libfaketime leaving the monotonic clock alone (`-exclude-monotonic`), and with no libfaketime. Under libfaketime in either form the benchmark's median was 2 ms, against 1 ms without it, and two of its four runs under it again waited out its reader's timeout; Quiver's receiver's usual trip went from 1 ms to 2. libfaketime, not the offset, slows both JVMs. The Pravega benchmark's T2 therefore moves this machine's own clock, as the Go tools' does, and Quiver's reference, from which T2's offsets are taken, is timed under libfaketime at no offset. The third pilot ran T1 and T2 on both so. Quiver's staircase moved its median 1.03 ms per millisecond added. In two of its ten runs, both in the receiver's namespace, nothing reached its receiver	A pilot tests instruments, never a prediction.	**Yes, as to the instruments:** the pilot.

No.	Change	Reason	Prompted by results?
D40-7	Google's Pub/Sub load test framework is the one tool of the second October reading still unrun: it drives Google Cloud's Pub/Sub, a managed service of another provider, and would need a project and billing of its own there. Pepper-Box and xk6-kafka time no message *(D39-7)*.	A tool is run under predictions fixed before its runs.	**No.**
D40-8	The order. The second x86 pair's own list runs it: the two installs, then for each round Artemis's broker up, Quiver's T1 to T4 and S1; Pravega's server up, the Pravega benchmark's T1 to T4 and S1. The list ends by taking Pravega's server down, copying the runs off the pair, and the pair stopping itself.	One broker at a time fits the broker machine's memory beside the servers already there.	**No.**

What changed since version 38, and why

Version 38 left five tools of the second October reading unrun, because each needs a broker the testbed did not run *(D38-7)*. Three of them come as binary releases whose measuring code is the code that reading read, and whose brokers fit on a broker machine one at a time. This version runs those three: one that refuses a value below zero by throwing, one whose maximum cannot show one, and one whose own pacing enters every latency it times.

No.	Change	Reason	Prompted by results?
D39-1	Three tools of the second October reading *(data/tools_audit/batch_11.json)* are run, each from the Apache binary release whose measuring code is the code that reading read, checked against the SHA-512 the Apache archive publishes beside it: Artemis 2.57.0, whose perf package is the same tree as on main at 50c178e5 *(tree 1f448498; HdrHistogram 2.2.2 in both)*; RocketMQ 5.5.1, whose benchmark Producer and Consumer are the same blobs as on develop at 9ea2ccdc; and BookKeeper 4.18.1, the release that carries its benchmark, whose BenchThroughputLatency is the same blob as on master at 773a988d. Each broker runs on the broker machine of the second x86 pair, alone, started before its tool's runs and stopped after them *(tools.sh server36)*: an Artemis instance on 61616 with its separate AMQP, MQTT, STOMP and HornetQ ports left out, since RabbitMQ and Mosquitto hold two of them; RocketMQ's name server and one broker, their heaps cut to 512 MB and 1 GB from the 4 and 8 GB their scripts ask for; and BookKeeper's `localbookie`, ZooKeeper and three bookies in one JVM. Each client runs on the driver, on Java 17. Each tool runs T1 to T4 under the rules every tool ran under, four rounds *(D4-2)*, in folders of its own (`runs/azure/tools_v39`, then `_round2` to `_round4`).	Each is a tool the industry uses, and its sentence in the paper rests on its source alone.	**No.** None of their runs under this plan has been read: the pilot *(D39-6)* ran them to check the instruments.

No.	Change	Reason	Prompted by results?
D39-2	How each is run *(`cloud/azure/tools_run.sh`)*. **Artemis's perf client**, its producer and its consumer as the separate processes its documentation runs them as, from an instance's own `bin/artemis`, which opens the JDK's microsecond clock to it: the consumer first, then the producer with 512-byte messages at 50 a second, 3,000 of them or until ten seconds past the block's minute, whichever comes first, non-persistent as by default; its queue purged before each run; the summary its consumer's `-json` file ends with is read, the transfer time's mean and percentiles in microseconds, the count its histogram kept and the messages it received, with the AMQ132001 warnings its console prints. **RocketMQ's benchmark**, its Consumer and its Producer as two JVMs, launched as its `bin/tools.sh` launches its example classes, since its benchmark scripts pass an option Java 17 refuses; a topic of one queue made for each run; the Producer with one thread and 512-byte messages, stopping at 3,000: it has no rate of its own, each thread sending and waiting for the reply, so it runs at one message a round trip; the Consumer's reports every ten seconds are read, its B2C average weighted by each report's rate and its largest B2C maximum. **BookKeeper's benchmark**, `bin/benchmark writes`, one process: 512-byte entries, the ensemble and quorums of its defaults, no warm-up, 60 s, and `-throttle 50`, its one way to set a rate; the latencies its dump file holds are read, with the 95th and 99th percentiles its log prints.	A tool is run as its own documentation runs it, with only the load set to the block's where it can be set.	**As to the instruments only:** the bound on Artemis's producer *(D39-6)*.
D39-3	Their references, which T2 takes its offsets from *(D15-1)*. No client of ours speaks Artemis's core protocol, RocketMQ's or BookKeeper's, so each tool is its own reference, run once more at no offset, as PerfKit and YSB were *(D36-3, D37-1)*: BookKeeper's every latency from its dump, and the finest Artemis and RocketMQ print of every message, the transfer time's median and the B2C average.	As D36-3 and D37-1 chose for tools no client of ours speaks to.	**No.**

No.	Change	Reason	Prompted by results?
D39-4	Their predictions, under T-P7, from each one's source. **Artemis's perf client** is **high**: microseconds of the wall clock. **RocketMQ's benchmark** is **low**: whole milliseconds. **BookKeeper's benchmark** is **high**: nanoseconds of the monotonic clock. Each crosses the delayed direction once *(D25-5)*, so **T1** holds each to a slope of 0.5 to 1.5. **T2**: Artemis's and RocketMQ's consumers subtract the producer's stamp from their own clock, which T2 moves under libfaketime on the consumer's JVM alone. RocketMQ's is therefore on **two clocks** *(D34-2)*. Artemis's is on two clocks as well, but what D34-2 reads of it – its kept count and its figures – is reported, not predicted, since a delivery refused below zero comes back later and can be recorded above it (T2-a); its own output answers the question in T2-a. BookKeeper's takes both readings in one process on a clock no offset reaches, **one**. **T2-a**: at each offset Artemis's consumer refuses every value below zero, HdrHistogram throwing before it counts it *(AbstractHistogram.java:2397-2400 at 2.2.2)*, logs AMQ132001 for it, and the message, rolled back, is delivered again, up to ten attempts *(AddressSettings.java:54)*: so it logs AMQ132001 at least once, receives more messages than were sent, and its summary still reads success; its control logs no AMQ132001 and receives no more than were sent. Its histogram's count and figures are reported, not predicted: a delivery late enough comes out above zero and is recorded, longer by its retries. **T2-r**: at each offset RocketMQ's B2C average falls by more than half the offset from its control's; its maximum, which starts at 0 and rises only past it, reset at each report *(Consumer.java:112, 171-177, 246-255, 265)*, so that it cannot show a value below zero, is reported beside it and not judged. **T3** is described. **T4-b**: at 50 entries a second BookKeeper's benchmark takes its start time just before the call in which its own rate limiter blocks for the next permit *(BenchThroughputLatency.java:187-188; LedgerHandle.java:249-250, 1716-1718)*, so every latency it keeps carries that wait, about a fiftieth of a second: its median, from its own dump, is above 15 ms. Each of T2-a, T2-r and T4-b holds if it holds in all four rounds *(scripts/block_v39.py)*.	As D38-4 fixed freeze 35's before their runs.	**In part.** Each is from the reading of each one's source, written before the pilot. A draft of this row, written while the pilot ran and before any of its runs with a moved clock was read, also gave D34-2's answer for Artemis as two; the design written before the pilot had left Artemis's figures under T2 unpredicted, for the reason given, and the row is brought back to it. The design also had T2-r say that every report whose average is below zero prints a maximum of 0. That held only if every value went below zero, and the messages slower than the offset stay above it: the pilot printed a maximum of 11 ms beside an average of −3.09 ms *(D39-6)*. The clause is dropped, not turned round.

No.	Change	Reason	Prompted by results?
D39-5	S1 *(D37-3, as made there)* on Artemis's perf client, four rounds, in the same folders, its producer bounded by time as well as by count, ten seconds past the block's minute. **S1-k**: under the steps its producer stops before its count – a step inside one of its own timed sends makes a difference below zero that its histogram refuses, in a thread that catches only JMS exceptions *(SkeletalProducerLoadGenerator.java:143, 148)* – so in its stepped run it sends fewer than the 3,000 it was asked to and its summary still reads success; in its control it sends all 3,000. Judged as D37-4 judges, in all four rounds. Its consumer's AMQ132001 warnings are reported beside it. RocketMQ's benchmark and BookKeeper's are not in S1: RocketMQ's average takes in the few values a step makes below zero and its maximum cannot show them, and BookKeeper's clock is monotonic and its filter, keeping only values above zero, would hide one anyway.	S1 asks what a tool does when the clock it reads steps back under it.	**Yes.** The pilot's stepped run of Artemis's perf client stopped after ten messages and printed no error *(D39-6)*. This clause replaces the one written before the pilot, which asked whether its consumer would refuse the values the steps made below zero, a question a stopped producer leaves unanswerable. It is judged on four new rounds.

No.	Change	Reason	Prompted by results?
D39-6	The pilots of 11 October 2026, on the second x86 pair, kept as `pilot_v39` to `pilot_v39c`. In the first, the three clients installed, each archive matching its published SHA-512. Artemis's broker started on the broker machine, and the step that started it reported that it never answered: it waited for the port in a loop whose status was its last test, false whenever it had waited at all, and it now reports success outright. Artemis's perf client then timed 3,000 of 3,000 messages at a median of 1.09 ms. With its consumer's clock 5 ms behind under libfaketime, it received 11,132 messages for the 3,000 sent, logged AMQ132001 8,132 times, kept all 3,000 in its histogram at a median of 1.41 ms, and its summary read success. Under S1's steps its producer sent ten messages and then nothing, printed no error, and waited for the rest of its count until the run's limit stopped it, eight minutes later. RocketMQ's broker did not start, the broker machine having no `unzip`; and the step that started BookKeeper's never returned, because the backgrounded command sent only its last program's output to its log. Both are changed. The second pilot ran what the first could not. RocketMQ's Producer sent 3,000 messages at 800 to 1,000 a second, one at a time, and its Consumer printed a B2C average of 1.79 ms and a maximum of 111 ms; with its consumer's clock 5 ms behind, an average of −3.09 ms and a maximum of 11 ms. BookKeeper's benchmark completed 3,001 adds at a median of 25.2 ms, the fastest 22.3 ms, its 95th and 99th percentiles 28.3 and 36.5 ms. Under S1's steps Artemis's producer stopped again, and neither its `-duration` nor the run's limit stopped its JVM, which the limit left running; that run's clock, stepped back 35.7 s in all, could not be put forward by the step and chrony brought it back. Its producer is now stopped by `timeout` twenty seconds past the block's minute, which signals the JVM itself, and what it sent is read from its own summary or, where it was stopped before writing one, from its intervals' rates. The third pilot ran S1 on Artemis's perf client so: its control sent all 3,000 messages; its stepped run sent 6, said success, and ended after 93 s, the clock put back within 0.01 ms of the PTP clock. The second pair's driver had run out of disk, most of it held by the kernel builds of the A2 sessions; clearing its	A pilot tests instruments, never a prediction.	**Yes, as to the instruments:** the pilot.

No.	Change	Reason	Prompted by results?
D39-7	Quiver and the Pravega benchmark are not run in this version: each builds from source on an older toolchain, Quiver against a Qpid Proton newer than Ubuntu 22.04 ships and the Pravega benchmark on Java 8 with Gradle 4.10.2, and each is for a later version with predictions of its own. Google's Pub/Sub load test framework needs Google Cloud's Pub/Sub; Pepper-Box and xk6-kafka time no message.	A tool is run under predictions fixed before its runs.	**No.**
D39-8	The order. The second x86 pair's own list runs it: the three installs, then for each round Artemis's broker up, Artemis's T1 to T4 and S1, its broker down; RocketMQ's broker up, its T1 to T4, down; BookKeeper's up, its T1 to T4, down. The list ends by copying the runs off the pair and the pair stopping itself.	One broker at a time fits the broker machine's memory beside the servers already there.	**No.**

What changed since version 37, and why

Version 37 ran none of the twelve tools the second October reading read at source, because their predictions were not written *(D37-6)*. The author decided on 11 October to run every one the testbed's servers can serve. Four can be, with no broker added: two time a message on a clock that cannot run backwards, and two time it on the wall clock, one keeping what comes out below zero and one counting it. This version runs those four.

No.	Change	Reason	Prompted by results?
D38-1	Four tools of the second October reading *(`data/tools_audit/batch_11.json`)* are run, each cloned at the commit that reading read and built as its own repository builds it, on the driver of the first x86 pair: mqtt-benchmark at f46efba1 *(`go build`)*; Sarama's kafka-producer-performance at 36f7a14e *(`go build ./tools/kafka-producer-performance`, Go 1.26.0 as its module asks)*; SBK at 39af60da, release 10.8 *(its Gradle wrapper's `:installDist`, on Eclipse Temurin 25.0.4.1+1, checked against Adoptium's SHA-256, since its build stops on any other Java)*; and kgo-repeater at 6567f2f5 *(`go build`, Go 1.23.11 as its module asks)*, copied to the broker and checked there by its SHA-256. Each runs T1 to T4 under the rules every tool ran under, four rounds *(D4-2)*, in folders of its own (`runs/azure/tools_v38`, then `_round2` to `_round4`).	Each is a tool the industry uses to time messages, and its sentence in the paper rests on its source alone.	**No.** None of their runs under this plan has been read: the pilot *(D38-6)* ran them to check the instruments.

No.	Change	Reason	Prompted by results?
D38-2	How each is run *(`cloud/azure/tools_run.sh`)*, at the block's rate, 50 messages a second for 60 s, 512 bytes each. **mqtt-benchmark**, against the broker's Mosquitto: its model is many clients, each publishing once a second, since its interval between a client's messages is in whole seconds, so it runs as 50 clients of 60 messages each, started across one second, at QoS 1, its default; one process, each client timing publish to acknowledgment in one goroutine; its JSON results are read: its minimum, its maximum and the mean of every message. **Sarama's tool**, on its defaults but the load: an asynchronous producer, acknowledgment from the leader and its assumed Kafka version 0.8.2.0, which is why it speaks to the 3.9.1 broker, Kafka 4 having removed those protocol versions; 3,000 messages, 50 a second, which it sends together at each second's tick; its last metrics line is read, the mean and 50th to 99.9th percentiles of the latency of every request on every connection, in whole milliseconds. **SBK**, its Kafka driver with one writer and one reader in one process, the one mode in which it times a message end to end, a reader on its own timing its poll; 50 records a second for 60 s, its default time unit, whole milliseconds of the wall clock; its total line is read, with the latencies it called invalid. **kgo-repeater**, as two repeaters each run as its README runs one: A, on the broker, produces about 50 messages a second and echoes what it consumes, stamping every send with the broker's wall clock; B, on the driver, in remote mode in a group of its own, is never activated, consumes everything A sends and subtracts A's stamp from its own wall clock. B's figures are reset once A has run 10 s and read from its status after the block's minute: the median, 90th and 99th percentiles of its end-to-end latency, in microseconds. Every tool has a topic of one partition made for each run.	A tool is run as its own documentation runs it, with only the load set to the block's.	**As to the instruments only:** SBK's rate and its topic *(D38-6)*.

No.	Change	Reason	Prompted by results?
D38-3	Their references, which T2 takes its offsets from *(D15-1)*: our own Kafka client for the three Kafka tools, on the 3.9.1 broker for Sarama's tool, and our own MQTT client for mqtt-benchmark, which times publish to delivery where the tool times publish to acknowledgment; both cross the delayed direction once, to the same broker.	A reference of our own for each, as D33-6 chose one for the five of freeze 30.	**No.**
D38-4	Their predictions, under T-P7, from each one's source. **mqtt-benchmark** is **high**: Go's monotonic clock, in nanoseconds, printed in full. **Sarama's tool** is **low (chops)**: each request's latency truncated to whole milliseconds before it is kept. **SBK** is **low**: whole milliseconds of the wall clock. **kgo-repeater** is **high**: microseconds of the wall clock. Each crosses the delayed direction once *(D25-5)*: the acknowledgment, the produce response, the record read by SBK's reader, and the record read by B all travel toward the driver; so **T1** holds each to a slope of 0.5 to 1.5 per crossing. **T2**: mqtt-benchmark and Sarama's tool take both readings in one process on a clock no step reaches, and SBK both in one JVM, so each **times against one clock** *(D34-2: one)*; kgo-repeater takes them on two machines, the driver's clock is the one moved *(D15-1, a Go tool)*, and it **keeps every value**: B's median, 90th and 99th percentiles all fall by the offset, below zero where the offset exceeds the trip *(D34-2: two)*. **T3** is described, as for every tool, and **T4** is what each reports, read as D38-2 says.	As D35-2 fixed the October tools' classes before their runs.	**In part.** A draft also predicted that Sarama's tool would print a 50th percentile of 0.0 ms at the zero step. That rested on its requests taking less than a millisecond on this path, which is a property of the path and not of the tool, and its source says only that each is truncated to whole milliseconds. The pilot printed 1.0 ms *(D38-6)*. The clause is dropped, not turned round.

No.	Change	Reason	Prompted by results?
D38-5	S1 *(D37-3, as made there)* on three of them, four rounds, in the same folders. **S1-g**: SBK counts a value below zero: its stepped run's invalid latencies number at least one, its control's none. **S1-h**: mqtt-benchmark never meets one: its minimum is above zero in both runs, and its mean in its stepped run is within 1 ms of its control's. **S1-i**: Sarama's tool never meets one: its 50th and 99th percentiles in its stepped run are within 1 ms of its control's, as Kafka's own tool's are judged *(S1-f)*. Each holds if it holds in all four rounds, as D37-4 judges. kgo-repeater is not in S1: one repeater keeps such a value only inside its reservoir and prints no low percentile, minimum or count, so nothing it prints could show it; its T2 across two machines asks what it does with one.	S1-g asks the one tool of the four that counts what it keeps out to count; S1-h and S1-i ask the two on a monotonic clock whether the steps reach them.	**No.**

No.	Change	Reason	Prompted by results?
D38-6	The pilots of 11 October 2026, on the first x86 pair, kept as `pilot_v38` to `pilot_v38c`. In the first, each of the four installed alone *(`tools.sh install-for`)* at its commit, Sarama's tool and kgo-repeater with the Go their modules ask for, 1.26.0 and 1.23.11, and SBK in two minutes on Temurin 25; a Java program printing `System.currentTimeMillis()` under an offset of −5 s read 5 s early, so libfaketime reaches the Java SBK runs on. Its T4 runs stopped: the receiver's namespace had not been built again after the pair was started, and the stage will not run a tool outside it, so the pair's session was started again *(`cloud/azure/session.sh`)*. kgo-repeater, run with this machine's clock 5.007 ms early, printed its end-to-end median, 90th and 99th percentiles as −3.97, −3.62 and −2.38 ms, of the 2,904 messages it consumed. The second pilot ran T4 for the four and S1 for three. mqtt-benchmark timed 3,000 of 3,000 publishes, at a mean of 0.66 ms; Sarama's tool printed a 50th percentile of 1.0 ms and a 99th of 24.7 ms, the latencies of its first requests, up to 29 ms, staying in its reservoir to the end; kgo-repeater's median was 1.11 ms. Under the steps, mqtt-benchmark's minimum and mean stayed within 0.01 ms of its control's, and Sarama's 50th and 99th percentiles within one printed step. SBK printed nothing its reader could read, for two faults of the instrument, both now changed. A throughput of 0, which its help reads as writing its `-records`, lifted its limit instead, so it wrote 256,280 records a second and every latency was its queue's: its throughput is now left at its default, under which `-records` is the writer's rate. And its line names its unit `ms`, where its reader looked for another name. It also met its topic before the broker served it, and the topic is now made before it starts, as for the other tools. The third pilot ran SBK alone: 3,000 records at a median of 5 ms, none invalid; under the steps its control counted no invalid latency and its stepped run 58, and its median in the stepped run was 21 ms, which no prediction here concerns. SBK ends every run by interrupting its own flush, after its total line. Every pilot run ended with the clock within 0.06 ms of the PTP clock and chrony running.	A pilot tests instruments, never a prediction.	**Yes, as to the instruments:** the pilot.

No.	Change	Reason	Prompted by results?
D38-7	The other eight tools of the second October reading are not run in this version. Pepper-Box and xk6-kafka time no message. Google's Pub/Sub load test framework needs Google Cloud's Pub/Sub, which the testbed does not have. The Pravega benchmark, RocketMQ's benchmark, Artemis's perf, Quiver and BookKeeper's benchmark each need a broker the testbed does not run; which of them a later version runs, with its broker, is for that version.	A tool is run under predictions fixed before its runs, against the broker it is written for.	**No.**
D38-8	The order. The first x86 pair's own list runs it: the servers, the four installs, then for each round each tool's T1, T2, T3 and T4, then S1 for each of the three. The list ends by copying the runs off the pair and the pair stopping itself.	As version 37's block ran.	**No.**

What changed since version 36, and why

Version 36 left PerfKit Benchmarker's messaging benchmark unrun, because it needs a managed queue the testbed does not have, and that was the author's decision. The author decided on 11 October to make one. Version 35 also left open what three of the October tools do with a value below zero: in one process, or on one machine's clock, no constant offset can make one. This version runs PerfKit and the test that can.

No.	Change	Reason	Prompted by results?
D37-1	PerfKit Benchmarker's messaging benchmark is run, at commit e5fd49d7, with the Azure client its requirements name *(azure-servicebus 7.15.0, fingerprinted with its other packages)*, against an Azure Service Bus queue of the Standard tier made for it in the author's subscription and deleted when the runs are home. Its end-to-end scenario sends 100 messages of 10 bytes, with no warm-up, as `cloud/azure/tools_run.sh` invokes it from the driver of the first x86 pair. It runs T1 to T4 under the rules every tool ran under, four rounds *(D4-2)*, in folders of its own (`runs/azure/tools_v37`, then `_round2` to `_round4`). T1 is its zero step alone, with its reference: the staircase delays the broker's traffic toward the driver, and PerfKit's messages go to the managed queue, whose path the staircase cannot reach. Its reference is itself at no offset, every message it timed *(D36-3, as YSB's)*, because no client of ours speaks to that queue.	PerfKit is how a cloud's own benchmark suite times messages, and its sentence in the paper rests on its source alone.	**No.** None of its runs under this plan has been read: the pilot *(D37-5)* ran it to check the instruments.
D37-2	Its predictions, under T-P7, from its source. Its class is **low**: whole milliseconds, floor-divided from nanoseconds. Its publisher and its receiver are two processes of one program on the driver, both reading the driver's wall clock, and T2's offset moves the clock of the whole program, so in T2 it **times against one clock** *(D34-2: one)* and keeps every value. Its interval is entered in the tables as two processes on one clock. T1 tests nothing of it beyond the zero step, and T3 is described, as for every tool.	As D35-2 fixed the October tools' classes before their runs.	**No.** From its source.

No.	Change	Reason	Prompted by results?
D37-3	S1, the step test, added to the tools block. Each tool runs once as it is, the control, and once while `scripts/clock_stepper.py` steps the driver's wall clock back, chrony stopped, until the tool ends *(`cloud/azure/tools.sh, s1`)*. A step is 20 ms, longer than any of these tools' trips on the pair, and 50 ms for PerfKit, whose trip to the managed queue is some 20 ms. The gaps between steps are drawn evenly between 0.1 and 0.3 s from a seed written down with the run. The clock is measured against the hypervisor's PTP clock before and after the steps, put forward by what that measurement says they took, and chrony started again. Every step is written down: when it was asked for on the monotonic clock, the wall clock just before and just after. Five tools, four rounds, in the same folders as PerfKit's T1 to T4: PerfKit; Strimzi Canary and Xinfra Monitor, each a producer and a consumer in one process; RabbitMQ Stream PerfTest as one instance, its producer and its consumer in one process; and Kafka's end-to-end tool, the control, which times with a monotonic clock.	T2's constant offset moves both of a tool's readings together when they come from one clock, so it can never show what such a tool does with a value below zero. A step of the clock between the two readings can, and a user of Stream PerfTest met one during a cluster's rolling restart *(its issue 418)*.	**No.**

No.	Change	Reason	Prompted by results?
D37-4	S1's predictions, from each tool's source, and how they are judged. **S1-a** *(the instrument)*: a stepped run is judged only if its steps were made, each landing within 5 ms of the step asked for, and the clock was put back within 5 ms of the PTP clock. **S1-b**: PerfKit keeps a value below zero: its stepped run prints at least one sample below zero, its control none. **S1-c**: Strimzi Canary logs it: its stepped run logs at least one duration below zero, its control none. **S1-d**: Stream PerfTest drops it, uncounted: in its stepped run it keeps fewer latencies than it consumes messages, in its control as many. **S1-e**: Xinfra Monitor's metrics library pins it to its percentiles' floor and says so only at debug level: that line, made part of its console in both runs *(its logging configuration, with one logger at debug)*, appears at least once in its stepped run and never in its control. **S1-f**: Kafka's end-to-end tool does not see the steps: its median and 99th percentile in its stepped run are within 1 ms of its control's. Each holds if it holds in all four rounds; a round that cannot judge it leaves it undecided, as K1's are judged *(D35-7)*. Reported and not judged: how many values below zero each run showed, beside how many the steps made, the tool's rate and its median trip would catch.	S1-b to S1-e say what each tool's own figures show of a value below zero that the clock, not the network, made; S1-f says the steps reach only what reads the wall clock.	**In part.** The pilots *(D37-5)* showed PerfKit's samples below zero, the canary's logged durations below zero and the latencies Stream PerfTest did not keep, and Xinfra Monitor's line. The predictions are written from each tool's source and judged on four new rounds.

No.	Change	Reason	Prompted by results?
D37-5	The pilots of 11 October 2026, on the first x86 pair, kept as `pilot_v37` to `pilot_v37c`. In the first, PerfKit installed alone *(`tools.sh install-for`)*, timed 100 messages to the queue at a median of 21 and 22 ms, and took its reference. Under steps of 50 ms once a second, its stepped run printed two samples below zero, −29 and −14 ms. The canary and Stream PerfTest, which send every 20 ms on a timer, showed none, because one second is fifty of their periods, so every step fell at the same moment of their cycle, between two messages: the gaps are now drawn *(D37-3)*, and a step comes about five times a second, so that a run can catch some fifteen messages in flight where the pilot's could catch three. Xinfra Monitor's figures cannot show such a value at all, since it enters only its average, among thousands, so its library's own line is made part of its console *(D37-4)*. The second pilot ran S1 as D37-3 makes it on those three tools: the canary logged 12 durations below zero, of −19 and −20 ms, and Stream PerfTest kept 2,796 of the 2,810 latencies of messages it consumed. Two instruments changed after it. Xinfra Monitor's line went to a log file that every run shares, because the logger it inherited writes there and passes nothing on, so it now has a console of its own; and the clock, put forward by the steps asked, came back up to 5 ms behind after 509 steps, because each set lands a few microseconds late, so it is now put forward by what the PTP clock measured. The third pilot ran Xinfra Monitor alone, to check both: its console carried 58 values pinned to the floor, from −20 to −12 ms, and its control's none, while its 99th percentile read 2 ms in both; and the clock, put forward by the 10,345 ms the PTP clock measured where the steps asked for 10,340, came back within 0.05 ms.	A pilot tests instruments, never a prediction.	**Yes, as to the instruments:** the pilots.
D37-6	The second October reading *(`data/tools_audit/batch_11.json`)* read twelve more tools at source. This version runs none of them; which can run on the testbed's servers is for a later version, with predictions of its own.	A tool is run under predictions fixed before its runs, and those are not written yet.	**No.**

No.	Change	Reason	Prompted by results?
D37-7	The order. The first x86 pair's own list runs it: the servers, PerfKit's install, then for each round PerfKit's T1, T2, T3 and T4, then S1 for each of the five tools. The list ends by copying the runs off the pair and the pair stopping itself; the queue is deleted when the runs are home.	As version 35's block ran.	**No.**

What changed since version 35, and why

Version 35 left two of the October reading's tools for a version of their own. This one runs the Yahoo Streaming Benchmark, which needs a stream processor and its own way of being read, and says why PerfKit Benchmarker's messaging benchmark is still not run.

No.	Change	Reason	Prompted by results?
D36-1	The Yahoo Streaming Benchmark is run, at its last commit, d1aaf6bb, as its `stream-bench.sh` runs its Flink test, on what that script installs: Flink 1.11.0 for Scala 2.12, and the generator its README builds with Leiningen *(2.11.2, fingerprinted; the README names no version)*. It runs T1 to T4 under the rules every tool ran under, four rounds *(D4-2)*, in folders of its own (`runs/azure/tools_v36`, then `_round2` to `_round4`), on the second x86 pair, beside version 35's block on the first rather than after it: the same machine type and processor *(D8as_v6 and D2as_v6, AMD EPYC 9V74)*, its driver on the stock Azure kernel 6.8.0-1064 where the first's is on 6.8.0-1065. Each tool is judged against its own reference on its own pair, and the paper says which pair ran it.	It is the October reading's one tool of a stream processor, its notebook drops a window whose latency comes out below zero, and users have reported such values since 2018 *(its issue 36)*. Its sentence in the paper rests on its source alone.	**No.** None of its runs under this plan has been read: the pilot *(D36-6)* ran it to check the instruments.

No.	Change	Reason	Prompted by results?
D36-2	Its servers. Its generator, through clj-kafka 0.3.2, is Kafka 0.8.2's Java producer, whose requests Kafka 4 removed, so it uses the Kafka 3.9.1 broker version 35 installed for the canary *(D35-5)*, which still accepts them; its Flink job's client, kafka-clients 2.4.1, talks to it too. Its generator writes to the topic `ad-events` whatever its configuration says *(core.clj:98 and 202)*, so that is its topic, deleted and made again before each run. Its windows go to the broker's Redis, which its generator's new setup clears first, as its script's does; nothing else uses that Redis while a tool runs.	Its own script stands up a Kafka and a Redis of its own on one machine; on this testbed the servers are on the broker, where every tool's are.	**No.**
D36-3	Its reference is YSB itself, run once more at no added delay at T1's zero step, and its trips are every window that run wrote, less the window, sign kept: what YSB wrote, not what its notebook kept *(D36-6)*. T2 takes its offsets and its predictions from them exactly as it takes them from a client's trips *(D15-1)*.	T2 needs a reference *(D25)*, and no client of ours computes a window's latency: a window's last update depends on the processor's flush, once a second, which no message client sees.	**No.**
D36-4	How it is run and read: the addendum at the end *("YSB")*. In short: one JobManager and one TaskManager with four slots, the TaskManager told its address and each started once the one before it answers, its job at the parallelism its configuration asks for *(one host by four cores)*, its generator at its default 1,000 events a second for two minutes of its ten-second windows, then its own collection of each window's last update into `updated.txt`; read as its notebook reads it, keeping the windows at or above ten seconds and subtracting the ten seconds.	Each tool is run as its own documentation runs it and read where it reports, as every other was; YSB reports through its notebook.	**No.**

No.	Change	Reason	Prompted by results?
D36-5	Its predictions, under T-P7, from its source and the pilot's instruments. Its class is **low**: whole milliseconds, a window's last update written at the TaskManager's flush once a second. **T1**: its figures do not follow the staircase *(a slope outside 0.5 to 1.5)*: adding up to 2 ms moves a window's last update only where the last event of the window is pushed past a flush, about 2 ms in 1,000. **T2**: its generator stamps the events and its TaskManager writes the time, so its two stamps come from two clocks *(D34-2: two)*, and it **drops the negatives** and keeps a zero *(its notebook keeps a window at or above the window's length)*, uncounted. That a share of its windows comes out below zero with no clock moved *(D36-6)* is reported, round by round, and not judged.	T1 tests whether a tool can report a millisecond; a tool whose reports are set by a once-a-second flush is predicted not to, and the prediction says so before its runs.	**No.** From its source.
D36-6	The pilot of 11 October 2026, on the second x86 pair, kept in `runs/azure/collected/matched-b/` as `pilot_v36` to `pilot_v36c`. What it found, each now changed: the pair's disk had 30 MB free, most of it taken by the A2 sessions' kernel builds, and the whole install failed for want of room, so a tool can be installed alone *(`tools.sh install-for`)*, and nothing else on the disk was touched; in the receiver's namespace the machine's resolver is out of reach, Flink's TaskManager could not find its own address and gave up, and the job waited for slots while the generator ran, so the TaskManager is told its address and started once the JobManager answers; the generator ignores the topic its configuration names *(D36-2)*. And one thing about YSB itself, not changed: at its default 1,000 events a second a campaign's window holds about thirty views, the last of them on average some 300 ms before the window ends, and the flush once a second can write the window before it ends, so 301 of 1,200 windows came out below zero with no clock moved, and its notebook dropped them; with the TaskManager's clock half a second behind it dropped 783 of 1,300, and the windows it kept averaged 249 ms after their window where the control's averaged 557.	A pilot tests instruments, never a prediction.	**Yes, as to the instruments:** the pilot.

No.	Change	Reason	Prompted by results?
D36-7	PerfKit Benchmarker's messaging benchmark is not run in this version. It times a cloud provider's managed queue, which this testbed does not have, and running it would create one in the author's subscription; in the arrangement it runs, its publisher and its receiver are processes of one program on one machine, so T2 could not move one clock without the other, and T1's delay cannot reach a managed service's path. Its sentence in the paper rests on its source.	A run that could answer only "one clock" and that needs a service created for it is the author's decision, not the plan's.	**No.**
D36-8	The order. The second x86 pair's own list runs it: the servers, its install, then for each round its T1, T2, T3 and T4. The list ends by copying the runs off the pair and the pair stopping itself.	As version 35's block runs on the first.	**No.**

What changed since version 34, and why

The paper's survey of the industry's tools read the source of six message tools the plan had never listed. Their claims in the paper rested on that reading alone. This version runs the four that can run on this testbed through the block every other tool ran, under predictions fixed from their source before any of their runs, and adds an experiment on the one tool the survey says never measures a value below zero.

No.	Change	Reason	Prompted by results?
D35-1	Four message tools that the reading of 10 and 11 October found, and the plan had never listed, are run *(`data/tools_audit/batch_10.json`)*: Xinfra Monitor, LinkedIn's Kafka monitor, at release 2.5.16 *(commit 043db641)*; Strimzi Canary at its last commit, f3812fb5, the repository being archived; Trogdor, Kafka's own test framework, as the Kafka 3.7.1 release ships it, the build the sixteen's Kafka tools come from; and RabbitMQ Stream PerfTest at release 1.8.0, its jar checked against the publisher's checksum. Each runs T1 to T4 as the sixteen did, under the same rules *(D15-1 to D15-5, D23-1, D25-1 to D25-5, D26-2, D26-4, D34-1, D34-2)*, four rounds each *(D4-2)*, on the first x86 pair *(D19-2)*, in folders of their own (`runs/azure/tools_v35`, then `_round2` to `_round4`).	Each times a message through a broker, as the paper's survey counts message tools, and each one's sentence in the paper rests on a reading of its source alone. A claim read in the source and never run is a claim about code, not about what the tool reports. Two of the four take their two timestamps in two processes as run here, which is where a value below zero can arise.	**No.** None of the four has run under this plan, and nothing it would report has been read: the pilot *(D35-8)* ran each once to check the instruments. Prompted by the reading of the paper's table of tools, 10 and 11 October 2026.

No.	Change	Reason	Prompted by results?
D35-2	Their predictions, fixed from each one's source before any of its runs, under T-P7 *(every read-first tool behaves as its class predicts, in T1 and T2)*. All four are **low**: each subtracts two readings of a millisecond wall clock. Each crosses the delayed direction once *(D25-5)*, so T1 holds it to a slope of 0.5 to 1.5. In T2, Xinfra Monitor and Strimzi Canary take both readings in one process and **time against one clock** *(D34-2: one)*; Trogdor takes them in two agents and **replaces the negatives with zero** *(its timestamp processor puts every value below its histogram's floor into the lowest bucket, TimestampRecordProcessor.java:88-89)*; Stream PerfTest, as two instances *(D35-4)*, **drops the negatives** and keeps a zero *(a test of latency ≥ 0 with no else branch and no counter, DefaultPerformanceMetrics.java, since June 2026)*.	As D33-7 fixed YCSB's and nats bench's classes before their runs. Two of the four can be asked whether they drop or replace a value below zero; the other two can be asked only whether their figures move with a moved clock, which in one process they cannot.	**No.** From the reading of each one's source.
D35-3	How each is run, and which of its figures is read: the second addendum at the end. Xinfra Monitor runs as its README runs it, one JVM holding a single-cluster monitor that produces to and consumes from a topic of its own; its configuration requires a ZooKeeper address, which no code at this commit contacts, and is given one that does not exist. Strimzi Canary runs standalone as its README configures it, one process, at a tick of 20 ms and told the cluster has one broker. Trogdor runs as its README describes, a coordinator and agents given workloads as JSON specifications, with its producer and its consumer on two agents, the arrangement its `producerNode` and `consumerNode` exist for; the agent that consumes is the one wrapped.	Each is run as its own documentation runs it and read where it reports a latency, as the sixteen were. Where a tool reports one quantity in two places, which one is read is fixed here, before any run.	**No.**

No.	Change	Reason	Prompted by results?
D35-4	Stream PerfTest runs as two instances, a consumer and a producer *(`--producers 0` and `--consumers 0`)*, the consumer wrapped and started first on a stream of its own. Only the consumer's lines hold a latency, and only they are read. The consumer also serves its metrics *(`--prometheus`, a documented option)*, read once the producer has finished: its latency timer's count is how many latencies it kept, and the sum over it is its average.	Its source stamps each message with the wall clock "for interoperability with other tools and also across different processes" *(StreamPerfTest.java:1315-1316)*, and its options accept a producer or a consumer alone. Its filter of values below zero was added in June 2026 after a user saw one *(rabbitmq-stream-perf-test issue 418)*. Run as one instance it could answer only "one clock", as the OpenMessaging Benchmark could outside the distributed mode its end-to-end latency is designed for *(D33-4)*.	**No.** From its source, before any of its runs.
D35-5	Strimzi Canary runs against a Kafka 3.9.1 broker of its own on the broker machine, installed from Apache's release archive *(checked against its SHA-512)* and set as the study's 4.1.1 broker is set but for its ports *(19094 and 29094)*. Its reference runs against the same broker.	Its last commit pins the Sarama client 1.38.1, which commits consumer offsets with version 1 of Kafka's OffsetCommit request, and Kafka 4.0 removed that version. Against the study's 4.1.1 broker in the pilot, the broker closed the canary's connection at every commit, once a second, and each reconnection held its consumer for 2.1 s: its median latency read 568 ms and was its reconnections. Kafka 3.9 is the last release that accepts the request, so it is the newest broker the tool as built can use. It is installed natively, as the block's other servers are *(D15-6)*.	**Yes, as to the instruments:** the pilot *(D35-8)*. No prediction.
D35-6	**K1**, a run of Kafka's own end-to-end tool against an observer. A copy of EndToEndLatency from the 3.7.1 release, changed only to write down for each record its offset and the wall-clock instants at which it began, at which `send().get()` returned its acknowledgment and at which its poll returned *(`external/kafka_e2e`, with the diff)*, runs beside an observer: a consumer on the same host and the same wall clock, at real-time priority in every cell, that long-polls the topic from its end and writes down when each offset became readable to it. Paired by offset, each record gives S, its arrival less its acknowledgment, and $S < 0$ is a record a consumer could read before the tool had its acknowledgment. Four cells, as T3's: idle and at 88% load, the tool at normal and at real-time priority *(go-first)*; 3,000 records of 512 bytes with `acks=1` in each, as the tools block invoked the tool. Six rounds.	The tool polls for each record only after its acknowledgment *(EndToEndLatency.java:110-114)*, so it cannot report a record earlier than its acknowledgment however early the record was readable: the receipt before the acknowledgment that this paper is about is held at zero by the order of two calls, and nothing in its output says so. The paper says it never measures such a value; K1 measures how often there was one to measure.	**No.** From the tool's source.

No.	Change	Reason	Prompted by results?
D35-7	K1's predictions and how they are judged. **K1-a** *(the instrument)*: in every run, every record's traced elapsed time covers its own wait for the acknowledgment; a run where it does not is not judged. **K1-b**: at 88% load with the tool at normal priority, some records are readable before the tool has their acknowledgment. **K1-c**: load raises that share: the 88% cell's share at normal priority exceeds the idle cell's. **K1-d**: go-first takes it back: at 88% load, the share with the tool at real-time priority is below the share at normal priority. Each of K1-b to K1-d holds if it holds in all six rounds *(a one-sided sign test: 1 in 64 by chance)*. A run is judged only where the observer read at least 99% of the records the tool traced; one that read fewer is reported and not judged. For the early records, how much later than the observer the tool reported them, and the tool's own printed figures beside the observer's, are reported and not judged.	K1-b says the case the tool cannot see occurs. K1-c and K1-d say it occurs for the reason this paper gives: the tool's thread waiting for a processor while the observer, at real-time priority, does not.	**In part.** The pilot's two rounds *(D35-8)* showed shares of 2.6 and 2.9% idle, 27 and 40% at 88% load, and 0 and 0.16% at 88% with go-first. The predictions are directional, were written after them, and are judged on six new rounds.

No.	Change	Reason	Prompted by results?
D35-8	The pilot of 11 October 2026. On the first x86 pair each of the four was built and run in T4's arrangement until it ran as designed, and K1 once in each cell, twice; Trogdor and Stream PerfTest also ran with the consuming process's clock 5 ms behind. Its runs test no prediction and none is analysed; they are kept in `runs/azure/collected/matched/`, as `pilot_v35` to `pilot_v35d`, and the readers' tests are cut from them. What it found, each now changed: Xinfra Monitor ran under `timeout`, which stopped `sudo` and not the JVM in the namespace, and the monitor ran on for twenty minutes through every later run, so it is now stopped by a pattern only its run carries, and every run first stops any tool process an earlier run left and writes it down; the canary's connection closed at every offset commit *(D35-5)*; Trogdor's consume task reports its status only once it has ended, so it is given a fixed end and asked after it; Stream PerfTest's default leader locator needs RabbitMQ 3.10, the broker's is 3.9.27, and the tool at 1.8.0 accepts only one other value, `client-local`, which on one node places the stream where the default would; K1's observer was killed with its buffer full and lost the last 565 records of every cell, so it now stops by itself and writes after every poll, and the judge leaves out a torn row and counts it; with its consumer's clock 5 ms behind, Stream PerfTest printed a median of 0 ms for every second while it consumed fifty messages a second and kept none, so its metrics page is read for what it kept, and where it kept fewer than it consumed a second of four zeros is not read as a measurement; and Xinfra Monitor prints its percentiles as one-millisecond buckets, at a width of 5,000 over 5,002 ms, so their step is read as 1 ms and not from their digits.	A pilot tests instruments, never a prediction *(Section "What the pilot showed")*.	**Yes, as to the instruments:** the pilot. K1's numbers in it are in D35-7.

No.	Change	Reason	Prompted by results?
D35-9	The two other tools of that reading are not run in this version. The Yahoo Streaming Benchmark times a stream processor's windows, not a message through a broker: it needs Apache Flink 1.11, a Kafka client of 2016 and Redis, and its latency is the end of a ten-second window less the last event in it. PerfKit Benchmarker's messaging benchmark times a cloud provider's messaging service *(Google Pub/Sub, Amazon SQS or Azure Service Bus)*, none of which this testbed has. Each comes in a version of its own, with its own pilot and predictions fixed from its source before any of its runs.	Neither runs on the servers this block has, and every design frozen before its own pilot has been changed by that pilot *(D33-8, D35-8)*.	**No.**
D35-10	The order. The first x86 pair's own list runs the block *(D19-2)*: the servers and the install record, then for each of rounds 1 to 4 each tool's T1, T2, T3 and T4 in turn and that round of K1, then K1's rounds 5 and 6. The list ends by copying the runs off the pair and the pair stopping itself.	As the block of 10 October ran.	**No.**

What changed since version 33, and why

The block of 10 October ran as version 33 froze it. Its T2 runs showed one clause of the judgement missing and one question asked only through predictions that could not answer it. This version supplies both, before any run is judged under them, and keeps every verdict already given.

No.	Change	Reason	Prompted by results?
D34-1	In T2's judgement *(D25-2)*, a count short of what the tool was asked to send rules out "times against one clock" as well as the behaviours that keep every value. A tool on one clock keeps every value: no value below zero can arise in it.	D26-2 held the count against the four behaviours alone. On 10 October pulsar-perf, round 1, kept 7 of the 3,000 latencies it was asked for with its consumer's clock moved back 5 ms. The count ruled out the behaviours that keep every value, a printed figure ruled out those that keep nothing, and the one hypothesis the count was never held against was left standing alone: "times against one clock", for a tool that had dropped 2,993 values.	**Yes:** the block of 10 October *(freeze 30)*. Frozen before any run is judged under it.

No.	Change	Reason	Prompted by results?
D34-2	T2 asks first whether the tool's two timestamps come from two clocks, as version 19 first asked it *(D23-1)*, and answers from the offset measured at the clock rather than from a prediction. **Two**, where the tool kept fewer than it was asked to send and fewer than its control kept, or where its average, median or minimum moved from the control by more than half the offset, either way. **One**, where every one of those figures it prints stayed within a quarter of the offset of the control, and its count, where it gives one, is whole. Otherwise **undecided**. Each offset run is answered; a round answers two where at least one of its two runs does and neither says one, one where both do, and is otherwise undecided.	The five hypotheses predict from our reference's trips. Where a tool's own trips differ from ours — a slow tail, batching, a clock in whole milliseconds — they can leave every hypothesis standing, or none, although the tool's figures moved by the whole offset. Whether they moved needs no prediction: on two clocks every difference moves by the offset, so a figure that keeps its value moves by all of it and a filter drops values; on one clock nothing moves at all. The bounds are halfway and a quarter of the way to the whole offset, so that what neither answer fits is answered by neither *(D15-3)*. The median is asked because T1's slope is read on it; a 99th percentile or a maximum is a few messages, and moves with them.	**Yes:** the same block, in which OMB's and pulsar-perf's figures moved in every round and the rule decided one of their sixteen offset runs, wrongly.
D34-3	Every T2 offset run of both tool blocks — the eleven of 23 to 25 September and the five of 9 and 10 October — is judged again under D34-1 and answered under D34-2, by `scripts/tools_block.py` from the readings, controls, reference trips, staircases, counts and measured shifts each run left. The verdict each run was given stays beside both. Where the paper says which tools time against one clock and which span two, it reads D34-2.	As version 21 judged every reading already taken again under its rule and reported the verdict under the old one beside it *(D25-2)*.	**Yes:** as D34-1 and D34-2.
D34-4	What stands, recorded and not corrected. The behaviours are still predicted from our reference's trips, not the tool's; their arithmetic is still a fine difference truncated toward zero *(D15-2)*, where three of the five subtract two stamps that are each already whole milliseconds; and each figure's allowance still comes from its own T1 scatter, which a few slow messages can widen to tens of milliseconds. A run these leave undecided is reported undecided.	Correcting the first would mean predicting from the tool's own trips, which none of these tools prints message by message. A correction that cannot be made is a limit, and is said to be one.	**Yes:** the same block.

What changed since version 32, and why

The plan listed a run for every tool that times a message, with a prediction for each. Version 15 ran eleven of them and left five unrun. This version runs the five, under the predictions and the rules the eleven were judged by, and records what version 15 did not.

No.	Change	Reason	Prompted by results?
D33-1	The five tools the plan listed for a run and version 15 left unrun are run: the OpenMessaging Benchmark *(T-P3)*, Apache Pulsar's performance client *(T-P4)*, emqtt-bench *(T-P2, beside Kafka's ProducerPerformance)*, and YCSB and nats bench, the two read-first tools version 15 did not name *(T-P7)*. Each runs T1 to T4 as the eleven did, under the same rules *(D15-1 to D15-5, D23-1, D25-1 to D25-5, D26-2, D26-4)*, four rounds each *(D4-2)*, on the first x86 pair *(D19-2)*, in folders of their own (`runs/azure/tools_v33`, then `_round2` to `_round4`). Each is pinned at the version the audit read *(T5, `data/tools_audit`)*: the OpenMessaging Benchmark at commit 5b1fa709, which has no releases; Pulsar at release 4.2.4, which the audit cross-checked against the master it read and whose consumer keeps a latency only when it is at or above zero, as master's does; emqtt-bench at its tag 0.6.3 *(cb86b188)*; YCSB at 66302f30; nats bench in the build of the NATS command-line tool that nats-latency runs *(cc0a8e32)*.	The plan registered a run for each, with a prediction. A prediction registered and never tested is a gap in the record. Three of the five take their two timestamps in two processes, the arrangement in which a value below zero can arise; of the eleven that ran, one does.	**No.** None of the five has run under this plan, and nothing it would report has been read: the pilot *(D33-8)* ran each once to check the instruments. Prompted by a reading, on 8 October 2026, of the paper's table of tools.
D33-2	Why version 15 left them out, recorded where it was not. Its D15-6 named the ten tools that would run and gave two reasons. The first, that the plan listed more tools than could be stood up at once, does not hold: each tool runs alone, one after another, and none needs another's server up beside it. The second, that a container puts a network hop inside the tenths of a millisecond being measured, is a reason to install servers natively, not to leave tools out: Pulsar and an MQTT broker install natively as RabbitMQ and NATS did. What decided it, as the script that set the servers up records, is that Kafka and Redis were already standing from the law campaigns and three servers were added; the tools whose broker was not yet on the machines were the ones left out. Nothing else in the record says why.	A plan's account of itself has to be true, including where an earlier version of it was not. The cut also shaped a result: of the four tools that take their two timestamps in two processes, the run kept one, so "ten of the eleven run time in one process" partly reflects which tools were run.	**No.** No result of these tools exists. The same reading.

No.	Change	Reason	Prompted by results?
D33-3	The two servers the five need beside those already on the broker. Mosquitto, Ubuntu's package, listening on every address and letting any client in, as RabbitMQ does here. Pulsar 4.2.4 as `pulsar standalone` *(broker, bookie and metadata store in one process)* on Java 17, with a gigabyte of heap, its web port moved to 8090 because nginx holds 8080 for the HTTP tools, and functions and stream storage off. Both are units that outlive the session that started them. The OpenMessaging Benchmark speaks to the Kafka, YCSB to the Redis and nats bench to the NATS already there. Before each stage the server is asked what the tool needs of it, from the driver: Pulsar to name its cluster, Mosquitto to let a client in.	D15-6's rule for servers stands: nothing between a tool and what it times that the tool would not meet in use.	**No.**
D33-4	How each is run, and which of its figures is read: the addendum at the end *("The five")*. Where a tool is made of two or three processes, the process that subtracts is the one T3's go-first and T2's moved clock reach, as for rdkafka_performance. The OpenMessaging Benchmark runs in its distributed mode, the mode its end-to-end latency is designed for.	Each is run as its own documentation runs it and read where it prints a latency, as the eleven were. Where a tool reports one quantity in two places, which one is read is fixed here, before any run.	**No.**
D33-5	How each keeps time, read from its invocation and its source before any of its runs *(the addendum's table)*, and so what T2 is expected to find in it. Each crosses the delayed direction once in T1.	As version 19 did for the eleven *(D23-1)*.	**No.**
D33-6	A reference of our own for each *(T2 takes its offsets and its predictions from one)*: the study's own program for the two that speak Kafka and Redis; the NATS reference for nats bench; and two written for this version, before any run of their tools — an MQTT client written by hand, as the NATS one was, and a Pulsar client through Pulsar's own library, pinned *(pulsar-client 3.13.0)*, because its protocol is too much to write by hand. Each times its own messages on the monotonic clock, one in flight at a time.	D25 gave every tool a reference; a tool without one cannot be asked T2's question.	**No.**

No.	Change	Reason	Prompted by results?
D33-7	Which clauses of the five's predictions these runs can reach, fixed before any run. **T-P2** for emqtt-bench: its average is judged from its histogram; its median cannot be, because it prints none and its histogram's finest bucket holds everything up to a millisecond. Its publisher paces in whole milliseconds on its own timer, in step with its clock's tick, so under D15-4 its T2 runs are T-P2's evidence and not an identification. **T-P3** for the OpenMessaging Benchmark: both clauses are about trips under a millisecond, and a clause about trips shorter than the tool's own zero-delay trip on this pair is reported out of reach, as D30-3 does for the law's; its retention cannot be read at all, because the tool counts nothing it drops. **T-P4** for Pulsar: the median clause is about trips under half a millisecond, under the same rule; the average clause is T2's question and is judged by T2's rule *(D25-2)*. **T-P7** for YCSB and nats bench: each behaves as its class predicts, in T1 and T2. YCSB's class was fixed in D12-1, medium *(microseconds)*. nats bench's is fixed here, from the audit's reading of its source: high *(Go's monotonic clock, printed in hundredths of a microsecond)*, timing a request to its reply in one process, so that no value below zero can arise in it.	A clause a run cannot reach is reported as that, not as failed and not as held.	**No.**

No.	Change	Reason	Prompted by results?
D33-8	The pilot of 9 October 2026. On the first x86 pair each of the five was built, its server put up, and the tool run once at no added delay, in the receiver's namespace and on the machine itself, to check that it runs, that its server answers it and that its output can be read; and a five-second offset was shown to reach the clocks of the JVM and of the Erlang VM through libfaketime. Its runs test no prediction and none is analysed; they are kept, with the shakedown of the scripts that run the block, in `runs/azure/collected/matched/`, and the readers' tests are cut from them. Two things it found are in this version: the benchmark's distributed mode, which never completed a run on the July testbed, completed here; and a process in the receiver's namespace cannot reach the driver's own address *("no route to host")*, so the benchmark's coordinator and both its workers share whichever network the stage gives the tool. A shakedown of the scripts that run the block then ran each tool through them the same way, and each reference client once.	A pilot tests instruments, never a prediction *(Section "What the pilot showed")*.	**Yes, as to the instruments:** the pilot. No prediction.
D33-9	The order. The first x86 pair's own list runs the block *(D19-2)*: for each round, each tool's T1, T2, T3 and T4 in turn. The list ends by copying the runs off the pair and the pair stopping itself.	As the eleven's rounds 2 to 4 ran.	**No.**

What changed since version 31, and why

Nothing about what is measured or how it is judged; everything about finishing it. On 25 and 26 September the got-it brake stopped five campaigns part-way, and two faults of the kit cut or held two more.

No.	Change	Reason	Prompted by results?
D32-1	Every campaign from here on runs with the got-it brake recording rather than stopping, as A5's last thirteen runs did (D26-1): what the brake would have said travels with each run, the campaign goes on, and every prediction a recorded run bears on is judged with and without the runs the brake would have stopped. A pair is told by one word on its disk, so that a campaign its queue starts after a reboot records too. Every other stop rule stops as before.	Between 25 and 26 September the brake stopped five campaigns part-way — A1 on Kafka at 52 of 56 runs, the Redis bridge at 58 of 60 and again at 38 of 60, A9 on the Arm pair at 64 of 128 and on the first x86 pair at 60 of 120 — each on a move of the "got it" median against its own setup's earlier runs, not against anything the added delay does: 0.380, 0.508, 0.381, 0.309 and 0.299 ms. A design sized for full rounds and cut short answers with less power than it was given, and every run the brake stops is still recorded, so reading with and without them loses nothing a stop would keep.	**Yes:** the stops, not what the runs measured.
D32-2	A campaign that stopped before its runs were all done is finished from its own queue, under the calibration its first runs had and on the kernel they ran on, with the brake recording (`cloud/azure/resume.sh`); a run interrupted is recorded as failed and run again, and the resumed runs are that campaign's runs, marked where they begin in its log. This covers every campaign an audit of the pairs' disks finds with runs left: on the first x86 pair A3 on Kafka (19 September, 2 left), A1 on Kafka at 3 and 3.75 ms (4), A9 (60) and A2 on the HZ=1000 build (23 September, 19); on the second x86 pair the Redis bridge twice (2 and 22), A3 (1) and A2 on the HZ=1000 build twice (12 and 28), on HZ=100 (29) and on HZ=250 (25); on the Arm pair A9 (64) and whatever its own audit finds. It does not cover two false starts: an A3 with no run done, replaced within the half hour by a complete campaign of the same design, and an A2 with two, stopped by hand and followed the same afternoon by a complete session on its kernel. Finishing them would be running whole campaigns again. A campaign whose kernel cannot be booted is reported unfinished, never finished on another kernel.	Each stopped campaign's remaining runs are a gap in a design whose power was simulated for full rounds. The whole-campaign retries already on the lists run as well.	**Yes:** the stops.

No.	Change	Reason	Prompted by results?
D32-3	Two corrections of the kit that decide no verdict. The receiver's capture in M0's recorded runs is stopped by its own command line: stopped through the `sudo` that started it, it was never stopped at all, because `sudo` does not relay a signal sent from its own process group, and M0's first recorded run waited six hours; from 19:59 UTC on 26 September a watcher on the pair stopped each capture once its run had written its results, and the run that waited is complete and counts. And the queue waits on a job still at work after twelve hours rather than going on beside it: the 16-round A3 with its calibration ran past twelve hours, the queue started A9 while its last runs were going, cut it at 191 of 192, and the A9 stopped within a minute.	Faults of the kit, found when they cut runs.	No.
D32-4	Every run is registered with its machine and settings — the pair, the machine size and CPU model, the kernel, the tick and the CPUs online, the slice, the backend, the load, the delay set and the delay the broker held, the block, setup and round, its verdict and what the brake said, its times and the commit it ran — and every fault met by the campaigns is recorded with what it cost and what was done.	What ran is otherwise recoverable only from the run folders themselves.	No.

What changed since version 30, and why

How M0 is read, fixed before any of its runs is read (D30-7). The explanations and their tests are those of version 3; what is fixed here is the arithmetic each test was left to.

No.	Change	Reason	Prompted by results?
D31-1	M0 is read by `m0_read.py`, with `pcap_read.py` reading the captures. At each backend, and for Redis at each acknowledgement batch, the departure is $D = (b - 1) \times 8$ ms, b the slope of the recorded runs' median trip on the delay each run's broker held, with an interval from resampling whole rounds; the unrecorded runs' departure is reported beside it, with the recording's own effect at each setup and the 0.02 ms rule. An explanation accounts for at least half of D where its own contribution, in ms at 8 ms, has D's sign and at least half its size. M-H1's arrival shift is the broker's held delay plus the change in the wire's own transit, taken from the two captures as the request's way up plus the reply's way down, in which the two machines' clock offset cancels. M-H3's contribution is the change, from 0 to 8 ms, in the receiving thread's median wait for a CPU after each wake from sleep.	Version 3 fixed each test's threshold and left its arithmetic to the reading. The sign matters: in the pilot Redis's trip grew more than one-for-one and Kafka's less (D6-7), and an explanation that pushes the other way accounts for neither. The departure is read on the runs the explanations are read from.	No. Fixed before any M0 run.
D31-2	M-H2's replay is a simulation of each recorded run's messages through the receive loop's own rules: for Redis, a read waiting at the broker is answered with what the broker holds when the first message comes, up to the read's count, each message stamped one acknowledgement round trip after the one before where each is acknowledged, and the next read issued after the last; for Kafka, one fetch waits at the broker, is answered at the first message, and the next goes when the answer lands. It is fed with the run's messages, the delay its broker held, its wire transit, and the loop's own costs measured on the part's 0 ms runs; never with the run's per-message outcomes. Where those runs never carried two messages in one reply, the cost of the second is unknown and M-H2 is not tested there.	Replayed against the returns each run logged, a replay reproduces any slope by construction and tests nothing.	No.
D31-3	A request is held for an acknowledgement where it leaves the receiver within 50 μs of an incoming acknowledgement of data the receiver had outstanding for over 100 μs, after the program issued it; M-H4's contribution is the change in the total hold per measured message. It is not tested for Kafka.	Both clients set `TCP_NODELAY` (redis-py 7.4.1 and kafka-python 2.3.2, as installed on the pairs), so Nagle's algorithm, the usual way a request waits for an acknowledgement, cannot hold one; only the send window can, and the test is run as written. The Kafka library issues its fetches out of the program's sight, so no fetch can be shown to have waited.	No. Found reading the clients before any M0 run.

What changed since version 29, and why

The corrections version 27 recorded (D27-5), made in the programs; the reading of A9, fixed where this plan left it to the reading; and the date of M0's. No prediction and no run changes.

No.	Change	Reason	Prompted by results?
D30-1	P3a and P3b compare the lowest and the highest load whose plateau is at least 2%, read once from the campaign's own runs; a load under the bar is reported out of reach with its plateau, and a campaign with fewer than two loads over it did not test either prediction (D14-2, D14-3).	The program compared the lowest and the highest load the campaign ran, whatever their plateau, so every A3 verdict compared 50% load, whose plateau is 0.12–0.16% and which fixes no cliff position, with 88%. It is the one correction that moves a number already reported.	**Yes:** the check of 25 September, D27-5(a).
D30-2	P7 holds where the read-back slice matches at every core count and the halfway point is inside $[s, s+h]$ at every core count whose slice the runs reach; a core count out of reach is reported, as a slice is (D8-1).	The program also held P7 to the slope P9 asks across three slices, and to P9's two slices at least, neither of which P7 asks.	**Yes:** D27-5(b).
D30-3	A campaign that reaches fewer slices than P1 or P9 needs, fewer than two loads over P3a's bar, or no core count in reach is reported out of reach rather than not confirmed, and the command answers 3 for it. A slice in reach whose curve does not cross halfway is a miss: read through the anchor, it had dropped out of the count the rule is held to.	A campaign that could not test a prediction did not find against it, and the two read alike. Which slices are a test is read from the design's points, never from whether a curve crossed.	**Yes:** D27-5(c); the dropped slice was found in making it.
D30-4	A block of several campaigns is resampled campaign by campaign, each drawing its own rounds, so each keeps its number of rounds and every slice it carries is in every draw. Over one campaign the draws are the ones they were.	Drawn by round number, the same round of every campaign came out as one unit: four units where each campaign has four, and a draw that repeated a round repeated it in every campaign at once.	**Yes:** D27-5(d).
D30-5	P6 is judged on A6's recordings by `helper_waits.py p6`, frozen here: a run's rate is predicted, with nothing fitted, as the share of its recorded waits longer than $T - g$, its median trip less its median "got it" delay. A setup's ratio is the mean of its recorded runs' predictions over the mean of their measured rates, each campaign's setup its own; a setup whose recorded runs measured no negative reading has no ratio and is reported. Judged on each pair and backend, and held only where every one holds.	P6 had no program, so the recordings in half of A1's and A3's runs were never read against it. The share is E-A9's prediction, and A6's histogram, which has no clock and no thread, supports no other; $T - g$ is the margin A9-2 states for a reading to turn negative. The exploratory reading D28-1 records used the trip T itself, and put 7 of 18, 5 of 18 and 2 of 20 setups in the band; no setup has been read at $T - g$ before this version is public.	**Yes:** D27-5(e).

No.	Change	Reason	Prompted by results?
D30-6	How A9 is read where version 28 left it open (`helper_waits.py a9`): a message's stamper is the named thread on a CPU at the instant of its stamp; a run is read only where that identifies the stamper of at least 90% of its acknowledgements; a negative reading not shown to be waiting counts against A9-1; where several threads stamp, A9-2 weighs each thread's waits by its share of the stamps. Beside A9-2, and judged by P6's rule in the same way, A9-2b: a run's rate predicted as the share of its acknowledgements whose own wait for a CPU outlasted $T - g$. A9-2 is judged as version 28 froze it.	Version 28 wrote of one stamping thread, and the Redis producer stamps from sixteen send workers. A9-2's sum counts the time during which a message arriving at a random moment would find the thread already stalled, the right count where stalls come independently of the messages; both producers stamp from a thread that sleeps until an acknowledgement's reply wakes it, so the stall that decides a reading is the one its own acknowledgement began.	No. Found in building the reading, before any A9 run.
D30-7	M0's reading — the tests of M-H1 to M-H4 — comes in version 31, frozen before M0's runs are read.	The replay M-H2 asks for and the held requests M-H4 asks for are programs of their own, and M0 runs fourth on the first x86 pair's list, about twelve hours after that pair starts again.	No.

What changed since version 28, and why

The second study of the mechanism version 27 announced (D27-6): M0, stage 0's own study of the instrument, which was never run because its recording was never built. No prediction and no test of M-H1 to M-H4 changes.

No.	Change	Reason	Prompted by results?
D29-1	M0 is built and run as S0-2 set it out: on the first x86 pair, at 10 rounds, both backends at 0, 2 and 8 ms added and 75% load, Redis both acknowledging each message and in batches of 200, the football match replayed at the pilot's speed *(the speed-up that gives ten messages a second on average)*. In a random half of its runs: a capture of the packets between the receiver and the broker, in the receiver's namespace and on the broker; the receive loop's cycle, each read's start, length and messages *(the Redis consumer has always written it, the Kafka consumer now does when asked)*; and every `python3` thread's scheduling events, with the clients naming their threads, as in A9, from which the receiving thread's wake-up delays are read.	Stage 0 ran S0-1, S0-3 and S0-4; S0-2 was to follow once M0's recording was tested, and the recording was never built, so M0 never ran. Without it the trip's departure from one-for-one under bursts stays an open property of the clients; the law's tests do not need M0, because each run is judged at the trip it had. The pieces A9 needed — the event recording and the clients' record of their threads — are most of what M0's third recording needs.	**Yes:** the check of 25 September.

No.	Change	Reason	Prompted by results?
D29-2	A run that replays the football match is held to that plan's own count and rate: the events whose send time, at the run's speed-up, falls after the warm-up and within the run, and the rate among them (`replay_window.py`, and `run_integrity.py -planned`). A Redis setup that names its acknowledgement batch is held to that batch. Every other rule of the integrity check holds unchanged.	The integrity check held every run to a rate times a time and every Redis run to a batch of 200, which is what every steady campaign does; M0 sends in bursts, nearly half its gaps under a millisecond and some seconds long, and half its Redis runs acknowledge one message at a time on purpose. Held to the steady rules, every one of those runs would be repeated for doing what it was asked.	No. Found in building M0.
D29-3	The corrections D27-5 records come in version 30, with the program that reads A9 and the program that judges P6, frozen before either is read.	Neither decides a run.	No.

What changed since version 27, and why

The first of the two studies of the mechanism version 27 announced (D27-6). No prediction changes.

No.	Change	Reason	Prompted by results?
D28-1	A9, one campaign on each of the three pairs at 4 rounds: the 3 ms slice, the eight trips, 75 and 88% load, both backends. In a random half of its runs every `python3` thread's scheduling events are recorded one by one, and the clients name the threads that stamp their sends, acknowledgements and arrivals and read both clocks (`-thread-record`). How it is read is fixed below, before its runs. The Kafka consumer can also log each of its reads (`-read-trace`), in the file and columns the Redis consumer has always written, for M0.	The recording A6 rides on counts every `python3` thread's waits in one histogram, and an exploratory reading of three campaigns' recordings put 7 of 18, 5 of 18 and 2 of 20 setups inside P6's band (D27-6). Wherever the free reading and the fitted shape disagree the fitted shape fails, and on Redis the rate already falls well below the slice. The law is about the waits of the one thread that stamps the acknowledgement, and those can be read message by message.	**Yes:** the check of 25 September.
D28-2	M0, with the recording it never had, and the corrections D27-5 records come in version 29, frozen before M0 runs.	M0 replays the football match in bursts, and the per-run checks were written for steady traffic of 5,000 messages; its recording — packet captures at both ends, the receive loop's cycle, the receiving thread's wake-ups — needs more than A9's. The corrections decide no run, so they need not hold A9 back.	No.

What changed since version 26, and why

The check of all the data made on 25 September, before the testbed is taken down: what each block set out to see, what its runs can show, and which runs could still improve it. No prediction changes. Nothing measured is set aside; runs that are not used are kept beside a note saying why.

No.	Change	Reason	Prompted by results?
D27-1	A2's Redis session on our HZ=1000 build, the second of its two days, is run again whole, under version 26, from 25 September. Its attempt of 22 September failed its calibration's gate, whose fault was then fixed; its attempt of 24 September stopped on the got-it brake at its 32nd counted run. That campaign is kept whole, with its cause beside it, and none of its runs is used.	A2 gives each kernel two days per backend (D4-7), and Redis's HZ=1000 had one: found by counting the sessions against the design. The stop was a got-it move of −0.394 ms at 0.864 ms added, where the delay sits on a path the got-it note does not travel, and every other check of the run passed: the kind of stop D26-1 recorded for A5. The pair's next two Redis sessions ran without a stop.	No. Found by counting the sessions against the design.
D27-2	A second bridge session on the stock kernel for each backend: Kafka on the first x86 pair, Redis on the second, at A2's setups and 4 rounds.	The bridge asks our HZ=1000 build and the stock kernel to agree within 0.1 ms at the cliff's halfway point. Kafka at the 3 ms slice gives 3.61 and 3.58 ms on our build's two days and 4.14 on the stock kernel; Redis at 1.5 ms gives 2.30 on our build's one day and 1.87 on the stock kernel. Our build's two days agree within 0.07 ms, so a second stock day says whether the stock kernel's own day-to-day spread explains the difference or the kernels differ.	**Yes:** the bridge's reading of 25 September.
D27-3	A1 gains a slice of 3.75 ms, and one campaign: Kafka at 3 and 3.75 ms, on the first x86 pair, at 4 rounds, through the 3 ms anchor.	P1 asks for at least four slices a pair can reach. The A1 sessions' own Kafka zero-delay trip was 2.21 ms, above the 2.25 ms slice's plateau trip of 2.03 ms, so Kafka reached 3, 4.5 and 6 ms: three. This plan said Kafka reaches 2.25, from the calibration of 18 September at 1.964 ms, and no campaign reported the slice as out of reach with the trip that excludes it. 3.75 ms is the step of the ladder's 0.75 ms spacing between the two smallest slices Kafka reaches.	**Yes:** counting the slices behind P1 on 25 September.
D27-4	A third x86 A3 Kafka campaign, at 75 and 88% load only, on the second x86 pair, calibrated at both loads, at 16 rounds.	P3a and P3b are judged between the lowest and the highest load whose ordinary plateau is at least 2% (D14-2); at 50% the plateau is 0.12–0.16% on every A3 campaign, so the loads judged are 75 and 88, and a run at 50% decides nothing. Judged that way, the first pair's two campaigns put P3a's move at −0.01 and −0.19 ms by the free reading, the first with its whole interval inside the band and its fitted reading 0.003 ms outside it. The second x86 pair has not run A3: this is its answer, beside the first pair's, and nothing is pooled across them. 16 rounds is D9-3's count, at which D14-2's simulation confirmed P3a in 98% of campaigns under the law with the guard.	**Yes:** P3a read as frozen on 25 September.

No.	Change	Reason	Prompted by results?
D27-5	The programs that judge depart from this plan's text in five places, recorded here and corrected in version 28, where every verdict they touch is judged again with the reading as it was beside it. (a) P3a and P3b compare the lowest and the highest load a campaign ran, not the lowest and highest whose plateau is at least 2% (D14-2, D14-3). (b) P7 is held to P9's slope as well as to P1's band, where P7 asks only the band. (c) P1 is reported not confirmed, rather than out of reach, where fewer than four slices can be reached. (d) A block of several campaigns is resampled by round number, which draws the same round of every campaign together: four units, where each campaign has four. (e) P6 has no program, so the waits recorded in half of A1's and A3's runs have never been read against it.	Found by the check of all the data on 25 September. Only (a) moves a number already reported: every A3 verdict compared 50 with 88%.	**Yes:** the check of 25 September. No verdict changes in this version.
D27-6	Two studies of the mechanism are frozen in version 28, before they run: a trace of the got-it helper thread's own waits, message by message, on all three pairs; and M0, whose recording was never built.	Wherever the free reading and the fitted shape disagree — P1, P2d, P7 and P9 — the free reading follows the law and the fitted shape does not, and on Redis the rate already falls well below the slice. The recording of waits counts every Python thread's, and an exploratory reading of three campaigns' recordings put 7 of 18, 5 of 18 and 2 of 20 setups inside P6's band: the helper's own waits are what the curve's shape is made of. M0 was never run.	**Yes:** the check of 25 September.

What changed since version 25, and why

The corrections found on 25 September, while the tools block's first round was written up and the pairs' remaining work was counted against the campaign table. No prediction changes and no campaign is re-designed. Runs set aside are kept beside a note saying why; none is deleted.

No.	Change	Reason	Prompted by results?
D26-1	A5's session at 2 CPUs runs its last thirteen runs with the got-it brake recording rather than stopping: what the brake would have said is written beside each run, and nothing it says stops the campaign. It is set for that campaign alone, by name (`GOTIT_BRAKE=record`); every other campaign keeps the rule. P7 at 2 CPUs is judged twice, on the 27 runs taken under the rule and on all forty, and both answers are reported.	The session stopped twice on its got-it: a move of 0.505 ms with 0.98 ms added, then 0.660 ms with 0.046 ms added, where the delay cannot be what moved it. Every other check on both runs passed, and the same pair's sessions at 4 and 8 CPUs ran 56 and 60 runs without a stop. At two CPUs under 75% load the producer's acknowledgement time makes excursions the brake reads as a broken instrument. The brake asks whether the instrument broke mid-campaign (D20-2); P5(c) is judged where it always has been, in the session's own calibration.	**Yes:** both stops of this session. Decided after them, and frozen before the thirteen runs start.
D26-2	T2 holds a tool's count against what the tool was asked to send, which `tools_run.sh` writes beside every run, and a count short of that rules out only the behaviours that keep every value at the offset. Every T2 reading is judged again.	The judge held the count against the number of trips in our reference, another run of another client, and compared a dropping behaviour's count of our trips with the tool's count of its own. wrk2 counted all 3,001 it sent; held against our 5,000 it read as short, and its run at 1.83 ms was decided on "3001 where this would count 50". Rule 5 of the addendum names what was sent, and a reference that is not a record of the tool's own messages cannot predict its count as a number.	**Yes:** that run's verdict, which becomes undecided. No tool's answer changes.
D26-3	The Go tools' T2 moves the driver's own clock, with nothing else running on it, instead of a second machine: stepped, measured against the hypervisor's PTP clock, which stepping does not move, put back after each run and measured again, and T2 does not go on from a clock more than 0.25 ms from it.	A second machine keeps other work off a stepped clock; refusing to step it under a campaign does the same on the machine the tools' T1 ran on. The first run of the four Go tools never put the clock back — the step was recorded in a subshell — so each second offset landed on its first, 5.3 to 5.7 ms from the PTP clock where 3.8 was planned. Those twelve runs are set aside and were run again.	**Yes:** found from the job's own last line. The verdicts, one clock for all four, are the same either way.
D26-4	The tools block runs the four rounds D4-2 has always asked for. Round 1 stays where it ran; rounds 2 to 4 run in folders of their own, and no stage writes over runs already there.	The block ran once. Its stages wrote into folders named only for the tool and the step, so a second round would have overwritten the first, and nothing said the block was a quarter done.	No. Found by counting each pair's remaining work against the campaign table.

No.	Change	Reason	Prompted by results?
D26-5	Corrections that decide no verdict, recorded as D25-8 asks. hey's reader takes "50%% in", which is what hey prints, as well as the "50% in" its documentation shows, and hey's runs are read again from their raw output. T1's step finder still reads only the average; for the two tools that print none, T2 uses their printing step. The tools block's server setup is done on the broker with the broker's own address passed to it. The version history on the plan's first pages is one box per version: it was one box, which cannot break across a page, and in version 25's PDF everything below the foot of page 3 was cut.	Each was found writing up round 1. None changes a verdict: T2 judges the average and the minimum, and hey's are read as before; judged with a 0.1 ms step, the two tools' four runs keep their verdicts, and changing the step finder after those verdicts were read would change T2's inputs for the other nine.	No.

What changed since version 24, and why

Every correction found on 24 September, when the running campaigns were checked and the check was then turned on the instrument itself. No prediction changes and no campaign is re-designed. Where a correction sets runs aside, they are kept, with a note saying why, and run again; no run is deleted.

No.	Change	Reason	Prompted by results?
D25-1	T2's four behaviours are held against the control run taken in the T2 session, as D23-1 says. The program held them against T1's zero-delay step, taken hours earlier under a different arrangement of the delay, and used the control only for the one-clock question.	D23-1 took the control precisely so that nothing would be compared across sessions. For a tool whose figures wander between sittings the difference is not small: rdkafka_performance's average moved 3.450 ms under a shift of 3.434 ms, which is the signature of a subtraction across two clocks, and every behaviour was still ruled out against a reading from hours before.	**Yes:** rdkafka_performance's T2, 24 September. A program that did not do what its freeze says.

No.	Change	Reason	Prompted by results?
D25-2	T2 allows for each tool's own run-to-run noise, measured from the ten runs of its T1 staircase, and judges the one-clock hypothesis as a fifth beside the four behaviours, by the same allowance and on the same figures — the average and the minimum. The predictions are made at the offset measured at the clock. The whole rule is set out in Section 17.	Every tolerance the judge used was a tool's printing step: 0.001 ms for valkey-benchmark, 0.00001 ms for memtier_benchmark. Two runs of the same tool minutes apart differ by 0.07 to 0.16 ms. With no allowance for noise at all, a tool whose figures did not move was ruled out of "did not move" by its own scatter: valkey-benchmark's average rose 0.075 ms under a clock moved back 1.78 ms, which is one clock by any reading, and the judge could not say so. A second control would not supply the noise: an offset run falls outside the range of k equally noisy controls with probability $2/(k+1)$, which at two controls is a coin. T1's staircase already holds ten runs of each tool on the same pair.	**Yes:** the first T2 runs under D23-1, 24 September. Fixed here before any reading is judged under it, and the verdicts as D23-1 was implemented are reported beside.
D25-3	A tool's reading is taken only from a run that did the work. k6 takes its settings as flags, which `sudo` keeps, where it took them from the environment, which `sudo` clears. The reader takes k6's request round trip (`http_req_duration`) and never its iteration time, and a run that sent nothing reported nothing. k6's T1, T3 and T4 are run again.	Every stage runs k6 inside the receiver's namespace through `sudo`, and k6 was the one tool configured through the environment. It got no URL and no rate: 4,895,826 iterations in 60 s, each failing at once on "invalid URL", not one byte sent or received. The only figure it printed was the time its iterations took — 9 μs — and the reader took that for a latency, so its fifteen runs were counted as fifteen readings. This paper's artifact, in our own harness.	**Yes:** found by the slope check of D25-5, 24 September.
D25-4	The servers the tools talk to are set up on the broker and checked from the driver, the way the tools will be answered: the fixed page and its size, a Redis PONG, a NATS greeting, a RabbitMQ that lets the tools' user in from another machine. Every stage checks its own tool's server before it runs. NATS runs as a service. Whether a jar can be started is asked of the jar, not of the install record. The runs that met no server are run again.	`tools.sh brokers` is written to run on the broker and says so. It was run on the driver, set the driver's servers up, asked 127.0.0.1 whether they answered, and certified a machine the tools never talk to. The broker kept RabbitMQ's default, which admits its user from the broker only, and its NATS server, started with nohup over a session, had not outlived it: rabbitmq-perftest and nats-latency measured nothing for fifteen runs each. PerfTest's jar also had no Main-Class; the install record predated the question and said it was present.	**Yes:** the check of 24 September.

No.	Change	Reason	Prompted by results?
D25-5	T1's slope — how far a tool's reading moves per millisecond added — is recorded for every tool and held to the number of times the tool's measured interval crosses the delayed direction: one for a round trip, two for a send, an acknowledgement and a fetch. A slope less than half that count or more than one and a half times it is a staircase that did not measure the path, and it is reported as that.	The staircase has always assumed the delay reaches the tool; nothing checked that it did. Held to it: vegeta 1.003, hey 1.054, wrk2 1.008, valkey-benchmark 0.979, memtier_benchmark 1.002, rabbitmq-perftest 0.995 and kafka-producer-perf 1.141 against one; kafka-end-to-end 2.028 and rdkafka_performance 1.958 against two — and k6 0.000, which is how D25-3 was found.	**Yes:** the check of 24 September.
D25-6	The Arm pair's A3 is read at the loads it ran at: 52.7% for the level designed as 50, 74.9 for 75 and 88.4 for 88, measured in the trial, flat from round to round. The judges group runs by designed level, as registered, and the measured levels are printed beside every A3 verdict on that pair. The load correction is not changed for any run of this plan.	The correction samples the machine before the trial starts, so it sees stress-ng alone — 48.7% at a duty of 50 — raises the duty to correct it, and cannot see the client's own load, which then lands on top. It serves 75 and 88 well and the 50% level badly. Changing it now would split a running campaign into two instruments, which is worse than a known offset carried beside every run.	**Yes:** the check of 24 September.
D25-7	Green tags mark only the changes of this version, and every earlier one is grey.	The legend has said since version 8 that green marks what changed since the version before. Each version added its own green tags and left the earlier ones green, so by version 24 fifty-four tags from fifteen versions claimed to be new.	**No.** Read in the document.
D25-8	Corrections to programs that decide no verdict are recorded here, because no freeze has recorded them; Section 17 lists them.	A freeze is where a reader looks for what changed in the instrument, and a correction left out of it is one a reader cannot see.	**Yes:** each is listed with what found it.

What changed since version 23, and why

One campaign is added on the Arm pair. No prediction changes, no campaign is re-designed, and nothing already measured is re-judged. Every prediction is still decided within one pair and one backend, so the Arm answer to P3a will stand beside the x86 answer rather than be mixed with it.

No.	Change	Reason	Prompted by results?
D24-1	A3's Kafka campaign runs on the Arm pair as well as the first x86 pair, at **24 rounds**: 50%, 75% and 88% load, the 3 ms slice, six trips, 18 setups, 432 runs. The schedule under "Which pair runs what" gains it. The rounds are P3a's own simulation at that pair's spread *(0.256)* and the levels its spread pilot measured *(plateau 0.0568, floor 0.0123)*: confirmed in 88% of campaigns under the law and in 0% of campaigns where it is false, from 1,000 simulations of 200 resamplings.	P3a and P3b have been judged on one processor family. A4 asks A1's question — does the cliff follow the slice — of the Arm pair already, and nothing asks this one there. The second of this paper's three parts is whether the effect appears on machines and processors it was never seen on, and load is the condition a reader is most likely to say our machines were merely quiet under.	**No.** The shape of what has been answered, not anything a campaign found.
D24-2	A3's Redis campaign is not run on the Arm pair. The block there is its Kafka half alone.	P3a and P4 cannot be tested on Redis: its plateau is 0.55–1.15%, too shallow for the cliff to be located within it, which is why A3's Redis half was not run on the x86 pair either. The Arm pair's own spread pilot reads the same shape without being told — a plateau of 0.0115 and a floor of 0.0001 that `law_design.py` marks `below_resolution` — so the reachability is read from that pair's own instrument rather than carried over.	**Yes:** the Arm pair's spread pilot of 18 September, read again on 23 September.
D24-3	P3b is judged on the same campaign and is recorded, before any of its runs exist, as underpowered. Simulated at the Arm pair's own spread and levels it confirms in 26, 27, 28, 36 and 43% of campaigns at 4, 6, 8, 12 and 16 rounds, and in **76%** at the ceiling of 40, where the rule asks for 80%. Power climbs with rounds, so the 24 this campaign runs lies between the 43 and the 76 and under the bar either way, and no number of rounds this plan would spend clears it. A non-confirmation of P3b on the Arm pair is therefore reported as this instrument failing to reach the prediction, not as evidence against it.	The alternative is to spend a campaign we cannot afford chasing a bar the rule cannot clear here, or to report a non-confirmation that reads like a result. A3's design has always taken its rounds from P3a *(D9-1)*, and P3b rides on the same runs; what is new is only that its power is now written down in front of them. The plan has said the same of A2 since version 14, where the rounds rule cannot set a number at all *(D14-5)*.	**No.** P3b's own simulation at the Arm pair's measured spread and levels, before any run of the campaign exists.

What changed since version 22, and why

An addendum, and one run added to each T2 session. No prediction changes, no campaign is re-designed, and nothing already measured is re-judged — the T1, T3 and T4 runs already taken stand as they are, and the T2 runs already taken are read again under the new verdict from the figures they already carry.

No.	Change	Reason	Prompted by results?
D23-1	A T2 session runs the tool three times: once at no offset and once at each of the two offsets. The offset achieved at the clock is measured and recorded beside every run. Where the clock moved by what was asked and the tool's figures did not move from the no-offset run by more than the tool's own step, the verdict is that the tool times against one clock and no value below zero can arise in it. Where the figures moved by about the offset, the four behaviours are held against them as before. Where neither holds, it is undecided as before, with both numbers beside it.	T2's question has an answer only where the subtraction spans two clocks. Ten of the eleven tools write both timestamps in one process, so the moved clock moves both and the difference is untouched. Until now that came out as "no behaviour accounts for what it printed", which is the same sentence a tool doing something unforeseen would produce; the two are told apart by whether the tool's own figures moved.	**Yes:** valkey-benchmark's first complete T2, 22 September, which reported 0.4660 ms under a confirmed 1.806 ms shift of its clock.
D23-2	rdkafka_performance's two-clock arrangement is established by a shakedown before any of its runs are measured: a short producer and consumer, with the topic's partitions, the consumer's mode and its start offset recorded, repeated until the consumer reads what the producer sent. Where no invocation does, that is T4's answer for the tool — what it can report at all — with the invocations tried, and it is reported as this harness failing to drive the tool rather than as a limit of the tool.	Its consumer read none of its producer's messages, over a whole staircase of ten steps and again after the order of the two was reversed. A tool that measured nothing must not be reported as a tool that measured zero, and a staircase of ten such steps looks exactly like one.	**Yes:** its T1 and T4 on the first x86 pair, 22 September.

What changed since version 21, and why

One gate is read across the staircase rather than step by step. No prediction changes, no campaign is re-designed, and nothing already measured is re-judged: the sessions that passed this gate pass it under either reading, and the one it ended is re-run.

No.	Change	Reason	Prompted by results?
D22-1	A gross departure of the “got it” note is read across the whole staircase, as the slope of the median shift against the delay added, and fails the gate when that slope is further from zero than 0.25. A shift larger than the whole delay added is still gross at any one step on its own. Version 21 asked of each step that its shift be no more than a quarter of that step’s delay. The slope is written out beside the gate.	The test exists to catch the added delay reaching the note, and leakage of a fraction of the delay is that fraction at *every* step — it is a slope. Step by step the same quarter cannot tell leakage from drift: a note that wanders by a fixed amount d fails wherever d is more than a quarter of the delay, which is every step below $4d$ and no step above it. So the rule systematically ends sessions at their smallest step and is blind to the same wander at their largest.	**Yes:** the second x86 pair’s HZ=1000 Redis calibration, 22 September. Its slope was 0.997 with a 95% interval of 0.981 to 1.013 and it was known within 0.220 ms, and it was ended on a +0.342 ms shift at 1 ms added — while the Kafka half of the same sitting passed with −0.315 at 4 ms and −0.282 at 8, both outside their own 90% intervals. The two staircases’ slopes are −0.010 and −0.038.

What changed since version 20, and why

One campaign’s Kafka half moves to the other x86 pair, and one rule that was always meant is now one the programs keep. No prediction changes and no campaign is re-designed. A4’s P9 and A8’s P8 are restated from the same runs, read the way this plan already asked.

No.	Change	Reason	Prompted by results?
D21-1	A2's six Kafka sessions and its Kafka bridge run on the first x86 pair. The second x86 pair keeps A2's six Redis sessions, the Redis bridge and A5's two remaining sessions. The schedule under "Which pair runs what" gives all of A2 to the second pair. [D19-2]	Every prediction here is judged within one pair and one backend and nothing is pooled across either, so what has to stay together is a backend's three kernels, not the two backends; each keeps its counterbalancing of each kernel on two days. [D4-7] The load is what changed: the first pair finishes the ten tool campaigns today and the second is carrying about forty-five hours alone. Splitting by backend has the last measurement land about nineteen hours earlier, and the HZ=1000 Kafka session the second pair has already run becomes a replicate of that cell on a second machine rather than the only reading of it.	**No.** The shape of the work left.
D21-2	A prediction is judged on the runs the integrity rule passed, and the answer names every run it left out. Runs it stopped or marked for repeat stay on disk, stay reported and decide nothing.	The reader that feeds every judge kept anything carrying a trip and a negative rate, whatever the brake had said. A stopped run is one whose own instrument was in doubt and a repeated one was run again, so counting it counts a sitting twice; `run_integrity.py` has always refused to compare a run against neighbours that did not count, and the judges did not.	**Yes:** counting A8's x86 runs by design point gave thirteen where the design has twelve. Five runs of 3,179 are affected; A3's two answers reproduce to four decimals and A8's ratio moves from 3.12 to 3.25, not confirmed either way.

What changed since version 19, and why

Two brakes, wider, and one gate asked of the right thing. No prediction changes, no campaign is re-designed, and no campaign already run is re-judged: A8's x86 campaign keeps the 86 runs it had counted when it stopped and continues from the 87th under the wider floor.

No.	Change	Reason	Prompted by results?
D20-1	The calibration is required to be known at every step to within the larger of 0.30 ms and 0.15 of the machine's scheduling tick, where version 19 asked for a flat 0.30 ms. The bound each session used is written beside its gate, and the check's name no longer carries the old number.	0.30 ms was set on the stock kernel, whose tick is 1 ms; A2 boots two kernels where the tick is 4 and 10 ms. A calibration cannot be known to a fraction of the quantum its trips are quantised by. On the second x86 pair Kafka's halfwidth came in at 0.410 at HZ=250 and 0.387 at HZ=100, against 0.128 to 0.237 for Redis on the same boots; the two-stage calibration ran four rounds and did not move it, because this is not imprecision to be averaged away.	**Yes:** three sessions on the second x86 pair refused in a row, each after about two and a half hours of calibrating, and six of A2's twelve sessions blocked behind them.
D20-2	The floor under the got-it brake's allowance goes from 0.10 ms to 0.25 ms. Everything else about the brake is unchanged: the allowance is still the largest of a quarter of the added delay, three times the campaign's own pooled scatter, and this floor.	0.10 ms is P5(c)'s equivalence margin, which is the right size for the question P5(c) asks — whether the added delay moved the got-it — and the wrong size for this one, which is whether the instrument broke mid-campaign. A8's within-setup scatter is 0.012 to 0.036 ms, so three times the pooled figure is a very tight allowance and the floor is what stops it collapsing onto the noise. At 0.10 it sat below the drift a sound campaign shows: 0.124 ms on the Arm pair and 0.134 on the x86 pair, both over hours, both with nothing else wrong. Nothing is given up, because every run's got-it and its two distances are written beside it either way *(D17-3)*.	**Yes:** A8's x86 campaign stopped at 86 of 144 runs on a move of 0.134 ms where its own scatter allowed 0.132 — an overshoot of two microseconds, after four earlier stops.
D20-3	A campaign is placed only from its own backend's calibration, so it is only that backend's gate that has to have passed. A campaign that names no backend runs them all and still needs them all, as A5 does; a backend the calibration never fitted is refused rather than passed over.	The gate walked the whole calibration file and required every fit in it, whichever backend the campaign ran. A Redis campaign is placed from Redis's fit and from no other, and the state of Kafka's says nothing about whether it can be.	**Yes:** three sessions on the second x86 pair, each refused after about two and a half hours of calibrating, for the state of a fit they were not placed from.

What changed since version 18, and why

One program, corrected, and one pair's share of the tool campaigns. No prediction changes, no campaign is re-designed, and no campaign already run is re-judged against a different rule: A8's Arm campaign was judged again under the corrected program and returns the same verdict.

No.	Change	Reason	Prompted by results?
D19-1	The program that judges P8 tests all three of that prediction's clauses. It tested that Java's rate in the middle of the cliff lies between its plateau and its floor, and that Python's plateau is at most 1.5 times Java's. It now also computes how many times go-first cuts Java's plateau, at the trips both clients ran, and requires the five times this plan has asked for since version 8 *(D4-9)*. The cut is written beside the verdict, including where go-first reaches zero, which is reported as the cut it is.	A8 runs every setup ordinary and go-first, so the campaign already carried the clause and nothing further had to be measured; only the program was short. Worse than short: it printed a rule naming two clauses while its own documentation said it tested the priority effect, so a reader of either would have believed the third was tested. A judge that omits a clause its campaign passes returns exactly what a complete judge would, which is why this could not be seen in a verdict.	**Found while judging a result, and it does not move one.** A8's Arm campaign was judged under two clauses and again under three: not confirmed both times, on the ratio, 4.33 against the 1.5 allowed. The cut the corrected judge measures is 469.7, the figure computed by hand from the same runs.
D19-2	The ten tool campaigns run on the first x86 pair. The schedule under "Which pair runs what" gives five to each x86 pair.	What this plan requires is that each tool runs entirely on one pair *(Section "One pair per test")*, so that nothing in a tool's reading depends on which machine it ran on; that is untouched. The five-and-five split was how the load was balanced when it was written, and the load is no longer that shape: the second x86 pair carries A5's two remaining sessions and all fifteen A2 campaigns, about 46 hours, and the tools are about 47. Ten on the first pair and none on the second has both finishing within hours of each other, where five and five leaves the first idle for a day.	**No.** The shape of the work left, not anything a campaign found.

What changed since version 17, and why

One brake, wider, and nothing else. No prediction changes and no campaign already run is re-judged. A2's first session ran under version 17 and is reported under it: 28 runs counted, the 29th stopped it, and those 28 are kept.

No.	Change	Reason	Prompted by results?
D18-1	The scatter the brake allows is pooled over the campaign's own setups — every counted run's distance from its own setup's centre, taken together — instead of being estimated from the two or three earlier runs of the one setup being judged. What a run is held *against* does not change: it is still that setup's own earlier counted runs, so no drift from another sitting enters.	A standard deviation from two points has one degree of freedom and is not an estimate of spread; version 17 guarantees that the first braked run of every setup is judged against exactly that. Run-to-run scatter in the got-it belongs to the sitting rather than to one setup, and A2's first session shows ten estimates of it — 0.053 to 0.227 ms — scattering around a pooled 0.133 exactly as two or three points each would. The campaign was stopped by the smallest of the ten.	**Yes:** A2's first session on the second x86 pair, stopped at 28 of 40 runs by a run 0.275 ms from its setup's centre where three times the pooled scatter is 0.398.
D18-2	The brake waits until the pooled scatter has at least five degrees of freedom, counting one for each counted run beyond the first in every setup. Below that a run is recorded and not braked.	A pool of two or three runs is the same fault one level up. Five is where the estimate stops moving much with each new run, and a campaign of this plan's size reaches it within its first two rounds.	**Yes:** the same session, where the fault was a spread taken from too little.
D18-3	The pooled scatter, the degrees of freedom behind it, and the centre a run was held against are written beside that run, with the allowance they produced.	A brake that stops a campaign should be arguable afterwards from the campaign's own files. This one was: what it did was read off A2's runs within the hour, and the numbers in the row above are that reading. Recording them means the next reader does not have to reconstruct it.	No.

What changed since version 16, and why

One brake again, and nothing else. No prediction changes, no campaign is re-designed, and no campaign already run is re-judged: results are reported against the freeze in force when they ran, and A3's two Kafka campaigns, A8's Arm campaign and A2's first session all ran under earlier versions and are reported under them.

No.	Change	Reason	Prompted by results?
D17-1	The brake on a run's "got it" median compares it against that campaign's own earlier counted runs of the same setup — the same client, the same place for the note, the same slice and the same added delay — and not against another sitting's calibration. Until a setup has two earlier counted runs, the run's got-it is recorded and not braked.	A campaign runs some hours after the calibration it is placed from, and its got-it drifts in that time by an amount that has nothing to do with the delay. The brake charged that drift against an allowance built from the delay. Measured on two campaigns: A3's Kafka campaign on the first pair sat 0.022 ms from its calibration over 216 judged comparisons; A8's on the Arm pair sat 0.124 ms over 14. Their noise floors were 0.170 and 0.199 ms, so the floors are not what separates them — the drift is, six-fold. Within one campaign that drift is common to the runs being compared and cancels.	**Yes:** A8 on the Arm pair, which stopped itself at 28 of 144 runs.
D17-2	That comparison's allowance keeps the shape it has had since version 8: the largest of a quarter of the run's own added delay, three times the scatter of the runs it is held against, and 0.10 ms. What changes is only what it is measured against.	The quarter-share is this plan's pre-registered margin for P5(c) and is not being widened. The fault was never the size of the allowance but the size of the thing it had to absorb. Sorting A8's fourteen judged comparisons by the room each had left splits them with nothing in between: its three Python runs at $p09s$ had -0.031, $+0.023$ and $+0.220$ ms of room, and the other eleven had 0.305 to 0.796. That rung is the shortest delay of the client that needs the least delay — 0.886 ms where the other client needs 1.66 — so a quarter of it is the smallest allowance in the campaign, while the drift it must cover is the same everywhere.	**Yes:** the same campaign. Checked against its runs, this rule would not have stopped it.
D17-3	The distance between a run's got-it median and the session's calibration is still computed and written beside the run, with the place its note was taken and the delay it added, and reported with the campaign. It stops nothing. P5(c) continues to be judged where this plan has judged it since version 3: in the session's own delay calibration, by two one-sided tests at each step against zero delay.	The calibration sweeps the added delay from nothing to eight milliseconds with everything else held fixed, which is the experiment P5(c) describes; a campaign varies the delay together with the trip it is placed at. Four calibrations read this way put the slope between -0.011 and $+0.003$ ms per ms, with the sign changing between loads on one pair. Keeping the number without the brake keeps the evidence and drops the false alarm.	**Yes:** read from the four calibrations and from A8's own campaign, whose within-campaign slope is $+0.004$ ms per ms.

What changed since version 15, and why

One brake, and nothing else. No prediction changes, no campaign is re-designed, and the brake is not loosened: a run that takes its "got it" note the way the calibration took it, and whose median moves as far as A8's did, stops exactly as before. What changes is which runs the comparison is made for at all.

No.	Change	Reason	Prompted by results?
D16-1	The got-it brake compares a run only against a baseline that took the note the same way the run takes it. Every calibration in this plan takes it at the client's own default, because stage 0 never asks for another; a run that takes it elsewhere has no like-for-like baseline.	A8 varies where the note is taken, and taking it inline forces the producer to one message in flight where the calibration ran with sixty-four. Those are two different producers on purpose: it is the treatment A8 exists to measure. Comparing them asks what that treatment did, and P5(c) asks only what the added delay did. On the Arm pair the Python run's median sat 0.167 ms from the calibration's where the brake allowed 0.145, and what moved it was the note's place: the Java runs, whose two medians happen to lie 0.06 ms apart, passed the same brake.	**Yes:** A8 on the Arm pair, which stopped itself on its first Python run, 3 of 144 done.
D16-2	Such a run is recorded rather than braked. Its got-it median, and where its note was taken, are written beside it, and P5(c) is asked of those runs by comparing the ones that share a note's place across the delays they ran at — which is the question P5(c) actually poses.	A check that cannot be made is not a check that failed. Version 5 already says that where the got-it cannot be held to the calibration the session is analysed with each run's own got-it distribution and the departure reported; this says which runs that applies to, and why, instead of stopping them. Nothing is lost: within a client and a note's place, the campaign runs several delays, and a got-it that follows the delay would show there.	**Yes:** the same campaign.
D16-3	A campaign counts from its own queue, before any run starts, how many of its runs the brake can judge and how many it cannot, and writes it beside the campaign.	The mismatch was there the moment the queue was made. It was found on the third run, three hours after the pair was started, by the campaign stopping itself — and the same campaign had already been restarted once that evening for a different fault in the same comparison, a calibration read as though it were not keyed by client. Both were visible in the queue and the calibration before anything ran.	**Yes:** both faults, on the same block, on the same night.

What changed since version 14, and why

The tools block only. No prediction of the law changes, no campaign of it is re-designed, and no tool campaign has run — T1 to T4 have not started on any machine, so nothing here reaches back to a result. What prompted it is a synthetic run of the block against made-up tools, which is a test of the design and never of the world: it says what T1 and T2 could resolve on the traffic this plan sends, and says nothing about how any real tool behaves.

No.	Change	Reason	Prompted by results?
D15-1	T2's offsets are taken from the trip they act on — the session's own median trip plus 1 ms and plus 3 ms — instead of a fixed 0.5 and 2 ms.	Half a millisecond against a 3 ms trip leaves every value positive: 0 of 3,000 go below zero. The experiment would have run on all ten tools, cost what it costs, and asked nothing. At 2 ms only tools reading a microsecond or finer answer. Simulated across the six classes at 3,000 messages, the trip plus 3 ms names every one of them and names none of them wrongly; the trip plus 1 ms names five of six.	**No machine result.** A synthetic run of the block, with no tool campaign yet run.
D15-2	What each behaviour would have produced is worked out from our own record of the same messages *after* those trips pass through the step T1 measured for that tool, and T2 is given that step.	A tool's rule for odd values runs on what its own clock handed it, not on the truth. These tools are written in Java and C, where integer division truncates toward zero, so a count of nanoseconds divided by a million turns −0.1 ms into 0 and not into −1. Everything between minus one step and plus one step lands on zero: a tool reading whole milliseconds never meets a negative and never drops one, however many trips were "really" negative. Predicting from the true trips instead has the tool dropping values it never saw, and then no behaviour fits what it printed. Five of the ten tools read such a clock.	**No machine result.** The same synthetic run.
D15-3	The verdict names what survives rather than what is nearest. Each behaviour is held against every figure the tool printed, to within what its printing can express, and ruled out by one it cannot account for, with the reason recorded. One left standing is the answer; two is a run that did not answer, and is reported as that. Where the same tool's reading with nothing moved is available *(T1's zero step)*, the comparison is on how far each figure moved.	Something is always nearest. On a tool whose clock is coarser than the gap between two behaviours, which is nearest is decided by the rounding and not by the tool: in the synthetic run the first form of this rule named a tool as replacing negatives with zero when its clock had erased them before its handling of them ever ran. That is the same rule the law's own predictions are held to — a point estimate inside a band is not a result when the instrument could not have put it anywhere else *(D14-2)*. Comparing how far a figure moved cancels the tool's own bias, which sits in both readings.	**No machine result.** The same synthetic run, which produced the wrong naming that this rules out.

No.	Change	Reason	Prompted by results?
D15-4	T2's identification is read from runs whose sends are spread evenly. Sends paced in step with the tick are run and reported as T-P2's evidence, and are not read as an identification. An offset is also recorded as too large where it leaves a tool nothing to report at all.	Paced sending is the adversarial case for the class that reads its clock twice, and this plan already predicts it: within 0.05 ms on evenly spread sends, up to 0.5 ms out when the sending is in step with the tick *(T-P2)*. It is a condition to measure, not a condition to identify under: in the synthetic run it is where the remaining wrong answers sit. At the trip plus 3 ms a tool that drops zero and below, sending on the tick, has nothing left to print.	**No machine result.** The same synthetic run.
D15-5	Each figure a tool prints is recorded with the step its own printing can express, and a tool is judged on the step of the figure being asked about, not on one step for the whole tool.	Kafka's EndToEndLatency prints `Avg latency: %.4f ms` beside `50th = %d`, and its ProducerPerformance prints `%.2f ms avg latency` beside `%d ms 50th`: an average that can show a tenth of a millisecond sitting next to percentiles that cannot show one at all. That gap is exactly what T-P1 and T-P2 predict, and one step for the tool reports the average as blind along with the percentiles and loses the finding. Read from each tool's own format strings.	**No.** Found in the tools' source, before any of them ran.
D15-6	The ten tools that will run are named, with the servers they speak to installed on the machine rather than in containers: Vegeta, hey, k6 and wrk2 against a fixed page this block serves; valkey-benchmark and memtier_benchmark against Valkey; Kafka's EndToEndLatency and ProducerPerformance and librdkafka's rdkafka_performance against Kafka; RabbitMQ PerfTest against RabbitMQ; and the NATS CLI against a NATS server.	The plan lists more tools than can be stood up at once, and which ten run is part of what it fixes. RabbitMQ PerfTest and the NATS CLI are kept because they are the only two the audit rates high, and T-P6 cannot be judged without them. A container puts a network namespace and a set of firewall rules between the tool and the broker, and this block measures tenths of a millisecond, so an extra hop would land inside the thing being measured.	**No.** A decision about what to stand up, taken before any of it ran.

What changed since version 13, and why

One guard added to a prediction, and the instrument that sets the rounds corrected. A3's Kafka campaign has run and is reported under version 9, the version in force when it ran; the guard below does not reach back to it. A3's Redis campaign, A2, A5, A8 and the tool campaigns have not run.

No.	Change	Reason	Prompted by results?
D14-1	The spread pilot *(P0)* measures the plateau and the floor as well as the spread, per backend and at the anchor slice, and the rounds rule is simulated at them. It has been simulated at a plateau of 0.30 and a floor of 0.01. The pilot reads, at the 3 ms anchor: 0.0346 and 0.0093 for Kafka on the first x86 pair, 0.0568 and 0.0123 on the Arm pair, and 0.0448 and 0.0161 on the second. A queue that can give neither level is refused rather than falling back to the old figures.	Every round count in this plan was set at 0.30 and 0.01, figures from the machines this work began on and from no pair that has run. The measured plateau is five to fifty times lower, and on Kafka the fall is under four times where the simulation assumed thirty. A shallower cliff is harder to locate, so those counts were set for an easier experiment than the one being run. The levels are read per slice as well as per backend: Kafka's floor is 0.0093 at 3 ms and 0.0321 at 1.5 ms, and one median over both is no slice's floor.	**Yes:** A3's campaign failing to resolve P3a is what sent us to look at what the rounds had been simulated at. The levels themselves are the pilot's, measured before any campaign ran.
D14-2	P3a is judged only where the ordinary plateau is at least 2%, which is the guard P4 has carried since version 4. Simulated at the levels A3 measured and at its 15 complete rounds: as frozen, P3a confirms in 9% of campaigns under the law and 0% where it is false; with the guard, 98% and 1%. The plan asks for at least 80% and at most 5%.	P3a asks where the cliff is at the lowest load a campaign runs. On this pair that load's plateau is 0.15%, and a curve falling from 0.0015 to 0.00047 does not fix a cliff position. Resampled from A3's own runs the halfway point there has a 95% interval 0.625 ms wide, against 0.141 and 0.237 at the other two loads, and it carries 87% of the variance P3a compares. The guard is not new machinery: it is the condition P4 already puts on the same plateau. The same answer follows without the simulation, from the widths A3 measured: the two remaining loads give a difference known to ±0.167 ms, so under the law the interval runs −0.167 to 0.167 and lies inside the band, while the 0.171 ms the falsifier moves the cliff over those two loads puts it at 0.004 to 0.338 and outside. That matters because the world these simulations are drawn from is quieter than the runs at these two loads *(D14-4)*, which would flatter the power rather than understate it.	**Yes:** A3's campaign. It is decided on the simulation above and the identifiability below, not on whether it would rescue that campaign's own answer, and it does not reach back to it.

No.	Change	Reason	Prompted by results?
D14-3	A load whose plateau falls below that bar is reported as out of reach, with the rate that excludes it, and the prediction is judged on the loads the pair can reach — as version 8 already does for a slice whose plateau lies under the client's own zero-delay trip *(D8-1)*. The finding behind it is recorded: on the first x86 pair, at 50% load, the cliff cannot be located, and no amount of running locates it.	Read off A3's own 270 runs without a model. Holding 5, 8, 11 and 15 of its real rounds, the halfway point's interval at 50% load runs 0.474, 1.111, 1.280 and 0.625 ms — it does not converge, where 75% narrows 2.8 times over the same range. Pooling runs so that a point carries the counts a three-times longer run would give moves that interval from 0.624 to 0.626 ms, while the same test at 75% narrows it 2.4 times on five times the messages, close to the square root counting predicts. The trouble is not precision, which more rounds or longer runs would buy.	**Yes:** A3's campaign, read back without a model.
D14-4	The made-up world counts. A run draws its negatives from the messages it sent, with an overdispersion, instead of taking a rate multiplied by a lognormal spread; the cliff moves from one run to the next, so the curve's own slope decides how much of that reaches the rate; and the levels may be given per load.	One spread cannot hold what A3 measured. Its scatter at 50% load is within a fifth of what counting four to seven negatives produces by itself, while at 75 and 88% the counts are in the hundreds and most of the scatter is the fall's position moving: solving for that wobble gives 0.257 ms at one load and 0.245 at the other, one number for both. The old form also cannot represent the messages a run sends, so it cannot be asked whether a longer run buys what another round buys — which D14-3 needed answered. Reproducing every point's scatter is not yet within it, and that is recorded rather than claimed: what it does reproduce is the steep part of the fall, where the halfway point is read, and the outcome A3 had.	**Yes:** A3's campaign is what it was held against.
D14-5	The rounds cannot be set for the tick predictions, and that is stated rather than left to look like a number. The rounds rule simulates one tick at a time, and P2, P2b and P2c compare two kernels. A2's fifteen campaigns run at the floor of 4 rounds until it can.	Found while re-deriving the counts: asked for P2, the rule answers that the prediction never confirms, which is what it now says where a rule cannot be satisfied rather than where it is weak *(D13-2)*. A2 is the largest block in this plan, 682 runs over 15 campaigns, and its repeats have never come from a simulation.	**No.** Found in the instrument, with no tick campaign yet run.

What changed since version 12, and why

Two corrections to the instrument, and nothing to any prediction. A8 had not run, so everything there was written before the runs it would be judged against. A3 was running under version 9 and was untouched by it; A1, A4 and A7 had finished unanalysed.

No.	Change	Reason	Prompted by results?
D13-1	The program that judges P8 read the middle of the cliff from $s + 0.4h$, a trip A8 does not run; it now reads whichever trip across the cliff the campaign carries, nearest the middle first. On the four-trip ladder that is the trip it already read, so no other campaign's judgement moves. A8 runs **6 rounds**, from P8's corrected simulation at each pair's own Kafka spread *(0.250 on the x86 pair, 0.256 on the Arm pair)*: confirmed in 88% and 86% of campaigns under the law, and in 0% of campaigns where it is false. That is 288 runs over the two campaigns where all three trips are reachable.	P8 has asked about the rate at $s + 0.5h$ since version 8 *(D4-9)*, and A8's design has run that trip since the same version. Only the program disagreed, and it disagreed in the direction that cannot fail: a rule that names a trip its own campaign never runs is not a hard test but an unsatisfiable one, and it reported as a weak prediction rather than a broken rule.	**No.** A simulation of the rule against its own campaign's design, before any A8 run exists.
D13-2	Where a rule confirms in no simulation at all, at any number of rounds, the rounds rule says so instead of answering with the ceiling and the word underpowered.	The two are different things and read alike: 0% at every step is not a weak test of the law but a rule that cannot be satisfied, and rounds cannot mend it. Reported as underpowered it would put a broken instrument in the plan's table as a weak result, at the ceiling's cost.	**No.** It follows from D13-1: the defect was visible in the simulation's own output and was read as a number instead.
D13-3	P8's ratio is taken only at the trips both clients ran, and its answer records which those were and whether there were any. Runs a trip only one client reached are kept and reported as before; they are left out of the ratio alone.	The plan takes the ratio "at matched setups", and each client is calibrated on its own because the delay's effect on the trip belongs to the client *(D4-9)*. Their zero-delay floors therefore differ, and a trip can be out of reach for one and not the other *(D8-1)*. Where the trip lost is the plateau, P8's ratio is undefined, and an undefined ratio was reported as a prediction that did not hold rather than as a campaign that could not test it — after the campaign had run.	**No.** Found by placing A8's trips from two calibrations that differ, which is what two clients calibrated separately will give.

No.	Change	Reason	Prompted by results?
D13-4	Both clients measure the clock they are about to stamp with, refuse to start where it steps coarser than 25 μs, and record what it read in the run's own files. The Java client has refused since it was written, at 100 μs; the bar is now 25 μs and the Python client holds it too.	Metrology's rule of ten: an instrument should divide the tolerance being judged into about ten parts, and below four to one it is not a measurement of that tolerance at all. The tolerance is not the cliff's width but the narrowest bound this plan pre-registers — P2d asks whether the fitted start stays within a quarter of a millisecond, and P3a whether the halfway point's whole interval lies inside 0.25 ms either way. Ten parts of 250 μs is 25. At 100 μs the ratio against those two bounds was 2.5 to 1, under the four-to-one floor. Measured: Windows steps in 998,600 ns whether the runtime is the JVM or Python, which is the operating system's answer and not the client's; the Linux testbed reads 1,000 ns from the JVM and 232 ns from Python. A8 is therefore run on Linux, and the clients enforce that themselves.	**No.** The clocks of the machines were measured; no campaign was run.

What changed since version 11, and why

The tools are read and their predictions fixed. No tool campaign has run, so everything there is written before the runs it will be judged against. On the law's side nothing changed: A3 was running under version 9, and A1, A4 and A7 had finished unanalysed.

No.	Change	Reason	Prompted by results?
D12-1	The seven tools that carried a question mark are classified, and the classification is the prediction T1 to T4 will test. **Vegeta: high** *(monotonic nanoseconds that survive into the subtraction; nothing dropped, clamped or replaced)*. **valkey-benchmark, memtier_benchmark, librdkafka's rdkafka_performance, YCSB: medium** *(microseconds)*. **k6, hey: low**. Each tool's version is pinned as read. The falsifiable content is stated per tool: that Vegeta's per-request latencies track our own instrument to within the sub-millisecond and that it never emits a negative on a live run; that hey's output is quantised in steps of 0.1 ms, so differences our instrument resolves below that are invisible in it, and that a run of fewer than a hundred requests prints a 99th percentile of exactly zero; that no memtier percentile reads below 0.008 ms whatever the true latency; and that valkey-benchmark's reported minimum is 0.000 ms for any reply faster than 8 μs.	The plan requires a read-first tool to be classified before any run and states that the classification is itself a prediction *(Section 4)*. Seven were unread and are now read. Fixing them here is what makes them predictions rather than notes: the audit's readings are on the public record before the campaigns that will test them.	**No.** Documentation and source, read without running any tool.
D12-2	One earlier classification is recorded as refuted. RabbitMQ PerfTest was classified "keeps all, negatives included". It does not keep them: its difference function returns the size of the difference, so a negative becomes a positive latency of the same magnitude. Under the cross-machine mode its own documentation prescribes, a consumer clock behind the producer's reports the clock offset as delay, with nothing in the output to distinguish it. The corrected reading stands for the runs that follow; the prediction stands as made, and as failed.	A prediction that fails is reported, not amended away. It also fails in the more damaging direction than predicted: a visible negative might alert a reader, a sign-flipped one never will. Its tracker has never carried a report of it – the three sign and clock terms return nothing across 215 issues.	**Yes:** the audit refuted it. The audit reads documentation and source and no measurement of ours.

No.	Change	Reason	Prompted by results?
D12-3	Two limitations of the audit's own instrument are recorded with its results rather than repaired. First, the eight search terms are frozen with the plan and two of them earn little: "clock skew" returns nothing in projects whose trackers carry extended debates about unsynchronised processor clocks, because those threads use other words, and "negative latency" returns almost nothing anywhere. Had the protocol leaned on those two it would have found nothing in several projects. The terms were not changed mid-audit and are not changed here. Second, a tool's handling of odd values can differ between versions – one tool's guard on its histogram was removed in January 2025 – so every reading is recorded against the version read, and a tool campaign records the version it ran.	An instrument's weaknesses belong in the record beside what it found. Changing the terms after seeing which ones worked would make the counts unreportable.	**Partly:** the weakness of the terms was learned by running them. What they found is unaffected; what they may have missed cannot be quantified, and that is stated rather than estimated.

What changed since version 10, and why

One thing, and it is about A5's instrument rather than about anything A5 will find. Version 10 was frozen earlier the same day; A3 is running under version 9 and is untouched by this. A1, A4 and A7 have finished and none has been analysed.

No.	Change	Reason	Prompted by results?
D11-1	An A5 session measures its own baseline *(B0)* at the core count it will run, at the load its campaign runs, before its own calibration *(C0)*. Three sessions, six baseline runs each: eighteen runs in all, about an hour of machine time. The trips of that session's campaign are placed from that session's own baseline and calibration, and the campaign's design records which baseline placed it.	The plan already gives each core count its own session with its own calibration *(D4-6)*, and the reason it gives is that the delay's effect on the trip must be measured on the machine as the campaign will run it. A machine with six of its eight CPUs switched off is not the machine the calibration was taken on — and it is not the machine the baseline was taken on either. The baseline is the zero-delay trip that every target trip is placed from, the floor the law's plateau sits above; it moves with the core count for exactly the reasons the calibration does. Naming one and not the other left A5's two-CPU session placing its trips from a floor measured at eight, which puts every trip in the wrong position relative to the cliff the campaign exists to find. An instrument is calibrated in the state it measures, or it is calibrated for a different machine.	**No.** It follows from the plan's own argument for the calibration, applied to the other half of the same instrument. Nothing measured prompted it and A5 has not run.

What changed since version 9, and why

Nothing in this plan's rules changes here. What changes is a practice that departed from them, and this section records the departure rather than quietly repairing it. A3 is running under version 9 as this is written; A1, A4 and A7 have finished, and none of the three has been analysed. What prompted this version is a measurement of the instrument – each pair's spread pilot, read per backend instead of pooled – and a simulation of the decision rule. No result that tests a prediction was read to reach any of it.

No.	Change	Reason	Prompted by results?
D10-1	The spread a campaign is simulated at is the spread its pair's pilot measured for the backend that campaign runs. One figure per backend, never the two pooled. Where a block's campaigns run different backends they may now run different numbers of rounds, and each campaign's log states the spread its own number came from.	The plan has said this since version 4 *(D4-2, and the statistics section in every version since)*: the simulation uses the spread measured on that pair *for that backend*. The practice did not. One median was taken over every setup the pilot ran, both backends together, and used for both. Pooling two variances is licensed only by assuming the populations share one, and these pilots refute it: 0.250 and 0.423 on the first x86 pair, 0.256 and 0.289 on the Arm pair, 0.213 and 0.701 on the second x86 pair. The last is a factor of 3.3 in the scatter and about 11 in the variance; their pooled median, 0.457, is an estimate of neither.	**No.** An instrument measurement from each pair's spread pilot, and simulations of the decision rule.
D10-2	What the departure cost, campaign by campaign, checked by re-running the rule at each backend's own spread. **A1:** its Redis campaigns were re-simulated at 0.602, well above Redis's true 0.423 on that pair, and 4 rounds still confirmed P1 in 83% of campaigns under the law and 0% where it is false. Its Kafka campaigns ran at a pooled 0.284 where Kafka's own spread is 0.250, so they are quieter than the number used. **A4:** its Kafka spread, 0.256, is the pooled figure exactly, so nothing changes; its Redis campaigns ran at that 0.256 where Redis's own spread is 0.289, and re-simulated at 0.289 P9 is confirmed in 99% of campaigns under the law and 0% where it is false, comfortably above the bar. **A7:** one campaign over both backends at one slice, so the pooled figure is the one its analysis pools over too.	A departure that changes no finished result still has to be shown not to, campaign by campaign, rather than asserted. Where the check had to assume something it assumed the worse case: A1's Redis was re-simulated at a spread half again as large as the one measured.	**No.** Simulation only; no campaign's data were read.

No.	Change	Reason	Prompted by results?
D10-3	A5's rounds follow from this: 4 for its Kafka campaign, at a spread of 0.213, and 16 for its Redis campaign, at 0.701. The block's two campaigns therefore run different numbers of rounds, which D4-2's rule that a block's campaigns share a number does not survive here and is set aside for A5 *(the rule was written for blocks whose campaigns are compared with one another through a shared anchor; A5's two campaigns are judged separately, one per backend)*.	At the pooled figure both campaigns would run 8: twice what Kafka needs, and half what Redis needs to reach the power the rule asks for.	**No.** Simulation only.

What changed since version 8, and why

Version 8 was frozen on 18 September 2026 *(freeze 04)*, and the campaigns it covers began that night: A1 finished on the first x86 pair, A4 on the Arm pair and A7 on the first pair, all without a failed run. Nothing they measured prompts anything here. What prompted this version is the rounds rule itself, run for A3 before A3 ran: it asked for more rounds than its own ceiling allows and still fell short, and the reason turned out to be the shape of the prediction rather than the size of the campaign. No measured run was read to reach this change, and none of the three finished blocks has been analysed.

No.	Change	Reason	Prompted by results?
D9-1	P3 becomes two predictions, judged separately. **P3a:** load leaves the cliff where it is. **P3b:** load raises the plateau. Each has its own falsifier: for P3a the halfway point moving 0.5 ms between 50% and 88% load, as before; for P3b a plateau that does not move with load at all. A3 still runs at the 3 ms slice, three loads and six trips, in two campaigns, one per backend *(D4-4)*; its rounds come from P3a.	A prediction here is confirmed only where every part of it is confirmed, so a prediction of two parts is confirmed about as often as its weaker part *(statisticians call this an intersection–union test, and its power is no more than the least of its components)*. Simulated a thousand times at the first pair's measured spread, the cliff part held in 97% of campaigns under the law and the plateau part in 20%: P3 as written held in 18%. And the plateau part held in 20% of campaigns in the world P3's own falsifier names, because that falsifier moves the cliff and leaves the plateau alone. A part that answers the same in both worlds carries no evidence about which world we are in; joined to another by "and", it can only subtract.	**No.** The rounds rule is a simulation of the decision rule. No measured run was read.

No.	Change	Reason	Prompted by results?
D9-2	P3a is judged by the whole interval, not by the point estimate: the 95% interval of the halfway point's movement between the lowest and the highest load must lie inside ±0.25 ms. The difference is taken signed, not as a size.	P3a claims that something did *not* change, and the standard for such a claim is that the whole interval sit inside the band — two one-sided tests, as in equivalence testing and in the bioequivalence rules that have used it for decades. Version 8 asked only that the point estimate be inside the band, which a scattered estimate can satisfy by luck: simulated under the law, that reading confirmed 97% of campaigns at four rounds, where the interval reading confirms 23%. The looser reading is the more flattering one, which is reason to distrust it rather than to keep it. Taking the difference signed matters too: folding the draws about zero pushes the interval's top outwards and would refuse sound campaigns for a reason that is arithmetic rather than physical.	**No.** Simulation only.
D9-3	A3 runs 16 rounds, so 576 runs across its two campaigns, in place of the 216 version 8's table estimated. P3b is expected to be reported as underpowered at that number, and that expectation is written into the campaign's log before it runs.	The rounds are what the rule returns for P3a at the first pair's own measured spread. P3b would need far more rounds than the ceiling allows, and the plan's ceiling rule covers exactly that case: the campaign runs, and the prediction it cannot power is reported as underpowered rather than quietly dropped or quietly believed.	**No.** The spread is an instrument measurement from that pair's spread pilot, taken before any campaign of this block.

What changed since version 7, and why

Version 7 was frozen on 18 September 2026 *(freeze 03)*, and stage 0 ran under it that night on all three pairs: the first pair passed its staircase calibration and its baseline and spread pilots, the Arm pair passed all of stage 0, and the second x86 pair was stopped by a rule halfway through its spread pilot. Nothing the law predicts changes here. What changes is which slices count as a test of it on a given pair — decided by each pair's own calibration, before any campaign of the block runs — and where one emergency brake sits. The session stopped on 18 September is reported with its cause, and the pair starts a new one.

No.	Change	Reason	Prompted by results?
D8-1	A slice is a test of the cliff on a pair only where that pair's runs can reach the level the halfway point is measured from *(*$0.9s$*)*, the level past the cliff *(*$2(s + h)$*)* and at least three of the four trips across it. A slice that cannot is reported as out of reach, with the zero-delay trip that excludes it, and is not run. P1 is judged on the slices the pair can reach: the halfway point inside its band in all but one of them, and at least four such slices. P9 asks for all of them, and at least two. The slope rule is unchanged.	A message cannot arrive sooner than the client's own zero-delay trip, and the law's plateau lives at trips below the slice. On 18 September the first pair's calibration measured that floor at 1.964 ms for Kafka and 0.837 ms for Redis. At 0.75 ms Kafka can place 2 of the 8 trips and at 1.5 ms 4 of 8, and neither reaches $0.9s$: there is no plateau to fall from, so those slices cannot test the prediction however the law behaves. Reachability is read from the calibration, an instrument measurement taken before any campaign of the block runs.	**Yes:** the first pair's calibration of 18 September
D8-2	The brake that stops a session when a run's "got it" median moves must clear the instrument's own noise as well as its share of the delay: more than a quarter of the added delay *and* more than the larger of 0.10 ms *(the margin P5(c) already gives the got-it median)* and three times the scatter of the session's own zero-delay runs. Each session's calibration records that scatter.	On 18 September the second x86 pair's spread pilot stopped on a shift of 0.268 ms where 0.225 ms had been added, a brake set at 0.056 ms. Across four zero-delay runs of that pair's own calibration, with nothing added at all, its Kafka got-it median ran from 1.718 to 2.077 ms *(SD 0.183 ms)*. A brake below the noise of the thing it brakes stops sound sessions and says nothing about the instrument.	**Yes:** the stopped session of 18 September and that pair's own calibration
D8-3	The pilot's harness verdict holds the got-it median to the same 0.10 ms and records how far the trip moved per millisecond added, instead of failing when it does not move one-for-one.	The harness replays the match in bursts, and in bursts the trip does not grow one-for-one — which is why every session's calibration judges that on the campaign's own steady traffic, as version 3 already fixed *(D3-5)*. The verdict was failing on a number it does not decide, and holding the got-it to half the margin the plan gives it.	**Yes:** every pair's pilot on 18 September

What changed from version 6 to version 7, and why

Version 6 was frozen on 17 September 2026 *(freeze 02)*, and the first pair started again under it that evening. Its shakedown turned the pair away on the check version 6 had just added, and the measurements that followed showed the check itself was at fault: it read the paths with ping, and ping is not the path our messages take. This version moves every judgement onto the instrument that reads the treatment where it is applied, and leaves ping as a record. Nothing else changes: no prediction, no campaign, no number of rounds, no rule that decides a prediction. Sessions already judged under version 6 keep version 6.

No.	Change	Reason	Prompted by results?
D7-1	Every run's added delay — the number its check is held to, and the number its calibration is fitted at — is read from the broker's own capture of that run's pings: how much longer it held the replies to the receiver than the replies to the host, within 0.05 ms of the delay set. A run whose capture is missing is repeated. Ping's own figure is written down beside it and decides nothing.	The capture reads the treatment where it is applied: on all three pairs it came within 0.022 ms of the delay set. Ping does not read the path our messages take. Measured side by side on 17 September, over ten blocks in nine minutes, ping read 0.859 and 1.127 ms on the two paths where TCP read 0.447 and 0.611 and UDP 0.439 and 0.695; ping's 90th percentile was 3.2 to 3.7 ms against TCP's 0.46 to 0.62, with single readings of 8.75 and 11.1 ms. Microsoft's own guidance for Azure says ICMP is treated differently from application traffic and names a TCP or UDP tool instead.	**Yes:** the shakedown of 17 September and the side-by-side probe that followed it
D7-2	With no delay, the two paths are measured by ping, by TCP and by UDP, and recorded. They no longer decide whether a pair may run. What a pair's runs vary by is measured by B0 and sets its rounds, as version 4 already fixed *(D4-2)*.	On the same probe, ping put the two paths 0.283 ms apart, steadily, where TCP put them 0.009 ms apart: in the receiver's namespace the difference is an artefact of how ICMP and UDP are handled, not something our messages meet. On 17 September that figure turned away a pair whose messages were unaffected — its trips matched the previous session's within noise — and a gate on a proxy also chooses which machines contribute data.	**Yes:** the pair turned away on 17 September, and the probe
D7-3	Every session installs the two measuring tools and records what they read: tcpdump, which times the delay at the broker, and sockperf, whose TCP and UDP round trips on both paths are kept with the machines' facts.	A number that decides nothing still has to be there to explain a pair afterwards. Nothing in the record said what the paths looked like in TCP terms until 17 September, when the question could not be answered without new measurements.	**Yes:** follows D7-1 and D7-2
D7-4	The session of 17 September that was turned away is reported as not admitted, with its cause, and the pair was stopped, started again and admitted. Its runs test nothing.	A check that was later found to be the wrong instrument still produced a verdict, and the record has to show both the verdict and the correction.	**Yes:** that session

What changed from version 5 to version 6

Version 5 was frozen on 16 September 2026 *(freeze 01)*, and stage 0 began that day on three machine pairs. Those runs found faults in the machines, in our code and in two of version 5's own checks, and they were stopped. This version corrects all of them, and stage 0 starts again from the beginning under it. A change prompted by results is exactly what a reader must be able to see, so the table says which result prompted each one. No prediction, and no rule that decides a prediction, changes. The runs of 16 September test nothing; they are kept and reported, and what they showed is the reason for the changes below.

No.	Change	Reason	Prompted by results?
D6-1	P5(b)'s second part asks that the calibration be *known* within 0.3 ms at every step: its 95% interval there, from the scatter of the runs about the line, or within the steps where the step medians are joined, lies within 0.3 ms of it. Version 5 asked every run to lie within 0.1 ms of the calibration. How far each run and each step median lies from it is still reported.	A limit on every run tests how much runs vary, not the calibration, and more runs make it fail more often. On the second x86 pair, run trips scattered around the calibration with a standard deviation of 0.27 ms *(Kafka)* and 0.18 ms *(Redis)*. Simulated, 0.1 ms on every run fails every such session, and more than half even at the first pair's 0.06 ms. A check on the step medians alone would pass by itself whenever the calibration *is* the joined step medians, which is when a calibration from few runs is least certain. An interval means the same under both forms. Every run is analysed at the trip it actually had *(Section 11)*, so this limit sets how closely runs land where they were aimed, not what they show.	**Yes:** the second x86 pair's calibration failed version 5's check. The first pair's calibration, 51 of its 56 runs, was also fitted before this version was written.
D6-2	C0 runs in two stages on every pair but the first: 2 rounds, then 2 more, fitted together with the first 2, only when the calibration's precision *(D6-1)* is the one part of its gate that failed. The first pair's staircase S0-1 keeps its 4 rounds.	More rounds only where the data need them, by a rule fixed now. Simulated at the spreads seen: at the first pair's 0.06 ms no session needs more rounds; at 0.18 ms 16% do, and none then fails; at 0.27 ms 77% do, and 7% then fail. A second stage adds about 1.5 hours to a session.	**Yes:** follows D6-1 and the second pair's spread
D6-3	The shakedown's delay check is made by the broker's own packet capture: at each step, the broker holds the receiver's replies for the set delay beyond the host's, and the host's no longer than with no delay, within 0.05 ms. Ping still checks the delay end to end, within 0.25 ms plus 5%, the limit every run is held to.	On the Arm pair, one check by ping varied with a standard deviation of 0.03–0.04 ms, and the zero-delay reading drifted by up to 0.1 ms within half a minute. Ping cannot decide 0.05 ms there: the Arm pilot failed the check although the capture shows its delay right to within 0.01 ms. The median of five ping readings would still fail about 8% of sound Arm shakedowns. The capture times the delay where it is added, to a few microseconds, on both processors *(between −0.001 and +0.022 ms on all three pairs)*.	**Yes:** the Arm pair's shakedown
D6-4	With no delay, the host's and the receiver's ping round trips must agree within 0.25 ms, as the median of five readings. Each pilot keeps both machines' network and timing facts. A pair that passed its shakedown repeats this network part in every later session.	The receiver-only delay assumes the two paths are the same apart from the delay. Captures on both machines put the gap in Azure's network, not in either machine, and it moved between +0.06 and −0.44 ms from one start to the next: a start after a stop can put a machine on another physical host. The limit is the one every run's delay is held to, and it was chosen after the gaps were seen: 0.44 ms on the second x86 pair on 16 September, 0.11 and 0.17 ms on the two x86 pairs on 17 September.	**Yes:** the second x86 pair's paths, and the probes of 17 September

No.	Change	Reason	Prompted by results?
D6-5	Every run's added delay is measured beyond the same ping taken with no delay just before that run, and the broker's capture of the run's pings is kept beside it.	With no delay, the second x86 pair's paths differed by 0.44 ms, so its zero-delay runs read as −0.44 ms of delay and were repeated. A difference that is there without any delay is not delay. The capture gives the delay to the microsecond at the 16 and 32 ms steps, where ping prints only tenths *(D6-13)*.	**Yes:** the second pair's first calibration runs
D6-6	Every campaign run uses the paper's own client settings, and each client's log must show that they took, or the run is repeated: Kafka's producer keeps up to 64 requests in flight, as before, and Redis's consumer acknowledges messages in batches of 200, which is new.	The supplement names both settings as learned; the law campaign passed only Kafka's. With one acknowledgement per message, a message waits one round trip for every message read before it. At 16 ms of delay the Redis consumer fell behind to six messages a read and 128 ms trips, which measured its own backlog, not the delay; at 32 ms it would never catch up. On the campaign's steady traffic, up to 8 ms, every read returned one message *(6,500 of 6,500)*, so nothing changes there. Batches of 200 send one acknowledgement every 4 s, which can make at most one message in 200 wait one extra round trip. This setting failed to apply, silently, three times in earlier work; Kafka's producer now writes its setting down too.	**Yes:** the first pair's 16 ms staircase step
D6-7	M0 replays the football match in bursts, as the pilot did, and changes the Redis consumer's acknowledgement between one per message and batches of 200.	The departure from one-for-one that M0 exists to explain appears only in bursts. In all five pilots, 2 ms of delay lengthened Redis's trip by 2.5–3.2 ms and Kafka's by 1.7–2.0 ms, while on the campaign's steady traffic both x86 calibrations found slopes of 0.99–1.02 up to 8 ms. In the pilot's replay, 46% of sends came under 3 ms after the one before, and 11–13% of Redis reads return several messages, each waiting for the one before it to be acknowledged. M0 as frozen would have run steady traffic and found nothing to explain. Changing the acknowledgement is an intervention on M-H2, the receive loop; M-H1 to M-H4 and their tests are unchanged.	**Yes:** the calibrations and pilots of 16 September
D6-8	Before every run the driver checks that the receiver's address is only in the receiver's namespace, that the broker answers, and that no package manager is running; a failed check fails the run. Automatic package upgrades are off on every machine, and the receiver's address is removed from the driver's own network settings.	On 16 September an automatic upgrade restarted the first pair's network service during its pilot. The service put the receiver's address back on the driver, the driver lost its broker, and the load read 78.7% instead of 75%. The shakedown failed, as it should.	**Yes:** a fault of the machines in a pilot

No.	Change	Reason	Prompted by results?
D6-9	Each run's folder carries its campaign's name, and a run never writes into a folder that already exists: the campaign stops instead.	Run keys repeat from one campaign to the next. On 16 September a restarted calibration wrote its first run into the folder of the attempt before it, and the watch showed a 35-minute trip.	**Yes:** a fault of our code
D6-10	A session broken by a fault, or stopped to fix one, before its calibration's gate is *void*: kept, copied, and reported with the cause, never analysed. Everything a driver measured outside a campaign *(a pilot, a void session, a probe of the instrument, a log)* is copied and fingerprinted like a campaign, and a pair's whole folder of runs is copied before the pair is deleted. Every run also keeps the TCP counters of both machines and the broker's own log.	Four sessions were void on 16 September, two on each x86 pair: one broken by the upgrade of D6-8, one by the zero-delay reading of D6-5, and the two running when the fault of D6-9 was found. Kafka runs on both pairs held pauses of 0.15–0.9 s, in 18 of 54 runs and never more than 39 messages, whose cause no file could show; a TCP segment sent again, or the broker pausing, would show in these files.	**Yes:** faults of the machines and of our code
D6-11	A failed calibration gate ends the session, and the pair starts a new one, with a new C0. The 20-minute deallocation of an idle pair stays; no session is held open while a rule changes.	Version 5 said only that the main runs do not start. Holding a session open across a rule change would judge its data by a rule written after they were seen. A new C0 and B0 cost about two hours.	**Yes:** a draft of this version would have carried a running session across the change
D6-12	Stage 0 starts again from the beginning on every pair: the shakedown, then S0-1 on the first x86 pair and C0, B0 and P0 on the others. After any change to the code, the first x86 pair runs alone until its shakedown and first calibration pass, and then the others start.	Every run this plan analyses is judged by rules written before it. On 16 September every fault reached all three pairs at once; a staggered start makes a fault in new code cost one pair a few hours.	**Yes:** the runs of 16 September
D6-13	Delays measured by ping are known to the precision ping prints: 0.001 ms below 1 ms, 0.01 ms from 1 to 10 ms, and 0.1 ms from 10 to 100 ms. The broker's capture *(D6-5)* gives the finer value at the 16 and 32 ms steps.	All eight runs of the first pair's 16 ms step read the receiver's round trip as exactly 17.0 ms.	**Yes:** the first pair's staircase

What changed from version 4 to version 5

Version 4 was prepared for freezing but was never frozen, and no run of the campaign has taken place. A change prompted by what a pilot showed is exactly what a reader must be able to see, so every change is listed here and tagged where it appears. Version 5 adds how the campaigns are run, checked, stopped, watched and kept (Section 13). No prediction, campaign, number of rounds or decision rule changes.

No.	Change	Reason	Prompted by pilot results?
D5-1	Every new machine pair passes a shakedown before its first campaign: the pilot's scheduler-settings, receiver-only-delay, load and never-negative checks. Its harness-delay check is replaced by the pair's own C0, and its go-first result is recorded but is not a gate (Section 13).	The first pair's pilot caught a false assumption before any run counted *(D3-5)*. A pair in another region or on another processor gets the same chance to fail before its runs count. Go-first is left to A8, which tests it as a prediction.	Partly: what the first pilot caught
D5-2	Every run is judged as it ends, on its own driver, by fixed limits with three verdicts: it counts; it is repeated, when a condition it was meant to have did not take; or it stops its campaign, when the instrument is in doubt. Two checks are added: at least 99% of the planned messages sent and arrived, and the added delay within 0.25 ms plus 5% of its setting. P5(c)'s stop is applied run by run, against the session's zero-delay "got it" median.	Version 4 listed the integrity checks but not their limits, when they are applied, or what follows a failure. Limits applied by a program cannot be bent after a result is seen, and they hold when nobody is watching.	No
D5-3	A campaign stops itself when its attempts keep failing: the last 3 all failed, or more than 20% of the last 20, once 10 have finished. A run is attempted at most 3 times, and a run whose third attempt fails is abandoned and reported.	A machine that has gone wrong at night should cost minutes, not a night of failed runs.	No
D5-4	A finished campaign's runs are copied from its driver, with a SHA-256 fingerprint for every file checked after the copy, before the next campaign starts on that pair. The fingerprints, and a record of the machine, the code version and the time, are kept with the data. A pair is deleted only after all its campaigns are copied.	Results get lost silently in copies. A fingerprint makes a truncated or changed file visible, and copying after every campaign means a lost machine costs at most the campaign in progress.	No
D5-5	A watch outside the testbed reads every pair every 5 minutes, raises alerts, warnings and notices, and deallocates a pair idle for 20 minutes.	Campaigns run unattended, often at night. A stopped campaign or a broken machine is seen within minutes, and idle machines stop billing.	No
D5-6	Every run records its pair, its driver and the version of the code *(the commit)* that ran it. All pairs run the same version, and a campaign's queue runs on one pair only.	D4-10 records the pair. Recording the code version too ties every result to the exact code behind it, and one version on every pair keeps the pairs comparable.	No
D5-7	All three machine pairs run in Sweden Central, so the second x86 pair does not go to Italy North. The subscription's limits there were raised on 16 September 2026 to 40 CPUs in total, 20 in the x86 family and 10 in the Arm family.	One region is cheaper *(EUR 0.4165 an hour a pair against 0.4577)*, puts both x86 pairs on the same hardware generation, and lets all three pairs run at once, about 5 days. The prices were read again on 16 September and had not moved.	No

What changed from version 3 to version 4

Version 3 was approved but not frozen.

No.	Change	Reason	Prompted by pilot results?
D4-1	The work is split into 47 small campaigns, each answering one question in one sitting *(3.5 to 8.4 hours at the floor of 4 rounds)*, with the machines stopped between them (Section 12). Version 3's two stages are replaced.	A design decision, 15 September: results arrive in pieces that can each be checked, and no machine waits idle between blocks.	No
D4-2	Rounds are set for each campaign by simulating its own prediction and decision rule at the run-to-run spread its machine pair measured, with a floor of 4 and a ceiling of 40. Tool campaigns run 4 rounds. This replaces version 3's fixed floors of 15 and 10 (Section 11).	The fixed floors came from old setups whose spread was about as large as the rate itself. The pilot's repeat runs varied far less, and simulating the actual decision rule measures power directly.	Yes: the pilot's run-to-run spread
D4-3	A1 runs as six campaigns, three per backend, each including the 3 ms slice as a shared anchor. Each campaign's offset is removed through the anchor before slices are compared.	The slices no longer share one session; the anchor keeps the comparison fair across days.	No
D4-4	A3 tests one slice *(3 ms)* instead of two, in two campaigns, one per backend. P3 is judged at that slice.	Earlier campaigns *(E-A3, E-A6)* already showed load raising the negative rate. What is new is whether load moves the cliff, and one slice tests that.	No
D4-5	A7 tests one slice *(3 ms)*, both backends, in one campaign. P4 is judged at that slice.	Go-first priority was shown on the earlier machines *(E-A5)* and again, 27-fold, in the pilot on this machine. What is new is its effect at the cliff.	Partly: the pilot reproduced go-first
D4-6	A5 runs as two campaigns, one per backend. Each core count is its own session with its own calibration.	Switching CPUs off changes the machine, so the delay's effect is measured again.	No
D4-7	A2 runs as a build campaign, twelve sessions *(kernel × day × backend)* and two bridge sessions on the stock kernel.	Sessions of about 3.5 hours, with each kernel on two days in a balanced order.	No
D4-8	A4 opens with its own instrument campaign on the Arm pair *(C0, B0 and P0)*, then four campaigns: two per backend, at slices of 1.5 and 3 ms, and of 3 and 4.5 ms.	The repeats rule needs each pair's own spread *(D4-2, D4-10)*.	No
D4-9	A8 is kept, narrowed to Kafka, whose official client is written in Java, in two campaigns: one on an x86 pair and one on the Arm pair. P8 says how the cliff is judged at three trips.	The earlier client comparison *(M1)* tested a second Python client on a different delay, so it does not answer P8.	No
D4-10	Several machine pairs may run at the same time. Every prediction is tested on runs from one pair, every pair opens with its own B0 and P0, and every run records its pair.	A shorter calendar, without mixing machines inside any test.	No
D4-11	Straight reruns of earlier campaigns are dropped. A new section lists what earlier campaigns already answered and what is still run (Section 6).	Nothing is repeated that a new prediction does not need.	Partly: the pilot reproduced go-first

No.	Change	Reason	Prompted by pilot results?
D4-12	Stage 0 opens with a staircase campaign *(S0-1)*: added delays up to 16 ms, both backends, 4 rounds, before any other campaign.	A two-round session calibration has little power to test whether the line is straight; this campaign has more, and it shows what later calibrations should look like.	Yes: the pilot's delay result
D4-13	The plan is frozen by committing it in full to the public repository before the first Stage 0 run; a later change is a new, numbered freeze (Section 15).	Version 3 listed three ways and left the choice open.	No
D4-14	P9 is restated for what A4 measures: three slices at one load, with the anchor rule of A1.	Version 3's P9 also named P2 and P3, which A4 never tested. A prediction must match its experiment.	No

What changed from version 2 to version 3

No.	Change	Reason	Prompted by pilot results?
D3-1	The paper's three parts frame the plan, and the tools block *(T1–T4)* is in this paper.	A design decision, 15 September.	No
D3-2	Azure only. A4 compares Azure's Arm machine with its x86 machine; the Oracle options are dropped.	A design decision, 14 September.	No
D3-3	Default slices are predicted from the per-CPU constant read off each machine, not from the kernel's version *(A5, P7)*.	The Azure 6.8 kernel carries the constant Linux 6.15 adopted: 2.8 ms at 8 CPUs, where the 6.8 rule gives 3 ms. A reading of the machine's settings, not an experimental result.	No
D3-4	New section: what the pilot showed.	A record of the pilot.	–
D3-5	Version 2's built-in delay check is replaced by a calibration in every session *(C0)*. The "got it" check becomes an equivalence test. The law is analysed at each run's measured trip. P5 is rewritten.	The pilot showed the trip does not grow one-for-one with the added delay, and in opposite directions for the two clients.	**Yes**
D3-6	New study M0: why the trip does not grow one-for-one, with four explanations and the test that decides each.	The same pilot result. A law's measuring setup has to be understood, not only corrected.	**Yes**
D3-7	A2 uses three kernels *(ticks of 1, 4 and 10 ms)* built by us from one source, with a bridge to the stock kernel, checks at every boot and counterbalanced sessions. P2b, P2c and P2d are added.	Three levels test a proportion, not only a direction.	No
D3-8	New T5: an audit of each tool's documentation and issue tracker, and prediction T-P8.	It turns "the industry is not dealing with it" into a measurement.	No
D3-9	Size and schedule corrected: each backend counts as its own runs, and a run takes 3.5 minutes as the pilot measured, not 1.5.	Version 2 underestimated the machine time about four-fold.	Pilot timing only

No.	Change	Reason	Prompted by pilot results?
D3-10	How the plan is frozen is specified.	Version 2 said "committed with a date and hash" without saying where. Settled by D4-13.	No

1 The three parts, and the questions they ask [D3-1]

1. **The mechanism (Mode 1).** Is there a law that predicts, from the machine's scheduler settings alone, for which trip lengths a latency measurement goes negative? Section 2 states the one we test.

2. **That it is happening.** Does the effect appear on machines, processors and kernels it was never seen on? Every run of this campaign measures it on new hardware, and A4 moves to a different processor family.

3. **That the industry is not dealing with it (Mode 2).** Does a tool's source code predict what it reports? We check whether it rounds, whether it averages over negatives, whether it drops or replaces odd values, and whether it measures the whole trip at all. We test this on every free, widely used tool we can run, not just the one we audited. And we check whether its documentation says so, and what happened when users reported it.

2 The law we are testing for Mode 1: plateau, cliff, floor

The idea in plain words. Most of the time the helper that writes down "got it" gets a core almost at once. But sometimes another task is holding the core. The scheduler lets that task finish its protected time slice, plus up to one tick, before the helper runs. The kernel's own developers state this bound: the wait stays within "slice plus tick". So a kept-waiting helper waits roughly between one slice and one slice plus a tick. That gives three parts:

- **Plateau.** If the trip is shorter than the slice, almost every long wait outlasts it, and the negative rate stays roughly flat.
- **Cliff.** Between one slice and one slice plus a tick, fewer and fewer waits outlast the trip, so the rate falls roughly in a straight line.
- **Floor.** For trips longer than one slice plus a tick, long waits almost never outlast the trip. Only a small floor remains, from ordinary short delays.

The same thing as a formula. Write s for the time slice, h for the tick, p for how often the helper is kept waiting, and c for the floor. Then

$$\text{negative rate}(D) \;\approx\; c \;+\; p \times \begin{cases} 1 & D \le s \quad \text{(plateau)} \\ \dfrac{s + h - D}{h} & s < D < s + h \quad \text{(cliff)} \\ 0 & D \ge s + h \quad \text{(floor)} \end{cases}$$

What the settings predict, and what we still have to measure.

- **Predicted from settings alone:** *where* the cliff starts, at s, and *how wide* it is, h. Both can be read from the machine, or set by hand, before any run.

- **Measured, not predicted:** *how high* the plateau is, p, which rises with load, and the floor c. Very short trips, under about 0.5 ms, also pick up short waits the formula ignores. So the tests below focus on where the cliff sits and how wide it is.

Does our old data fit? On the old 8-core machines ($s = 3$ ms, $h = 1$ ms), 13% of traced waits were over 2 ms but only 2.8% over 4 ms. That is the cliff, between 3 and 4 ms. It fits, but it is one machine, and a law has to survive new ones. What the first Azure pilot saw is in Section 5.

How it could fail. It fails if the cliff does not move when we change the slice (P1). It fails if its width does not follow the tick (P2 to P2c). And it fails if load moves the cliff rather than just raising the plateau (P3).

3 Machines and money [D3-2] [D4-10]

Pair	Machines and region	Price while running
First x86 pair	Driver Standard_D8as_v6 *(8 CPUs, AMD EPYC Genoa)* and broker D2as_v6 *(2 CPUs)*, in Sweden Central. The pilot ran here.	about EUR 0.42 an hour
Second x86 pair	The same sizes, in Sweden Central beside the first pair, in its own resource group and network. [D5-7]	about EUR 0.42 an hour
Arm pair	Driver D8ps_v6 *(8 CPUs, Azure Cobalt 100)* and broker D2ps_v6, in Sweden Central, the only region that offers these sizes to this account. For A4 and A8.	about EUR 0.32 an hour

All pairs live in one pay-as-you-go Azure subscription. Each region allows a set number of CPUs, and a stopped machine still counts against that limit. On 16 September 2026, on request, Sweden Central's limits were raised to 40 CPUs in total, 20 in the x86 family and 10 in the Arm family, so all three pairs fit in that one region. [D5-7] A stopped machine costs only its disk and its fixed address, about EUR 26 a month per pair. A pair that will not be used for more than about two weeks is deleted and later recreated, which takes about 15 minutes.

Every tool below is open source and free. Each tool's licence is recorded when it is downloaded.

4 The tools

Precision classes come from reading each tool's code. Tools marked "read first" are classified before any run, and that classification is itself a prediction (Section 10). Each tool's version is pinned by release or commit before any run.

Tool	Times	Class	Odd values (zero or negative), from source	Runs on
OpenMessaging Benchmark	send → receive; send → "got it"	Low; High	end-to-end: drops ≤ 0, uncounted. Publish: keeps all	Kafka, Valkey, RabbitMQ, NATS
Kafka EndToEndLatency	send → receive	Low (chops)	keeps all; average exact	Kafka
Kafka ProducerPerformance	send → "got it"	Low	keeps all	Kafka
Apache Pulsar perf	send → receive	Low	drops negatives, keeps zeros, uncounted	Pulsar
emqtt-bench	send → receive	Low	keeps all	EMQX or Mosquitto
wrk2	request → response	Medium	negatives never recorded; may crash	nginx

Tool	Times	Class	Odd values (zero or negative), from source	Runs on
RabbitMQ PerfTest	send → receive	High	keeps all, negatives included	RabbitMQ
NATS CLI (latency)	send → receive	High	keeps all	NATS server
Read first, then run (widely used, class not yet known)				
nats bench	to confirm	?	to confirm	NATS server
valkey-benchmark *(free Redis fork)*	request → reply	?	to confirm	Valkey
memtier_benchmark	request → reply	?	to confirm	Valkey
librdkafka rdkafka_performance	send → "got it"?	?	to confirm	Kafka
YCSB, k6, Vegeta, hey	request → reply	?	to confirm	nginx or Valkey
Kept as source readings only (they do not time a message trip)				
fio, btt, Rezolus	disk and scheduler timings	High	fio replaces negatives with 0; btt with 1 ns; Rezolus counts its discards	–

The five left unrun. [D33-1] Version 15 named ten of these to run *(D15-6)*. Version 33 runs the OpenMessaging Benchmark, Pulsar perf, emqtt-bench, nats bench and YCSB as well, as the addendum at the end sets out.

The four of October, and K1. [D35-1] [D35-6] Version 35 runs Xinfra Monitor, Strimzi Canary, Trogdor and Stream PerfTest as well, and times Kafka's end-to-end tool against an observer, as the second addendum at the end sets out.

The Yahoo Streaming Benchmark. [D36-1] Version 36 runs it as well, as the third addendum at the end sets out.

PerfKit Benchmarker and the step test. [D37-1] Version 37 runs them as well, as the fourth addendum at the end sets out.

Four tools of the second October reading. [D38-1] Version 38 runs mqtt-benchmark, Sarama's tool, SBK and kgo-repeater as well, as the fifth addendum at the end sets out.

Three tools on brokers of their own. [D39-1] Version 39 runs Artemis's perf client, RocketMQ's benchmark and BookKeeper's benchmark as well, as the sixth addendum at the end sets out.

Google's own load test. [D41-1] Version 41 runs Google's Cloud Pub/Sub load test framework as well, as the eighth addendum at the end sets out.

Two tools built from source. [D40-1] Version 40 runs Quiver and the Pravega benchmark as well, as the seventh addendum at the end sets out.

Our own program is the reference instrument everywhere. It reads nanoseconds, keeps everything, and records all three times.

5 What the pilot showed [D3-4]

The first pilot ran on 14 and 15 September on the first x86 pair, at 75% load. A pilot tests the instruments, never a prediction.

Check	Rule written before the pilot	Result
Scheduler settings	a slice set by hand is still set 60 s later, and the default comes back	Passed: 1.5 ms held; the 2.8 ms default restored
Receiver-only delay, by ping	the set delay reaches the receiver's address only	Passed: 0.5 ms measured as 0.515 ms, 2.0 ms as 2.03 ms
Load	within 3 points of 75%	Passed: 75.3% in all three cells
Never negative	arrival never before sending	Passed: none in 18 runs, about 20,000 messages
Delay seen by the harness	the trip grows by the added 2.03 ms within 0.05 ms, and "got it" moves under 0.05 ms	**Failed.** Kafka's trip grew 1.80 ms and "got it" moved −0.14 ms. Redis's trip grew 2.51 ms and "got it" moved −0.02 ms.
Go-first priority	cuts the negative rate at least 5-fold, intervals apart, at the same load	Passed: 13.5% to 0.50% *(27-fold)*, at 75.3% and 75.4% load

What the failed check means. The delay does reach only the receiver: ping says so, and "got it" barely moves. But the trip does not grow one-for-one with it. Repeat runs agree within about 0.1 ms *(Kafka's trips were 5.29–5.40 ms with the delay and 3.48–3.92 ms without; Redis's 4.56–4.72 and 2.06–2.33)*, so this is a property of the two clients, in opposite directions, and not noise. Version 2's rule was an assumption, and it is false. Trips placed by "delay = target minus baseline" would miss by up to half a millisecond, as much as the detail of the cliff we want to see. Hence C0 and M0 in Section 7. [D3-5] [D3-6] The pilot replayed the football match at ten times its speed, in bursts. On the campaigns' steady 50 messages a second, the calibrations of 16 September found both clients tracking the delay within 2% up to 8 ms, so the departure belongs to bursty traffic, and M0 studies it there. [D6-7]

The machine's own settings. Kernel 6.8.0-1064-azure, tick 1 ms *(HZ=1000)*. The default slice was 2.8 ms on the 8-CPU driver and 1.4 ms on the 2-CPU broker. That is the per-CPU constant of Linux 6.15, not the 3 and 1.5 ms the 6.8 rule gives. [D3-3]

Consistent with the law, but not a test. Without the delay, Kafka's trip *(about 3.5 ms)* sat inside the cliff the law predicts for this machine *(2.8 to 3.8 ms)*, and Redis's *(about 2.2 ms)* sat on the plateau. With 2 ms added, both trips passed the start of the floor, and negative results fell from 7–12% of messages to 1–1.5% for both clients. That is the direction the law predicts. But pilot data never tests a prediction, and two points are not a curve.

6 What earlier campaigns already answered [D4-11]

Our earlier campaigns ran on Oracle Cloud machines of the same AMD generation. Where one of them already answers a question, it is not run again, and only what a new prediction needs is run.

Earlier campaign	What it showed	What this plan still runs
E-A5, and the Azure pilot	Go-first priority on the timestamping helper cuts negative results many times over at equal load; 27-fold in the pilot.	Only its effect at the cliff: A7, at one slice.
E-A3 and E-A6	Negative results rise with load; E-A6 also tested whether it matters where on the machine the load sits, at equal utilisation.	Only whether load moves the cliff: A3, at one slice.
E-A9	Helper waits traced from the kernel's scheduler events predicted the negative rate, with ratios of prediction to measurement between 0.78 and 1.32 in ordinary runs.	Rides along in half of the A1 and A3 runs at no extra cost *(A6, P6)*, on new hardware.

Earlier campaign	What it showed	What this plan still runs
E-A10	Lengthening the true trip by padding messages lowered the negative rate *(4.1-fold for a 77-fold longer trip)* without touching the scheduler.	The slice campaigns *(A1)* place the cliff; the padding sweep is not rerun.
E-X to E-X4	Four audits of the OpenMessaging Benchmark: its end-to-end result drops values at or below zero without counting them.	No new runs of that tool; its source reading stays in the tools table.
M1	A second Kafka client for Python *(confluent-kafka)*, compared on a different delay: the producer's scheduling lag at real-time replay.	It does not answer P8, so A8 compares Python with Java at the cliff.
Earlier mechanism campaigns, rerun unchanged	–	Not run. The pilot already reproduced go-first on this machine, and every campaign here measures the effect on new hardware.

7 Experiments for Mode 1 (the mechanism)

The full fix as a tool would read it. [D43-1] Version 43 adds the block F2, in which the clients read the kernel's own timestamps on their sockets beside the captures, as the tenth addendum at the end sets out. **A full fix, tested.** [D42-1] Version 42 adds the block F1, which times each message by the kernel's own stamps where it leaves and arrives, as the ninth addendum at the end sets out.

The fix: slow only the receiver. Our sender and receiver sit on the same machine, so the broker's replies to both travel the same link. A delay added to that whole link slows the "got it" reply as much as the message. Both times then move together and the test sees nothing. That is exactly why our old delay sweep failed. So:

1. Give the receiving program its own network address on the client machine *(a second IP address, or its own network namespace)*, and make it connect from that address only.
2. On the broker machine, add the delay only to traffic going to that address. Replies to the sender's address are untouched.
3. **Checked, not assumed:** the delay's effect on the trip is measured in every session (C0 below), and the "got it" delay must stay equivalent (P5). The added delay itself is measured on its own at every step, by the broker's own capture of those pings, where the delay is added, and the measured value is used, not the setting. [D3-5] [D6-5] [D7-1]

Always the same: background load is measured, not just set. Before every run we read back the machine's actual time slice *(the kernel's base-slice setting)* and tick *(CONFIG_HZ)*, and record its core count, CPU model, kernel version and machine pair. [D4-10]

C0. Calibrating the added delay, in every session. [D3-5] *Why:* the pilot showed that adding 2.03 ms on the receiver's path lengthened the trip by 1.80 ms for Kafka and by 2.51 ms for Redis, repeatably. The trip is what the law is about, so the delay needed to reach a planned trip has to be measured, not assumed.

- **Steps:** added delays of 0, 1, 2, 4 and 8 ms, plus 16 ms in sessions whose trips reach that far and 32 ms in HZ=100 sessions. The zero step runs twice per round, to catch drift. Each step's delay is measured by the broker's own capture of that run's pings — how much longer it held the replies to the receiver than the replies to the host — and that measured value is used. [D7-1] Ping is recorded beside it and decides nothing: it reads about half a millisecond high, and prints only tenths of a millisecond above 10 ms. [D6-13] At the 32 ms step, reads come back less often

than messages arrive, so trips there do not grow one-for-one for either client. [D6-6]

- **Order and repeats:** each backend and each load the session runs, 2 rounds before the session's main runs, shuffled as in Section 11, and 2 more when the calibration is too loosely known and nothing else is wrong. [D6-2]
- **First, a staircase campaign** *(S0-1)***:** [D4-12] before any other campaign, the steps up to 16 ms run for both backends at 75% load for 4 rounds, to test with more power whether the line is straight.
- **What it gives:** for each backend, a line of run median trip against measured delay. Each main run's delay is read off that line for its target trip.
- **Checked all session long:** every main run also records its delay and its trip. At analysis the line is refitted from all of the session's runs, and its slope is compared across slices and loads.
- **Gate:** a session's main runs start only if P5(b) holds. Otherwise the session ends, and the pair starts a new one with a new C0. [D6-11]

M0. Why the trip does not grow one-for-one *(campaign S0-2)***.** [D3-6] *Why:* a law about timestamps needs its measuring setup understood, not just corrected. The departure appears when messages come in bursts *(the pilot's replay)*, not on the campaigns' steady traffic. [D6-7] The four candidate explanations, and the test that decides each, are M-H1 to M-H4 in Section 10.

- **Setups:** the football match replayed in bursts, as in the pilot; both backends, added delays of 0, 2 and 8 ms, at 75% load on the first x86 pair, 10 rounds; Redis both with one acknowledgement per message and in batches of 200, an intervention on M-H2. [D6-7]
- **Recording, in half the runs:** a packet capture on the receiver's own address and on the broker; the times at which our receiving program issues each fetch or read and at which it returns; and the receiving thread's wake-up delays *(with the tracing tool the harness already uses for the "got it" helper)*. The other half runs without recording, so the recording's own effect is measured, as in A6.
- **Afterwards:** the request-timing log stays on in later runs only if the recorded and unrecorded halves differ by less than 0.02 ms in median trip and median "got it" delay.
- **As built, version 29.** [D29-1] Runs of 130 s after a 30 s warm-up, at the speed-up that gives the pilot's ten messages a second, each held to the football plan's own count and rate in its window (D29-2). The half that records does so in the runs whose queue key hashes even, as A6's does; its packets are captured with 200 bytes of each, enough for every header M-H1, M-H2 and M-H4 read.
- **As read, version 31.** [D31-1] By `m0_read.py`. The departure is signed, $D = (b - 1) \times 8$ ms on the recorded runs, and an explanation accounts for half of it only with D's sign. M-H1: the broker's held delay plus the change in the wire's own transit, free of the clock offset. M-H2: a simulation of the receive loop from its own rules and costs (D31-2). M-H3: the receiving thread's wait after each wake. M-H4: requests held for an acknowledgement, as D31-3 defines them; not tested for Kafka.

Trip lengths are placed around the predicted cliff. For each slice s and tick h we use 8 trips: two on the plateau ($0.5s$ and $0.9s$), four across the cliff ($s + 0.2h$, $s + 0.4h$, $s + 0.6h$, $s + 0.8h$), and two beyond it ($s + 1.5h$ and $2(s + h)$). Each trip is reached with the delay the session's calibration says it needs, and the analysis uses the trip each run actually had. [D3-5]

A1. Dose–response on the slice (the main test). On the first x86 pair, set the time slice by hand to seven values: 0.75, 1.5, 2.25, 3, 3.75, 4.5 and 6 ms, [D27-3] at 75% load, less those a client cannot reach. [D8-1] *Question:* does the cliff move with the slice, one for one? On the first pair, measured on

18 September, Kafka reaches 2.25, 3, 4.5 and 6 ms and Redis also 1.5 ms; the rest are reported as out of reach with the trip that excludes them. At the A1 sessions' own zero-delay trip, 2.21 ms, Kafka did not reach 2.25, and it reaches 3, 3.75, 4.5 and 6. [D27-3]

- **Six campaigns, three per backend:** [D4-3] slices of 0.75, 3 and 4.5 ms; of 1.5, 3 and 6 ms; and of 2.25 and 3 ms; 8 trips each. A seventh, Kafka at 3 and 3.75 ms. [D27-3]
- **The anchor:** the 3 ms slice is in every campaign. Compared across campaigns, its halfway point and plateau height measure each campaign's offset, which the analysis removes before slices are compared *(Section 11)*.
- All six run on one machine pair, in an order drawn with a recorded seed.

A2. The tick, with three kernels we build ourselves. [D3-7] The tick is fixed when the kernel is compiled, so it cannot be changed on a running machine. *Question:* does the cliff's width follow the tick, in proportion?

- **One source, three builds.** The driver's own kernel *(6.8.0-1064-azure)*, from Ubuntu's source package and with the running kernel's own configuration, is built three times on the same machine with the same compiler, changing only the tick: HZ=1000, 250 and 100 *(ticks of 1, 4 and 10 ms)*. Our HZ=1000 build is the control, so the three kernels differ in nothing else. Three levels test a proportion; two could only show "wider".
- **A bridge to everything else.** Short sessions on Azure's stock kernel run the same setups as our HZ=1000 build, so the tick results connect to A1, A3 and A5, which run on the stock kernel. They should agree within 0.1 ms at the cliff's halfway point.
- **Only the driver changes.** The broker keeps its kernel throughout, because the helper that writes "got it" runs on the driver.
- **Checked at every boot, before any run** *(manipulation checks)*: the tick setting read back from the running kernel; the tick measured by counting the kernel's tick timer on a busy CPU for 10 s, within 5% of the setting; the scheduler's high-resolution slice timer *(HRTICK)* switched off in every kernel, because with it on the tick would not set the width and the test would be void; the slice read back as set; and the same tickless settings *(NO_HZ)* in all three builds.
- **Booting safely.** A new kernel is booted once while the stock kernel stays the default, so a failed boot falls back to it on the next restart. Azure's serial console is the last resort.
- **Campaigns.** [D4-7] A build campaign compiles the three kernels and boots each once with its checks. Then twelve sessions, one kernel and one backend each, over four days: each kernel runs with each backend on two different days, and across the days each kernel runs first, middle and last *(a Latin square)*. Two bridge sessions, one per backend, run on the stock kernel, and a second day of each. [D27-2] Each session starts with its own C0 and a short B0 at 75% load, because the tick may change both the delay's effect and the baseline trip. All fifteen campaigns run on one machine pair.
- **Setups:** 2 slices *(1.5 and 3 ms)* × 8 trips placed with that kernel's tick, one backend per session. Each session runs half of the block's rounds, so each kernel and backend gets the full number over its two days.

A3. Load. [D4-4] Run at 50%, 75% and 88% load, at the 3 ms slice and 6 trips, in two campaigns, one per backend. Each campaign calibrates at all three loads. Its rounds come from P3a. [D9-1] *Questions:* does load leave the cliff where it is *(P3a)*, and does it raise the plateau *(P3b)*? The Kafka campaign runs on the Arm pair too, at 24 rounds from that pair's own simulation [D24-1]; the Redis campaign does not, because that pair's plateau is as shallow as the x86 pair's [D24-2]. A third x86 Kafka campaign runs at 75 and 88% load only, the loads P3a and P3b are judged at, on the second x86

pair. [D27-4] P3b is underpowered on the Arm pair at any number of rounds this plan would spend, and is reported as such before its runs exist [D24-3].

A4. Another kind of machine. [D3-2] [D4-8] On Azure's Arm machine *(Azure Cobalt 100)*: first an instrument campaign on the Arm pair *(C0, B0 and P0)*, then four campaigns, two per backend, at slices of 1.5 and 3 ms and of 3 and 4.5 ms, 8 trips each, less those that pair's own calibration puts out of reach. [D8-1] The 3 ms slice is the anchor, as in A1. *Question:* does the cliff follow the slice on a processor the law never saw?

A5. Default slice from core count (secondary). [D3-3] [D4-6] [D11-1] On an x86 driver with 2, 4 and 8 CPUs switched on, left at their default slices, check that the kernel's default follows its rule and that the cliff follows it. The rule is computed from the per-CPU constant each machine reports, not from its version: the first Azure driver's 6.8 kernel carries the constant Linux 6.15 adopted, which gives 1.4, 2.1 and 2.8 ms where the 6.8 rule gives 1.5, 2.25 and 3 ms. Two campaigns, one per backend. Each core count is its own session, with its own C0, at 8 trips. At 2 CPUs the session's last thirteen runs are run with the got-it brake recording rather than stopping, after it had stopped twice, and P7 there is judged with and without them. [D26-1]

A6. Predict from a recording, with nothing fitted. In a random half of A1 and A3 runs, record the helper's waits in 0.1 ms steps, and predict the negative rate from that recording. Tracing slows things slightly, so traced and untraced runs are compared too. *As read, version 30:* [D30-5] a run's rate is predicted as the share of its recorded waits longer than $T - g$, its median trip less its median "got it" delay *(E-A9's prediction; the recording holds every `python3` thread, with no clock)*, and judged by P6's rule on each pair and backend, by `helper_waits.py p6`.

A7. Go-first priority. [D4-5] At the 3 ms slice and three trips *(0.9s, $s+0.5h$ and $s+1.5h$)*, turn go-first priority on for the timestamping helpers only, at the same load. Both backends, in one campaign.

A8. Client language. [D4-9] Kafka only: our Python client against Kafka's official Java client, with the "got it" note taken on the background helper or on the sending thread, at the 3 ms slice and the same three trips as A7, ordinary and go-first. One campaign on an x86 pair and one on the Arm pair. Each client has its own calibration, because the delay's effect on the trip belongs to the client.

A9. The got-it helper's own waits, message by message. [D28-1] At the 3 ms slice and the eight trips, at 75 and 88% load, both backends, one campaign on each of the three pairs, at 4 rounds *(the floor: no rule here simulates A9's questions)*. In a random half of the runs every `python3` thread's scheduling events are recorded one by one — taken off a CPU while it could still run, put to sleep, woken, put back on a CPU — against the kernel's monotonic clock, and each client names the threads that stamp its sends, acknowledgements and arrivals and reads both clocks at its start and its end. *Questions:* for each negative reading, what was the thread that stamped the acknowledgement doing between the message's arrival and that stamp (A9-1); does the recording of that one thread predict the rate, with nothing fitted (A9-2); and where do its waits end, at each load (A9-3)?

How A9 is read, fixed before its runs. [D28-1] Only runs the integrity rule passes, and only messages sent after the warm-up, as everywhere. The thread that stamps acknowledgements is the one the producer names, and the recording's clock is laid on the clients' through the two pairs of clock readings each client takes. A run whose recording reports lost events is reported and not read. *A9-1.* For each negative reading, the time from the message's arrival to its acknowledgement's stamp is divided among what that thread was doing: waiting for a CPU *(taken off one while it could still run, or woken and not yet put back on one)*, asleep, or running. The law says it was waiting for a CPU. A9-1 holds at a backend and load where, for at least 90% of the negative readings, more than half of that time was spent waiting for a CPU; the other two shares are reported beside it. *A9-2.* P6, on the one thread: a run's rate is predicted as $\sum_i \max(0,\, W_i - (T - g))/D$, where the W_i are that thread's waits for a CPU in the measured window of length D, T is the run's median trip and g its median "got it" delay *(a message sent at t reads negative if its acknowledgement, ready at about $t + g$, is still waiting at its arrival, at about $t + T$)*. It is judged by P6's rule at each setup, on the mean of its runs' predictions against the mean of their measured rates. *A9-3.* The waits' 50th, 90th and 99th percentiles and the

longest, at each load and backend, are reported beside the slice and the tick, with no rule: the law puts their end near $s + h$ at both loads.

As read, version 30. [D30-6] By `helper_waits.py a9`. The stamper of an acknowledgement is the named thread that was on a CPU at the instant of its stamp; the Redis producer's sixteen send workers all stamp, and one stamp that finds none or several of them there is unidentified. A run is read only where the stamper of at least 90% of its acknowledgements is identified, and a negative reading whose stamper is unidentified, or whose interval comes out empty on the tracer's clock, counts against A9-1. Where several threads stamp, A9-2 weighs each thread's waits by the share of the stamps it made, a prediction above one is one, and a wait belongs to the window where it begins inside it. *A9-2b.* Beside A9-2 and judged by the same rule: a run's rate predicted as the share of its acknowledgements whose own wait for a CPU — from the later of the stamper's last wake from sleep and its previous stamp — outlasted $T - g$. A9-2's sum counts the time a message arriving at a random moment would find the thread already stalled; both producers stamp from a thread that an acknowledgement's reply wakes, so the stall that decides a reading is the one its own acknowledgement began.

8 Experiments for Mode 2 (the industry)

Each runnable tool is one campaign on one machine pair, with our own program recording the same messages. [D4-1] The ten that run are Vegeta, hey, k6 and wrk2 against a fixed page this block serves; valkey-benchmark and memtier_benchmark against Valkey; Kafka's EndToEndLatency and ProducerPerformance and librdkafka's rdkafka_performance against Kafka; RabbitMQ PerfTest against RabbitMQ; and the NATS CLI against a NATS server. Every server is installed on the machine and not in a container, because a container adds a network namespace and firewall rules between tool and broker and this block measures tenths of a millisecond. [D15-6] For each tool, at least two experiments, and three where it applies:

T1. Delay staircase *(receiver-only, as in the fix above).* Add 0, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9, 1.1, 1.5 and 2.0 ms, in random order. Record the tool's average, p50 and p99, and our reference at each step. Each figure is recorded with the step its own printing can express, not one step for the tool: two of these tools print an exact average beside percentiles chopped to whole milliseconds, and that gap is what T-P1 and T-P2 predict. [D15-5] The smallest step the tool reports at all is its answer here, and it is what T2 is then given. [D15-2] *Questions:* does the tool see a 0.1 ms step? Does it round, chop or keep? Does its average track the truth?

T2. Forced negatives. Make the receiver's clock read early by the session's own median trip plus 1 ms, and plus 3 ms [D15-1] *(libfaketime for Java, C and Python tools, checked on each tool first; for Go tools, which read the clock without the C library, the driver's own clock with nothing else running on it, stepped, measured against the hypervisor's PTP clock and put back after each run [D26-3]).* The offsets are taken from the trip because a fixed one cannot reach a tool with a coarse clock: truncating toward zero, as Java and C do, puts every value between minus one step and plus one step at zero, so the offset has to clear that step before the tool meets a negative at all. [D15-2] *Questions:* does it average over the negatives, drop them, replace them with zero, crash, or count them? Does its message count still match what was sent? *How it is judged:* what each of the four behaviours would have produced — its average, its smallest value and its count — is worked out from our own record of the same messages, put through the step T1 measured for that tool, and each behaviour is ruled out by a figure it cannot account for. [D15-2] One left standing is the answer; two is a run that did not answer, and is reported as that rather than as the nearest. [D15-3] The identification is read where sends are spread evenly; sends paced in step with the tick are run and reported as T-P2's evidence. [D15-4] An offset that leaves a tool nothing to report at all is recorded as too large for it. [D15-4]

T3. Load and go-first priority. Run each tool on an idle machine and at 88% load, with and without go-first priority on the tool's own process. *Question:* how much of the tool's own number is waiting in line?

T4. What it measures. Check which trips each tool can report at all: send to receive, send to "got

it", or only request to reply. Confirm from the documentation, the source and a real output file.

T5. Documentation and issue audit *(desk work, no machines).* [D3-8] For each tool in Section 4:

1. Read its documentation for three things: which clock and resolution it times with; what it does with zero or negative values; and any warning about scheduling or clock effects.
2. Search its issue tracker and mailing list with fixed terms: "negative latency", "negative", "clock skew", "resolution", "rounding", "millisecond", "nanosecond" and "latency 0".
3. Record every relevant report with its date and outcome: fixed, closed without a fix, or still open.

The terms and the reading rules are frozen with this plan. Results are counts per tool, with every report linked. *Question:* where a tool mishandles values, does anyone say so, and did anything change when users noticed?

9 What is already known

Mode 1: no law found.

- **The kernel's rule is only a bound.** The developers state a bound, slice plus tick, not a distribution. Their 2026 conference talk measured wakeup delays and explicitly presented no model.
- **Classic waiting-time laws cover jobs, not timestamps.** Kleinrock (1967) and Coffman, Muntz and Trotter (1970) give waiting-time laws under time slices for jobs in a queue; nobody has applied them to timestamps.
- **Nearby laws cover other things.** Tsafrir (2005) models noise from kernel ticks in parallel programs. Sharma (2026) gives a condition for clock differences, not for scheduling.
- **Timestamping and tracing studies are empirical.** Villain (2012), Giraldeau and Dagenais (2015) and Bielmeier (2025) measure delays or predict them from recordings, with no formula from settings.

Mode 2: comparisons exist outside brokers. OctoPerf (2022, practitioner blog) found four HTTP tools reporting averages from 20 to 150 ms on one site. Lancet (2019) found three load testers reporting 99th percentiles of 27, 40 and 63 µs. Every broker comparison found uses one tool per study, and none checks rounding, negatives or discards.

Practitioner sources (ordinary web search).

- **Why tools disagree.** Practitioner write-ups agree that tools report different latencies mainly because they start and stop the clock in different places. OctoPerf's comparison of four HTTP tools is the clearest case. The "coordinated omission" write-ups make the same point about *when* the clock starts: Gil Tene's wrk2, Tyler Treat's 2016 message-queue benchmarks, ScyllaDB (2021) and the YCSB fix. Kroxylicious (2026) notes that Kafka's producer perf test cannot show end-to-end latency at all.
- **Millisecond clocks.** It is widely known that a millisecond clock cannot time sub-millisecond work *(Shipilev's "Nanotrusting the Nanotime", the LMAX blog, many Java write-ups)*. None of these connects it to results being silently thrown away inside a benchmark's percentiles.
- **Negative latency in practice.** Negative latencies and negative span durations are reported often: a Kafka mailing-list thread, Jaeger and OpenTelemetry users, and a 2026 Kafka monitoring guide. They are almost always blamed on clock skew; RabbitMQ PerfTest's own documentation warns about clock sync only. Tools deal with them by reporting "not a number" *(Kafka's*

consumer latency metric), clamping to zero *(Splunk's OpenTelemetry collector)*, dropping them, or breaking dashboards.

- **Scheduler delays.** Kernel developers and practitioners report latency spikes and regressions tied to the EEVDF scheduler's slices *(LWN 2026; a 2026 write-up on fixing EEVDF regressions at Meta)*. None connects those delays to wrong timestamps in latency tools, and none gives a law.
- **Bottom line.** Practitioners know the symptoms. None of the sources we found ties them to scheduling or to a predictive law, and none compares broker latency tools against each other. T5 makes this a count rather than an impression. [D3-8]

10 Predictions, frozen before running

ID	Prediction	Counts as confirmed if
P1	**The cliff follows the slice.** For each slice s the pair can reach, [D8-1] the trip at which the negative rate falls halfway from the plateau *(the level at $0.9s$)* to the floor *(the level at $2(s+h)$)* lies between s and $s+h$. Across those slices, after each campaign's offset is removed through the anchor slice, [D4-3] that halfway point rises one-for-one with s.	In all but one of the slices the pair can reach, and at least four such slices, the halfway point is inside the band. [D8-1] The straight-line slope against s lies between 0.8 and 1.2, with a 95% interval that excludes 0.5 and 1.5. A slice out of reach is reported with the zero-delay trip that excludes it.
P2	**The cliff's width follows the tick.** The drop from 90% to 10% of the plateau is about four times wider at HZ=250 than at HZ=1000.	The width ratio's 95% interval overlaps 2.5–5.5 and excludes 1.
P2b	**And about ten times wider at HZ=100.** [D3-7]	The width ratio's 95% interval overlaps 6–14 and excludes 1.
P2c	**The width is proportional to the tick.** [D3-7] Across the three kernels, the fitted cliff width against tick length is a straight line through zero with slope one.	The slope's 95% interval overlaps 0.8–1.2 and excludes 0.5; the intercept's 95% interval includes 0.
P2d	**The tick does not move the cliff's start.** [D3-7] The fitted start stays at the slice whatever the tick.	In all three kernels, at both slices, the fitted start is within 0.25 ms of the slice.
P3a	**Load leaves the cliff where it is.** [D9-1] At the 3 ms slice, [D4-4] the halfway point moves by less than 0.25 ms between the lowest and the highest load whose ordinary plateau is at least 2%. [D14-2]	The whole 95% interval of the movement lies inside ± 0.25 ms *(two one-sided tests, on the signed difference)*. A point estimate inside the band is not enough: a scattered estimate can land inside it by luck, and this is a claim that something did not happen. A load below 2% is reported as out of reach, with the plateau that excludes it, and is not judged; where fewer than two loads remain, P3a is reported as out of reach on that pair. [D14-3]
P3b	**Load raises the plateau.** [D9-1] At the same slice and between the same loads, the plateau height rises.	The increase has a 95% interval above zero. Judged on its own; it is reported as underpowered where the campaign's rounds cannot reach the power the rule asks of it.
P4	**Go-first priority removes the plateau.** At the 3 ms slice, [D4-5] it cuts the plateau rate at least 5 times wherever the ordinary plateau is at least 2%.	Point estimate ≥ 5, 95% interval above 2.

ID	Prediction	Counts as confirmed if
P5	**Measurement checks.** [D3-5] (a) Arrival minus sending is never below zero. (b) The receiver-only delay lengthens the trip in a way we can rely on. (c) It leaves the "got it" delay unchanged.	(a) 0 negatives per run. (b) In every session's calibration, the slope's 95% interval lies above 0.5, and at every step the calibration's 95% interval lies within 0.3 ms of it *(from the scatter of the runs about the line, or within the steps where the step medians are joined)*; each run's and each step median's distance from it is reported. [D6-1] (c) Equivalent within ±0.10 ms on the median and ±10% on the p99, at every delay. Otherwise the session is analysed with each run's own "got it" distribution, and the departure is reported. A median shift stops the session when it is larger than a quarter of the added delay *and* larger than the session's own got-it noise — 0.10 ms, or three times the scatter of its zero-delay runs, whichever is larger. [D8-2]
P6	**A recording predicts the rate**, with nothing fitted, within a factor of 1.5, and not systematically too low.	At least two thirds of setups within 0.67–1.5, and the median ratio at least 0.8.
P7	**Default slices follow the core-count rule**, computed from the per-CPU constant read off each machine *(on the first Azure machine: 1.4, 2.1 and 2.8 ms at 2, 4 and 8 cores)*, and the cliff follows them. [D3-3]	Read-back values match; P1's band holds at each.
P8	**Language (Kafka).** [D4-9] Java shows the same cliff and priority effect as Python: the rate at $s+0.5h$ lies between its levels at $0.9s$ and $s + 1.5h$, and go-first cuts the plateau at least 5 times. Python's rate is at most 1.5 times Java's at matched setups.	All hold at the 95% level, on each machine pair.
P9	**New hardware.** [D4-14] On the Arm pair the cliff follows the slice: at each slice that pair can reach [D8-1] the halfway point lies between s and $s + h$, and it rises with the slice, after the anchor correction.	The halfway point is inside the band at every slice the pair can reach, and at least two such slices, and its straight-line slope against s lies between 0.8 and 1.2.

Why the trip does not grow one-for-one (M0). [D3-6] Each explanation is kept or dropped by its test, and more than one may hold. "The departure" is how far the measured slope is from one, times the added delay.

M-H1	**The network.** The delay that reaches the receiver's network card differs from the delay set.	Kept if the arrival shift in the packet capture differs from the set delay by more than 0.05 ms.
M-H2	**The receive loop.** A longer delay changes whether a new message finds a request already waiting at the broker, or has to wait for the next one.	Kept if a replay of each run's message times against its logged request cycle reproduces the measured slope within 0.05.
M-H3	**The wake-up.** With a delay, the receiving thread has gone to sleep when the reply arrives, and must be woken and scheduled.	Kept if the change in the thread's median wake-up delay between 0 and 8 ms accounts for at least half of the departure.

ID	Prediction	Counts as confirmed if
M-H4	**The TCP acknowledgement** *(expected for Redis)*. Delaying the broker's replies also delays its acknowledgements, which can hold the receiver's next request back.	Kept if requests held for an acknowledgement appear in the capture and account for at least half of the departure.
T-P1	**Low, chops after measuring** (Kafka EndToEndLatency). Percentiles equal the true value rounded down to whole ms: 0 for every step below 1 ms. The average is within 2% of our reference.	All staircase steps match.
T-P2	**Low, reads a ms clock twice, keeps all** (ProducerPerformance, emqtt-bench). With send times spread evenly, the average is within 0.05 ms of the truth, and the median flips from 0 to 1 at a true 0.5 ms. With sends paced in step with the tick, the average can be off by up to 0.5 ms.	Both patterns seen at the predicted steps.
T-P3	**Low, drops** ≤ 0 (OpenMessaging end-to-end). For trips under 1 ms, the median prints 1.0 ms at every step, and retention is about the trip divided by 1 ms.	Median constant below 1 ms; retention within 5 points of prediction.
T-P4	**Low, drops negatives, keeps zeros** (Pulsar perf). The median is 0 below a true 0.5 ms. Under a forced offset, the average rises by exactly what dropping the negatives removes.	Average shift within 10% of the computed value.
T-P5	**Medium** (wrk2). It tracks 0.1 ms steps within 0.05 ms at the median. Forced negatives are not recorded, or the run aborts.	Both behaviours as predicted.
T-P6	**High, keeps all** (RabbitMQ PerfTest, NATS CLI). They track 0.1 ms steps within 0.02 ms. A forced offset shifts every value by the offset, and negatives show in the minimum and pull the average down.	Shift within 0.02 ms; negatives present.
T-P7	**Every "read first" tool** behaves as its class predicts, in T1 and T2.	At least 90% of the predicted behaviours observed.
K1-a	**The trace is the tool's own order of events.** [D35-7] In every K1 run, every record's traced elapsed time covers its own wait for the acknowledgment.	Every record of every run; a run where one does not is not judged.
K1-b	**The tool cannot see a record that was readable first, and there are such records.** At 88% load with Kafka's end-to-end tool at normal priority, a share of records is readable to an observer before the tool has their acknowledgment.	A share above zero in all six rounds.
K1-c	**Load raises that share.** At normal priority, the share at 88% load exceeds the share idle.	In all six rounds.
K1-d	**Go-first takes it back.** At 88% load, the share with the tool at real-time priority is below the share at normal priority.	In all six rounds.
T-P8	**The documentation stays silent.** [D3-8] Where the source rounds, drops, clamps or replaces values *(T-P1 to T-P5, fio, btt)*, the documentation does not say so.	Documented in at most a third of those tools.

11 Statistics: how many runs, and in what order

How many rounds, from each campaign's own test. [D4-2] The number of rounds is not fixed in advance; the rule that sets it is. Before a campaign's main runs, and after its machine pair's spread pilot *(P0)*, the campaign is simulated 1,000 times on made-up data:

- **The made-up data follow the law exactly.** They use the plateau height and floor that the pair's P0 and B0 measured at the campaign's load; the campaign's own slices, trips, loads and tick; the planned number of messages per run; and run-to-run variation, on the logarithm of the rate, equal to the spread P0 measured on that pair for that backend — one figure per backend, never the two pooled, because pooling assumes a shared variance that the pilots refute. [D10-1]

Shifts between campaigns and between sessions are added, with a standard deviation of 0.1 ms in trip and 20% in plateau height.

- **Each simulated campaign goes through the real analysis** and the same "counts as confirmed if" rule as the real data (Section 10).
- **Rounds** are the smallest number, at least 4, for which the rule confirms the prediction in at least 80% of the simulations, and in at most 5% of simulations where the prediction is false in the way its falsifier states: a cliff that stays put as the slice changes *(P1, P9)*; a width that does not change with the tick *(P2 to P2c)*; a halfway point that moves 0.5 ms between 50 and 88% load *(P3a)*, judged over the loads that remain [D14-2]; no go-first effect *(P4)*; a cliff that stays put as CPUs are switched off *(P7)*; Python's rate twice Java's *(P8)*.
- **Blocks of several campaigns** *(A1, A2, A4)* are simulated as a whole, and every campaign of a block runs the same number of rounds.
- **Ceiling:** 40 rounds. If 40 rounds cannot reach that power, the campaign runs at 40 and its prediction is reported as underpowered, as written in the campaign's log before it runs.
- **Fixed numbers where no prediction is tested:** C0 runs 2 rounds per session and 4 in S0-1; B0 runs 3; P0 runs 5; M0 runs 10. Tool campaigns run 4 rounds: their predictions are patterns *(rounding, dropping, a median stuck at 1 ms)* that one round already shows, and the repeats measure how stable each pattern is.
- **Why not a fixed floor.** Version 3 required at least 15 runs per setup, from old setups whose run-to-run spread was about as large as the rate. The pilot's repeat runs varied far less *(a standard deviation of 0.14 to 0.23 on the log rate)*, and a simulation of the actual decision rule measures power directly. The simulation code is tested on made-up data before its first use, and its seed is recorded.

Spread pilot (P0): 5 rounds of 16 setups at 75% load on every machine pair, before that pair's main campaigns. [D4-10] They are used only to estimate run-to-run spread, never to test a prediction. The instrument pilot in Section 5 is a different thing.

Machine pairs. [D4-10]

1. **One pair per test.** Every prediction is tested on runs from one pair. A1, A3 and A7 run on the first x86 pair; A5 and all fifteen A2 campaigns on one x86 pair; A4 on the Arm pair; each tool on one pair. A8 runs once on an x86 pair and once on the Arm pair, and P8 is judged on each.
2. **Every pair opens with its own B0 and P0** and reads back its settings, so the pairs can be described against each other and the repeats rule uses that pair's own spread.
3. **Every run records its pair.** No estimate pools runs from different pairs.
4. **At the same time:** campaigns on different pairs may run at once; campaigns on one pair never overlap.

Random order, so a passing server mood cannot fake an effect:

1. **Rounds.** List every setup of the campaign. One round contains every setup once. Repeat for as many rounds as the rule gives.
2. **Shuffle.** Within each round, shuffle the order with a random generator whose seed is written in the run log.
3. **No back-to-back repeats.** If the last setup of one round equals the first of the next, reshuffle.

4. **Spread over time.** [D4-1] The campaigns of one block run in an order drawn with a recorded seed, over at least two days and at different times of day *(morning, afternoon, night)*.

5. **Settings that need a reboot, a new machine or another pair.** Tick, kernel, core count, machine type and machine pair cannot be shuffled run by run. They are *sessions*: the order of sessions is randomized, and each session type is repeated on at least 2 different days. Kernel sessions are counterbalanced across days. [D3-7] The slice *can* be changed without a reboot, so it is shuffled run by run inside a session like everything else. The analysis treats session as its own source of variation *(split-plot design)*.

6. **Failed runs.** [D5-2] If a run fails mechanically, or a condition it was meant to have did not take *(Section 13)*, it is logged with the reasons and re-queued at a random later position in the same session, 3 attempts in all. A run whose conditions held but that gives a surprising number is a result, never a re-run.

Integrity checks, the same for every arm. [D5-2] Before a run counts, it must pass all of these. Section 13 says when each is checked and what follows a failure. Runs that fail are counted, reported and never silently dropped.

- Arrival minus sending is never negative.
- The session's delay calibration passed P5(b), and the run shows no gross "got it" departure from its own campaign's earlier runs of the same setup. [D17-1] Its distance from the session's calibration is recorded beside it and brakes nothing *(P5(c) is judged in the calibration itself)*. [D17-3]
- At least 99% of the planned messages are sent after the warm-up, and at least 99% of those arrive. [D5-2]
- The achieved send rate is within 2% of target.
- Measured load is within 3 points of target, from at least 10 samples taken while messages are sent.
- The slice and tick read back as intended, before and after the run.
- The added delay, from the broker's own capture of the run's pings, is within 0.05 ms of the delay set, and that capture exists. What ping made of it is recorded. [D5-2] [D6-5] [D7-1]
- Before the run, the machine is as its session set it up: the receiver's address only in its namespace, the broker answering, and no package manager running. [D6-8]
- The clients ran with the campaign's settings, as their own logs state: Kafka's producer with up to 64 requests in flight, Redis's consumer acknowledging in batches of 200. [D6-6]
- The clock's offset is logged before and after the run.
- The first 30 seconds are discarded.

How the cliff is measured, fixed before the data. For each slice, the curve of negative rate against trip is summarized two ways:

- **Fitted shape:** fit the plateau, straight fall and floor to the runs, and read off where the fall starts and how wide it is.
- **Free shape:** fit a smooth curve that is only required to go down, and read off the halfway point and the 90%-to-10% width.

The law passes only if both agree with the prediction. Intervals come from resampling whole rounds *(bootstrap)*, each campaign's from its own where a block has several. [D30-4] **Each run sits at the trip**

it actually had, its own measured median trip, not the trip it was aimed at. [D3-5] As a secondary analysis, fixed now, every message is also placed at its own trip within its session, which traces the cliff inside runs as well as across them.

Removing campaign offsets through the anchor. [D4-3] In A1 and A4, a campaign's offset is its anchor slice's halfway point minus the mean of the anchor's halfway points over the block's campaigns; for plateau height, the same on the log scale. Each campaign's other halfway points and plateau heights are corrected by its offset before P1 or P9 is judged. Both predictions are also reported without the correction.

Analysis, fixed before the data.

- **Unit of analysis:** the run.
- **Comparisons:** ratios on the log scale, allowing for session and round.
- **Pairs and campaigns:** [D4-10] machine pair, campaign and session are sources of variation; nothing pools runs across pairs.
- **Several predictions in a family:** tested together with Holm's correction.
- **Calibration** *(C0)*: [D3-5] for each backend and session, a straight line of run median trip against measured added delay, with a test of the line against the per-step medians *(lack of fit)*. Intervals come from resampling whole rounds. If the line fails the test, the per-step medians are joined by straight segments instead, and the shape is reported.
- **"Got it" equivalence** *(P5(c))*: [D3-5] two one-sided tests at 90% at each delay step against zero delay, with margins of ±0.10 ms on the median and ±10% on the p99.
- **Code first:** the analysis code is written and tested on made-up data before the first real run it analyses.
- **Frozen predictions:** this plan is frozen as Section 15 describes, before the first campaign.
- **Every run tracked:** failures included, as in all our earlier campaigns, each with its verdict, its reasons and its fingerprints. [D5-4]

12 The campaigns [D4-1]

Stage	Campaigns	Question	Runs	Hours each	Hours
0 Instrument	4: S0-1 delay staircase; S0-2 M0; S0-3 B0 and P0; S0-4 B0 and P0 on a second x86 pair, if one is used	how the delay moves the trip, why, and how much runs vary	414	3.5–7.4	25.2
1 Slice *(A1)*	6: per backend, slices of 0.75, 3 and 4.5 ms; 1.5, 3 and 6 ms; 2.25 and 3 ms	does the cliff follow the slice?	592	4.7–6.7	36.0
2 Load *(A3)*	2: one per backend	does load leave the cliff where it is *(P3a)*, and raise the plateau *(P3b)*?	576 [D9-3]	6.5	33.6
3 Go-first *(A7)*	1	does priority remove the plateau at the cliff?	72	4.5	4.5
4 Core count *(A5)*	2: one per backend	does the kernel's default slice set the cliff?	264	8.4	16.9
5 Tick *(A2)*	15: the build, 12 sessions, 2 bridge sessions	does the cliff's width follow the tick?	682	3.5–5.2	55.5
6 Arm *(A4)*	5: the Arm instrument campaign, then 2 per backend	does it hold on another processor?	430	4.7–7.4	26.3
7 Client *(A8)*	2: an x86 pair and the Arm pair	is it the Python client?	288 [D13-1]	8.6	17.3
8 Tools *(T1–T4)*	10: one per runnable tool	what do the tools report?	680	4.7	47.2
T5 audit	desk work	does anyone say so?	–	–	–
Total	**47**		**3,638** [D13-1]		**242.0**

Runs and hours are at the floor of 4 rounds, except where a campaign's own rounds have been simulated and the row carries the change that set them: A3 at 16 rounds *(D9-3)* and A8 at 6 *(D13-1)*. A run takes 3.5 minutes, as the pilot measured *(warm-up, at least 5,000 messages, settling and checks)*; each campaign adds 0.25 hours of setup, and each reboot 0.5 hours. At 6 rounds the total is 4,826 runs and 312 hours. M0's second Redis arm adds 30 runs *(D6-7)*; a calibration's second stage, where one is needed, adds 24 runs to its session *(D6-2)*.

Which pair runs what. With one x86 pair, every x86 campaign runs on it and S0-4 is not needed. With two x86 pairs, the first runs S0-1 to S0-3, A1, A3, A7, A8's x86 campaign and five tool campaigns; the second runs S0-4, A5, all of A2, and the other five tool campaigns. The Arm pair runs A4, A8's Arm campaign and A3's Kafka campaign [D24-1] — not A3's Redis campaign, which that pair's plateau cannot carry [D24-2]. Version 27 gives the first x86 pair A1's Kafka campaign at 3 and 3.75 ms and the Kafka bridge's second day, and the second x86 pair the second day of A2's Redis session at HZ=1000, the Redis bridge's second day and the third x86 A3 campaign. [D27-1] Version 28 gives each of the three pairs A9's campaign. [D28-1] Version 29 gives the first x86 pair M0, as stage 0 always planned. [D29-1]

How long. At about 20 hours a day: one x86 pair and then the Arm pair, about 11.5 days; two x86 pairs at once and then the Arm pair, about 7 days; all three pairs at once, about 5 days. [D5-7] Each new pair's shakedown adds a few hours to these. The rules of Sections 11 and 13 hold in every case. [D5-1]

Money. About EUR 97 of machine time at list prices with all three pairs in Sweden Central *(EUR 126 at 6 rounds)*, plus about EUR 26 a month per pair for disks and fixed addresses while the pair exists. [D5-7]

Order of work:

1. **Now, with the machines stopped.** Freeze this plan, together with the code that judges each run, stops a campaign, watches the pairs and copies the data, which is written and tested before the freeze. [D5-2] Write, and test on made-up data, the code for M0's recording, the tick checks, the simulated-power rule and the law analysis. Read the "read first" tools and freeze their classes. Do the T5 audit. Prepare the kernel builds and the Java client.

2. **Stage 0.** On the first x86 pair: S0-1, then S0-3, then S0-2 once M0's recording is tested. On a second x86 pair, if one is used: its shakedown, then S0-4 and A2's build. The Arm pair's shakedown comes before its instrument campaign. [D5-1] Stage 0 starts again from the beginning under version 6. After any change to the code, the first x86 pair runs alone until its shakedown and first calibration pass, and then the others start. [D6-12]

3. **Rounds.** Each campaign's rounds come from Section 11 and are written in its log before it runs.

4. **The campaigns,** in the order of the table above on each pair, with each block's campaigns in their seeded order. Each campaign's runs are copied and checked before the next campaign starts on its pair. [D5-4]

5. **Analysis and report.** Every prediction reported as confirmed or not, with every run listed.

If money or time runs short: the slice and tick campaigns first, since they are the law; then A4 and the tools; then the rest.

13 How the campaigns are run, checked and stopped [D5-1]

Campaigns run unattended, often at night, on up to three machine pairs at once. So the rules for running them are fixed here, before any run, and a program applies them, not a person looking at results.

Lanes. [D5-6] Each machine pair is a *lane (a driver and its broker, with their own queues and logs)*. A campaign runs on one lane from its first run to its last, and the campaigns of one lane never overlap. Two lanes share no machine, broker, network link or disk, so a run on one cannot disturb a run on the other. Every lane runs the same version of the code, and every run records its lane, its driver and that version *(the commit)* next to its files.

A new pair's shakedown. [D5-1] Before a pair's first campaign, the pilot of Section 5 runs on it. Its scheduler-settings, receiver-only-delay, load and never-negative checks must pass, and a pair that fails one runs nothing until the cause is found. The receiver-only delay is checked by the broker's own capture of the pilot's pings, within 0.05 ms; [D6-3] with no delay, the two paths are measured by ping, TCP and UDP and recorded, and both machines' network and timing facts are kept with them. [D7-2] [D7-3] A pair that passed its shakedown repeats this network part in every later session, because a start after a stop can put a machine on another physical host. [D6-4] The pilot's harness-delay check is replaced by the pair's own C0, as on the first pair *(D3-5)*. Its go-first result is recorded but is not a gate, because A8 tests go-first as a prediction.

Every run, as it ends. [D5-2] A program on the driver checks each run the moment its messages stop, writes every check next to the run with the value found and the limit it was held to, and gives one of three verdicts:

Verdict	When	What follows
Counts	Every integrity check of Section 11 passes.	The run enters the analysis.

Verdict	When	What follows
Repeated	A condition the run was meant to have did not take: fewer than 99% of the planned messages sent after the warm-up, or fewer than 99% of those arrived; the send rate more than 2% from target; the load more than 3 points from target, or fewer than 10 load samples while messages were sent; the slice or tick different after the run from what was set; the added delay, from the broker's capture, more than 0.05 ms from the delay set, or that capture missing; [D7-1] the machine not as its session set it up before the run; [D6-8] a client not set as every campaign sets it; [D6-6] the clock's offset not logged before and after; or a step that failed, or files that cannot be read.	The run keeps its files and its line in the ledger, marked failed with its reasons, and a copy is queued at a random later place in the campaign, 3 attempts in all. A run whose third attempt fails is abandoned and reported.
Stops the campaign	The instrument is in doubt: a message arrived before it was sent *(P5(a))*, or the run's "got it" median moved from the median of its own campaign's earlier runs of the same setup by more than the allowance *(a quarter of the delay it added, three times the scatter pooled over this campaign's setups, or 0.10 ms, whichever is largest)*. [D17-1] [D18-1] A run at a setup with fewer than two earlier counted runs is recorded rather than braked, and its distance from the session's calibration is recorded for every run and brakes none. [D17-3]	The run is recorded as failed, and nothing more runs on that pair until the cause is found and recorded with the campaign. If the cause is in the code every pair runs, the other pairs stop too.

Nothing a run measured makes it a repeat: a run whose conditions held is a result, whatever its number. The "got it" comparison needs the session's calibration, so it applies to every run after the session's C0; within C0 itself, P5(c) is judged when the calibration is fitted.

When attempts keep failing. [D5-3] After every attempt the ledger is read again. The campaign stops itself when the last 3 attempts all failed, or when more than 20% of the last 20 failed, once at least 10 have finished. A machine that has gone wrong then costs minutes, not a night.

Void sessions. [D6-10] A session broken by a fault of the machines or of our code, or stopped to fix one, before its calibration's gate is void. Nothing in it is deleted: its files and ledger are copied and fingerprinted like any campaign's, a note beside them gives the cause, and the report lists it. It is never analysed. A session whose calibration fails its gate ends too, and the pair starts a new one with a new C0; no session is held open while a rule changes. [D6-11]

Watching every pair. [D5-5] A watch on a computer outside the testbed reads every pair every 5 minutes. It changes nothing on the machines, except that it deallocates a pair whose machines have been idle for 20 minutes, so that the pair bills only for its disks. It raises:

- **alerts,** which need a person now: a campaign that stopped itself, or a run judged to stop one; a message that arrived before it was sent; a campaign with no progress for 20 minutes; new failed runs; a broker that is not running; a disk 80% full, or less than a tenth of memory free; a machine that does not answer; one queue running on two pairs;
- **warnings,** worth a look: a repeated run; CPU time taken by other tenants above 5% *(steal)*; a clock more than 1 ms off; load far below its setting; pairs running different versions of the code;
- **notices:** a finished campaign, whose runs are to be copied before the next campaign starts on that pair.

Every look is kept in a dated log.

Keeping the data. [D5-4] A campaign's runs exist only on its driver until they are copied. When a campaign finishes, and before the next starts on that pair, its ledger and every file of every attempt, done or failed, are fingerprinted on the driver *(SHA-256: a 64-character code that changes if any byte of the file changes)*, packed, copied, and checked file by file against their fingerprints. The fingerprints are kept with the data, with a record of the machine, the code version and the time of the copy. A copy that fails its check is made again from the driver, where nothing is deleted. A pair is deleted only after all its campaigns are copied and checked. What a driver measured outside any campaign *(a pilot, a void session, a probe of the instrument, a log)* is copied and checked the same way, and before a pair is deleted its whole folder of runs is copied once more. Every run also keeps the TCP counters of the driver, the receiver's namespace and the broker, and the broker's own log for the run. [D6-10] Every run writes into a folder of its own, named after its campaign and its place in it, and a campaign that finds that folder already there stops. [D6-9]

14 What could go wrong, and what it would mean

- **P1 fails** (the cliff does not follow the slice). The "law" is only a description of our old machines. That is still worth reporting, and it would redirect the paper.
- **P2 to P2c fail** (the width does not follow the tick). The wait is not "slice plus up to one tick". The plateau–cliff shape may survive with a different width rule. If only P2c fails, the width grows with the tick but not in proportion, and something else sets it at long ticks.
- **P2d fails** (the tick moves the cliff's start). The slice is enforced only on tick boundaries, and the law's start becomes the slice rounded up to a tick. [D3-7]
- **P3 fails** (load moves the cliff). Waiting is more than one protected slice under heavy load, for example several tasks queued ahead. That would point to the classic time-sharing queueing laws, which do predict this.
- **P8 fails on Java.** The effect is tied to Python's lock or our client. That narrows Mode 1 to Python-style clients.
- **The slice setting does not stick**, because the kernel resets it when cores change. Read it back before every run and discard runs where it drifted.
- **The delay's effect is not a straight line** (C0). [D3-5] The per-step medians are joined instead, and the shape is reported. If a longer delay can give a shorter trip, the receiver-only delay cannot place trips, and that machine's main runs do not start.
- **No explanation survives M0.** [D3-6] The slope is reported as an open property of the clients. The law's tests still stand, because each run is analysed at the trip it actually had.
- **HRTICK is on in a kernel.** [D3-7] It is switched off, or the kernel rebuilt, before any tick run. A tick test with it on would be void.
- **A new kernel does not boot.** [D3-7] The machine falls back to the stock kernel, the failure is logged, and no run is lost.
- **The two backends disagree.** Both are reported, and the law's claim is narrowed to where it holds.
- **The machine pairs differ** [D4-10] in baseline trip or run-to-run spread. Each pair's tests stand on their own, with that pair's own repeats, and the difference is reported.
- **A simulation asks for more than 40 rounds.** [D4-2] The campaign runs at 40, and its prediction is reported as underpowered, as its log says before it runs.

- **The anchor moves between campaigns** [D4-3] by more than 0.25 ms at its halfway point. The movement is reported, and P1 is given both with and without the anchor correction.
- **A campaign stops itself.** [D5-3] Nothing more runs on that pair until the cause is found. Runs finished before the stop stand if the cause did not touch them; if it did, they are repeated, and both the cause and the repeats are reported.
- **A pair is lost in the middle of a block** [D5-4] *(a machine fails, or its disk is lost)*. The campaigns already copied are safe. Because no test pools runs from two pairs, the interrupted block runs again in full on a rebuilt pair, after its shakedown, B0 and P0, and the interrupted runs are reported.
- **A pair's runs vary too much for its calibration.** [D6-2] Two more rounds run; if the calibration is still too loosely known, the session ends, is reported with its spread, and the pair starts a new one.
- **The two paths differ with no delay.** [D7-2] It is reported with the session, in all three measurements, and the pair runs: what its runs vary by is what its rounds are set from. On 17 September a gate on ping's figure turned away a pair whose messages were unaffected.
- **A run has no capture of its delay.** [D7-1] The run is repeated. If captures keep failing, the campaign stops itself on the run of failures *(D5-3)*, because a treatment nobody measured is not a treatment we can report.
- **A client does not take its setting.** [D6-6] Its runs are repeated, and after three failed attempts in a row the campaign stops itself *(D5-3)*; the cause is found before it starts again.
- **Kafka runs keep pausing** [D6-10] for fractions of a second. The pauses are reported with the TCP counters and the broker's log of each run; the analysis uses medians and per-message placement, which a few paused messages barely move.
- **A copy does not match its fingerprints.** [D5-4] It is made again from the driver. A file that still does not match is reported, and its run is listed as lost, never silently dropped.

15 How this plan is frozen [D4-13]

A pre-registration is only worth something if a reader can check that it was written before the data. This plan is frozen by committing it in full, its source and this PDF with their SHA-256 fingerprints, to the `freezes` folder of the project's public repository, before the first Stage 0 run. The code that applies Section 13 is in the same repository, and every run records the version of it that ran. [D5-6] A later change is a new, numbered freeze whose first section lists what changed and why, never an edit of this one. This version is the third freeze *(freeze 03)*. [D6-12] [D7-4] Results are reported against the freeze in force when they ran, including every prediction that failed.

16 Addendum: how each tool keeps time, and the runs that decide it [D23-1]

Written on 22 September 2026, after the tools block ran for the first time. It adds no prediction and removes none; it says what T2 can and cannot ask of a tool, and what is run to find out which.

T2 needs two clocks. It moves the clock a tool reads back by an offset and asks what the tool does with the values that come out below zero *(Section "The tools")*. A tool that writes both of its timestamps itself has both of them moved by the same amount: the difference it reports is untouched, and no value below zero arises however large the offset. The question has an answer only where the subtraction spans two clocks — which is the same condition under which the artifact this paper is about arises at all.

Of the eleven tools, as each is invoked *(`cloud/azure/tools_run.sh`)*:

Tool	What it times	Clocks in the subtraction
vegeta, hey, k6, wrk2	an HTTP round trip, from the client	one, the client's
valkey-benchmark, memtier_benchmark	a Redis round trip, from the client	one, the client's
kafka-producer-perf	send to acknowledgement, at the producer	one, the producer's
kafka-end-to-end	produce and consume, in one process	one
rabbitmq-perftest	produce and consume, in one process	one
nats-latency	publish and receive, in one process	one
rdkafka_performance	the producer stamps the payload, the consumer subtracts	**two**: only the consumer's clock is moved

This is not a fault in the tools and not one in T2. It is a result: **a benchmark that reports a round trip against its own clock cannot exhibit this artifact, and cannot measure the one-way latency a streaming system cares about either.** What follows is how that is established by measurement rather than by reading the invocations.

The runs a T2 session makes

Three runs of the tool, in this order, each of the length every other tools run uses:

1. **At no offset.** The control. Its figures are what the other two are compared against. It is taken in the T2 session, minutes before the others and on the same machine in the same state, rather than borrowed from T1's zero-delay step, which was taken under a different arrangement of the delay and possibly hours earlier.
2. **At the first offset** *(the session's own median true trip plus one millisecond)* [**D15-1**].
3. **At the second offset** *(the same plus three)*.

Beside each of the two offset runs, the offset *achieved at the clock* is recorded: the median of twelve readings taken under that spelling, less the median of twelve taken at no offset, which cancels the cost of starting the tool and the clock-reading program. Asking for 1.812 ms recovers 1.806 on the first x86 pair. A run whose offset cannot be confirmed is not made at all, because a run with no offset looks exactly like a tool that handles negatives perfectly.

What the three runs decide

- **The clock moved and the tool's figures did not** — no figure differs from the control by more than the tool's own step *(what T1 measured it can report at all)*. **The tool times against one clock.** No value below zero can arise in it, its rule for such values is not reachable by this method, and the artifact cannot appear in what it reports. Recorded with the shift that was confirmed and the figures that did not move.
- **The figures moved by about the offset.** The subtraction spans two clocks and the offset reached it. The four behaviours *(keeps every value; drops the negatives; drops zero and below; replaces them with zero)* are held against the figures as before, and the verdict is whichever survives.
- **Neither.** Undecided, as now, with the confirmed shift and the figures printed beside each other so a reader can see what did not add up.

The audit *(T5)* read these tools' sources and predicted how each behaves. These runs measure it from outside. Where the two disagree, the prediction is reported as failed, not corrected.

The runs rdkafka_performance needs first

It is the only tool arranged across two clocks, and it measured nothing: its consumer read none of its producer's messages, over a whole staircase and again with the order of the two reversed. A shakedown therefore runs before anything of it is measured — a short producer and consumer, tens of messages rather than thousands, with the topic's partition count, the consumer's mode and its start offset written down, repeated until the consumer reads what the producer sent. Nothing of it is measured until one does.

Where no invocation reads them, that is T4's answer for this tool — what it can report at all — recorded with every invocation tried, and reported as *this harness failing to drive the tool* rather than as a limit of the tool. A staircase of ten steps that each measured nothing looks exactly like a staircase of ten steps that each measured zero, and the difference is the whole of what T1 asks.

What is deliberately not done

A tool that offers to run its two ends as separate processes could be given two clocks by wrapping only one of them, and PerfTest is one such. The tools are run as their own documentation runs them. Where a tool offers that split, it is a second configuration reported beside the first and never in place of it: contorting a tool into a shape it does not normally take would answer a question about our arrangement rather than about the tool, and what this block reports is what a reader of that tool's own manual would get.

17 Addendum: T2's judgement, and the corrections of 24 September [D25-2]

Written on 24 September 2026. The first part fixes how every T2 reading is judged, before any is judged under it. The second lists what was found wrong that day and what was done about it.

How a T2 reading is judged

For each tool, for each figure judged — the average and the minimum, where the tool prints them — and for each offset run of a T2 session:

1. **The change** is the offset run's figure less the same figure in the session's control run *(D23-1, D25-1)*.

2. **The noise** of the figure is the residual standard deviation of that figure about a least-squares line fitted against the added delay over the tool's T1 staircase: ten runs, eight degrees of freedom. A figure the staircase does not give at three steps or more has no noise, and is not judged.

3. **The allowance** for a change is three times the noise times $\sqrt{2}$ — the scatter of a difference between two runs — and never less than two of the steps the figure is printed in. With the noise estimated on eight degrees of freedom, a band of three holds about 98% of changes that are noise alone.

4. **Five hypotheses** each predict a change. The four behaviours predict theirs from our reference trips at the offset *measured* at the clock, not the one asked for. The fifth, that the tool times against one clock, predicts no change at all.

5. A hypothesis **survives** when every judged figure's change lies within the allowance of what it predicts. A count short of what the tool was asked to send rules out the behaviours that keep every value at the offset, and no others: how many a dropping behaviour keeps depends on the tool's own trips, which the reference does not record. [D26-2]

6. **The verdict** is the one hypothesis left standing. Where none is, or several are, the run is undecided, and the change, the allowance and every prediction are printed beside it.

What this cannot do is also fixed now. A tool whose own noise is comparable to the offset cannot be decided by it: rdkafka_performance's noise is about 1.1 ms on its average, so its allowance is about 4.7 ms, wider than its first offset. That is a limit of this instrument on that tool and is reported as one. Every reading already taken is judged again under this rule, and its verdict under D23-1 as it was implemented is reported beside.

The corrections of 24 September

Each with what found it. Runs set aside are kept beside a note saying why; none is deleted.

- **k6** made no request in any of its fifteen runs *(D25-3)*. Found by T1's slope, 0.000. Run again.
- **nats-latency** met no NATS server in fifteen runs, and **rabbitmq-perftest** met a jar that would not start in fifteen runs and then, rebuilt, a broker that refused its user through a staircase of ten *(D25-4)*. Found by a check that asked what each tool had reported. Run again.
- **The T2 judge** compared against the wrong run and allowed for no noise *(D25-1, D25-2)*. Found by laying each tool's movement beside its step and its shift.
- **stress-ng's correction** stopped the first stress-ng with a signal it can decline and waited for it; about one run in nine took 62.7 minutes instead of 2.8. Found by the gaps in a campaign log. The runs stand: each ran under the corrected duty. Corrected before this version and not recorded in one.
- **rdkafka_performance** was never attached to a partition, started from the beginning of the topic rather than the end, and was not paced. Found by its shakedown *(D23-2)*. Corrected before this version and not recorded in one.
- **The watch and the queue** counted a pair running the tools block as idle, and the watch deallocated the first x86 pair 32 minutes into a staircase. Found when the pair disappeared. Both count the tools block now.
- **A pair now stops itself** when its work ends, through the driver's own identity, without the PC that watched it; a waiting pattern that matches its own spelling is refused, because it would count its own command line as work for ever.
- **The quality report** failed mid-campaign on a run with no verdict yet, and on a directory named like a CSV beside the runs. Found by running it on the drivers.
- **The first x86 pair's driver** carried idle RabbitMQ, nginx and NATS servers from 15:01 UTC on 24 September until they were stopped, left there by the misdirected setup of D25-4. No tool talked to them; they are recorded because they were there.

18 Addendum: the five tools version 15 left unrun, how each is run, and how each keeps time [D33-4] [D33-5]

Written on 9 October 2026, before any run of these tools under this plan. It adds no prediction and removes none: the predictions are the ones these tools have carried since versions 3 to 12 *(Section "Predictions")*. It says how each is run, which of its figures is read, how it keeps time and so what T2 is expected to find, and what it cannot be asked.

How each is run, and how each keeps time

As each is invoked *(`cloud/azure/tools_run.sh`)*. "Wrapped" is the process that T1's namespace, T3's go-first and T2's moved clock reach.

Tool	How it is run	Clocks in its subtraction	What its source says it does with a value at or below zero
OpenMessaging Benchmark	Its distributed mode: a worker that produces, a worker that consumes, and the coordinator that drives them. All three in the namespace in T1, T3 and T4, and on the machine in T2; the consumer worker wrapped. Its Kafka driver is its own `kafka-latency.yaml` with only what one broker needs changed *(replication factor and minimum in-sync replicas of one)*. One topic of one partition, 512-byte messages at the block's rate for a minute, after its own default warm-up of a minute.	**Two**: the producer worker stamps the message, the consumer worker subtracts.	Drops it, and counts nothing: its filter keeps only a difference above zero *(T-P3: drops zero and below)*
Pulsar perf	A consumer *(wrapped)* and a producer, each run with its own subcommand, the consumer started first on a topic of its own and reading from its latest message. The consumer runs for the producer's minute and thirty seconds more, and writes its histogram file.	**Two**: the producer's client stamps the message, the consumer subtracts.	Drops a negative and keeps a zero, counting neither *(T-P4: drops the negatives)*
emqtt-bench	A subscriber *(wrapped)* and a publisher, each its own subcommand, the publisher stamping each payload with its millisecond clock and pacing in whole milliseconds; the subscriber's metrics page is read when the publisher has finished.	**Two**: the publisher stamps, the subscriber subtracts.	Its histogram keeps every value; its console average counts one as zero *(T-P2's "keeps all", which the audit found held in part)*
YCSB	One client thread reading 512-byte records of the broker's Redis at the block's rate, loaded first and outside the timing; wrapped whole.	One, the client's monotonic clock.	None can arise: it times against one clock
nats bench	Its request mode *(wrapped)* against its own service mode, started first on the machine itself.	One, the requester's monotonic clock.	None can arise: it times against one clock

Each crosses the direction T1 delays once *(D25-5)*: a reply to a request, or a message delivered to the process that subtracts, stamped before it left a producer that waits for nothing. So each is expected to move one millisecond per millisecond added, and the band T1 holds it to is 0.5 to 1.5.

Which of its figures is read

- **OpenMessaging Benchmark.** At this commit its log prints no end-to-end figure; the result file it writes does. Its end-to-end average, median, 99th percentile and maximum are read from that file, and its smallest value from the quantiles the file lists. Every sample is a whole number of milliseconds, so every figure but the average steps in whole milliseconds. Where the quantiles list nothing, the histogram recorded nothing, and its zeros are not read as measurements.

- **Pulsar perf.** The consumer's closing line: its mean, to a thousandth of a millisecond, and its median, 99th percentile and maximum, in whole milliseconds. How many it kept is read from its histogram file, which is the only place it says; "records received" counts every message, the refused ones among them. An empty histogram prints a mean of 0.000 and every percentile as 0 — the audit found it — so where the file shows it kept nothing, the line is not read as a measurement.
- **emqtt-bench.** The average is its histogram's sum over its count, the tool's own record of every sample, and the only one of its figures fine enough for T1. Its console prints a whole-millisecond average once a second; those are kept beside the reading and reported, not judged. It prints no percentile.
- **YCSB.** Its read summary, in microseconds: average, minimum, maximum, and the 50th and 99th percentiles it is asked for, as k6 is asked for its 99th.
- **nats bench.** Its summary line, in microseconds to two decimals: minimum, average, maximum, median and 99th percentile.

What T2's judge assumes, and where these tools differ

T2 predicts what each behaviour would print by putting our reference's trips, less the offset, through the tool's step, truncating toward zero *(D15-2)*. That is the arithmetic of a tool that subtracts two fine timestamps and then rounds the difference, as Kafka's EndToEndLatency does. The three two-process tools here subtract two stamps that are each already whole milliseconds, whose difference is not the true one truncated but the later stamp's millisecond less the earlier's. The judge is not changed for them: its verdicts are read as written, beside the figures and the counts behind them, and where its model and theirs part, that is reported. Its predictions also come from our reference's trips, which are not the tool's own: a tool that batches or lingers before it sends has longer trips than our client, meets fewer values below zero at a given offset than predicted, and can be left undecided. Neither touches the question T2 answers first and most firmly, whether the tool's figures moved with the clock it reads at all. What decides most against both is the count. Pulsar perf's histogram file says how many latencies it kept, so a behaviour that keeps every value is ruled out by a count short of what was sent, whatever the averages say.

What is built, and with what

The OpenMessaging Benchmark at 5b1fa709 with Maven 3.9.9 on Java 17, its docker module left out; Pulsar from its 4.2.4 release archive on Java 17; emqtt-bench at 0.6.3 on Erlang/OTP 28.1 as hex.pm builds it for Ubuntu 22.04, without QUIC; YCSB's Redis binding at 66302f30 with Ubuntu's Maven on Java 11. Every archive is checked against the checksum its publisher gives, and each build is fingerprinted where it was installed. Java 11 stays the machine's default, so the eleven's Java tools run on what they ran on.

19 Addendum: the four tools of October, how each is run and keeps time, and K1 [D35-3] [D35-6]

Written on 11 October 2026, after the pilot and before any run of these tools or of K1 under this plan. It adds the predictions of D35-2 and D35-7 and removes none. It says how each tool is run, which of its figures is read, how it keeps time and so what T2 is expected to find, and how K1 is run and read.

How each is run, and how each keeps time

As each is invoked *(`cloud/azure/tools_run.sh`)*. "Wrapped" is the process that T1's namespace, T3's go-first and T2's moved clock reach.

Tool	How it is run	Clocks in its subtraction	What its source says it does with a value at or below zero
Xinfra Monitor	One JVM, wrapped whole, holding a single-cluster monitor: a produce service sending a 512-byte record to a topic of its own every 20 ms, each waiting for its acknowledgment, and a consume service reading them; a reporter logs the consume service's metrics once a second. It runs the block's minute and forty seconds before it, because its latency metrics are a rolling minute.	One, the JVM's millisecond wall clock, read by the produce service into the record and by the consume service on arrival.	Its average keeps it; its percentiles, from the metrics library its build pins, pin it to 0 *(Percentiles.java in kafka-clients 2.8.2)*. None can arise in one process unless the wall clock steps back
Strimzi Canary	One process, wrapped whole, standalone: a producer and a consumer on a topic of its own, one message per partition each 20 ms tick, told the cluster has one broker, logging each message's duration; its `/metrics` page is read before it stops. Against the Kafka 3.9.1 broker *(D35-5)*.	One, the process's wall clock in milliseconds, at send and on receipt.	Its histogram bins it with the fastest messages and its sum keeps it; the reader it ships for its log skips it *(its `canary_latencies.py`)*. None can arise unless the wall clock steps back
Trogdor	A coordinator and two agents, each its own JVM; ProduceBench on one agent at the block's rate for its minute, with the payload that writes the producing agent's wall clock into each record, and ConsumeBench with its timestamp record processor on the other, which is wrapped.	**Two**: the producing agent stamps, the consuming agent subtracts.	Puts it in the histogram's lowest bucket, reported as the floor, uncounted *(replaces the negatives with zero)*
Stream PerfTest	Two instances, each its own JVM: a consumer *(wrapped)* started first on a stream of its own, and a producer at the block's rate and size for its minute; the consumer serves its metrics page, read once the producer has finished, and stops fifteen seconds after it, deleting the stream.	**Two**: the producer writes its wall clock into each message's first eight bytes, the consumer subtracts its own.	Drops a negative and keeps a zero, uncounted *(drops the negatives)*

Each crosses the direction T1 delays once *(D25-5)*: a record delivered to the process that subtracts, stamped before it left a producer that waits for nothing between its stamp and its send. So each is expected to move one millisecond per millisecond added, and the band T1 holds it to is 0.5 to 1.5.

Which of its figures is read

- **Xinfra Monitor.** The last reported value of its consume service's average, 99th percentile and maximum *(`records-delay-ms-avg`, `-99th`, `-max`)*, each covering the run's last minute. Its percentiles step in one-millisecond buckets and its maximum in whole milliseconds, whatever digits Java prints; its average keeps the digits it prints. Before its first record it prints NaN, which is not read. Its count of records consumed is kept beside the figures; it counts nothing it was asked to send.

- **Strimzi Canary.** Its average is its histogram's sum over its count, read off `/metrics`, the only one of its metrics finer than their 5 ms buckets. Its median, 99th percentile, minimum and maximum are read from its log's line per message, minus signs included, each the value at n times q in the sorted durations, in whole milliseconds.
- **Trogdor.** The consume task's status as the coordinator reports it once the task has ended: the timestamp record processor's average, median and 99th percentile, the average to its printed digits and the percentiles in whole milliseconds, and the count received. ConsumeBench's own figures beside them time the wait between batches and are not read; nor is the produce task's, which is the publish latency, the other span.
- **Stream PerfTest.** The consumer's line a second, each holding the last second's median and 99th percentile in whole milliseconds; the reading is the median of its medians and the median of its 99th percentiles, over the seconds in which it consumed anything, the first five of those left out as warm-up, and every line's figures are kept beside it. Its average is its latency timer's sum over its count, read off its metrics page once the producer has finished, and that count is how many it kept. A second in which it kept nothing prints four zeros, so where it kept fewer than it consumed such a second is not read, and where it kept nothing there is no figure. Its closing summary gives only a 95th of about its last second and is not read.

What T2 can ask of them

For Trogdor and Stream PerfTest, T2's question is the one it was built for: with the consuming process's clock moved back, values below zero arise, and the four behaviours predict what each would then print. Both subtract two stamps that are each already whole milliseconds, which is not the truncated fine difference T2's arithmetic assumes *(D34-4)*; their verdicts are read as written, beside the figures and counts behind them. Stream PerfTest's metrics page counts what it kept; Trogdor counts what it received, not what its histogram kept. For Xinfra Monitor and Strimzi Canary, which take both stamps in one process, T2 can ask only whether their figures move with the clock, and on one clock they cannot. A value below zero arises in them only if the wall clock steps back between a stamp and a receipt, as it did in one instance of Stream PerfTest during a cluster's rolling restart *(issue 418)*; T2 holds its offset constant, and does not reach it.

K1

The traced copy of Kafka's tool *(external/kafka_e2e/EndToEndLatencyTraced.java)* differs from the 3.7.1 release's EndToEndLatency only by writing, for each record, a line of its index, its partition and offset, the wall-clock microsecond at which the record began, at which `send().get()` returned and at which the poll returned it, and the tool's own elapsed nanoseconds; `EndToEndLatencyTraced.diff` beside it is the whole change. The observer *(ArrivalObserver.java)* assigns itself the topic's partitions, seeks to their end before the tool starts, long-polls with a fetch that the broker answers as soon as one byte is there, and stamps every record of a poll with the wall clock read the moment the poll returns. Its own lateness can only make its stamps later, and so make a record look less early than it was: an observer that reads late understates the case K1 is after. It stops once it has read the 3,000 records. Both run in the receiver's namespace; the observer at real-time priority in every cell, the tool at normal priority or, for go-first, at real-time priority. The 88% load is T3's: stress-ng on every processor. `scripts/kafka_clamp.py` pairs the two files by partition and offset, counts a record the observer never read, leaves out a torn row and counts it, and gives per run the share of records with $S < 0$, the quantiles of S, how much later than the observer the tool reported each early record, and the tool's printed average and its median and 99th percentile computed as the tool computes them, beside the observer's.

What is built, and with what

Xinfra Monitor at 2.5.16 with its own Gradle wrapper on Java 11; Strimzi Canary at f3812fb5 with Go 1.22.5, the Go the block's Go tools are built with, as its Makefile builds it; Trogdor and K1's traced tool and observer from the Kafka 3.7.1 release archive the block already installs, on Java 11; Stream PerfTest's 1.8.0 release jar on Java 11; and the canary's Kafka 3.9.1 from Apache's release archive on Java 17 on the broker. Every archive is checked against the checksum its publisher gives, and each build is fingerprinted where it was installed.

20 Addendum: the Yahoo Streaming Benchmark, how it is run and read [D36-4]

Written on 11 October 2026, after its pilot and before any of its judged runs.

How it is run, and how it keeps time

As `cloud/azure/tools_run.sh` invokes it. The topic `ad-events` is made afresh, and its generator clears the broker's Redis and writes its campaigns there *(its `-n`)*. A Flink JobManager starts, then, once it answers, a TaskManager told its address on the loopback, each its own JVM, the TaskManager wrapped; once the TaskManager has registered its job is submitted; its generator emits ad events into that topic on the Kafka 3.9.1 broker at 1,000 a second for two minutes *(its `-r`)*, each event's time computed from the generator's wall clock read once at its start. The job keeps, for each campaign and ten-second window of event time, a count, and the TaskManager writes each changed window to Redis once a second with its own wall clock as the window's `time_updated`. Five seconds after the generator stops, its collection *(its `-g`)* writes, for every window in Redis, `time_updated` less the window's start into `updated.txt`. Two clocks: the generator's, in the window's start, and the TaskManager's, in its last update.

How it is read

As its notebook *(`pynotebooks/Benchmark_combined.ipynb`)* reads `updated.txt`: the windows at or above ten seconds are kept and ten seconds subtracted, and the average, median, 99th percentile, minimum and maximum of what is left are read, the percentiles by numpy's linear interpolation, as its notebook computes them. The count kept is held against the count of windows written, which is what T2's count clause reads for this tool, since its notebook counts windows; every written window's value less the window, sign kept, is kept beside the reading. A second cell of its notebook drops what is left at or below a second; it draws a chart and is not read.

What T1 and T2 can ask of it

T1's staircase adds at most 2 ms to the path its TaskManager reads Kafka through; a window's last update is written at the first flush after the window's last event, so the staircase moves it only where the last event is pushed past a flush, and its figures are predicted not to follow *(D36-5)*. T2 moves the TaskManager's clock back by its reference's median latency after the window plus 1 ms and plus 3 ms, about half a second each: about half the windows' latencies come out below zero, and its notebook is predicted to drop them, uncounted.

21 Addendum: PerfKit Benchmarker and the step test, how each is run and read [D37-3]

Written on 11 October 2026, after their pilot and before any of their judged runs.

PerfKit Benchmarker's messaging benchmark

As `cloud/azure/tools_run.sh` invokes it: its Azure benchmark script, laid out as its messaging scripts expect, run from the driver with its end-to-end scenario, 100 messages of 10 bytes, no warm-up, against a topic and a subscription of the queue made for it. Its publisher and its receiver are two processes of one program; each message carries the publisher's `time.time_ns()`, and the receiver subtracts it from its own and floor-divides the difference to whole milliseconds. It prints its metrics as one line of JSON when it ends, with every sample; the samples, its mean, median and 99th percentile and its failure count are read. The connection string lives in a file on the driver, readable by its owner only, and every line the tool prints has its key blanked before it is written down.

S1, how a run is made

For each tool and round: the control, the tool as it is; then chrony stopped, the clock measured against the PTP clock, the stepper started with its step and a fresh seed, the tool run, the stepper stopped, the clock measured again and put forward by the difference of the two measurements, chrony started, the clock measured a third time. A message in flight across a step is received at a reading earlier than the one it left with, by the step less its trip, so with a step longer than the trip it comes out below zero. How many are caught is the number of steps times the tool's rate times its trip: at fifty messages a second, a millisecond's trip and five steps a second, some fifteen a minute; PerfKit, sending one message about every three-quarters of a second over a trip of 20 ms, some ten in its run.

S1, how a run is read

PerfKit: its samples, and how many are below zero. Strimzi Canary: the durations it logs, and how many are below zero; its Prometheus histogram and the log reader its repository ships are kept beside them. Stream PerfTest: the count its Prometheus endpoint gives of the latencies it kept, against the messages its consumer counted. Xinfra Monitor: each line in which kafka-clients pins a value below the floor of its percentiles, with the value. Kafka's end-to-end tool: its median and 99th percentile, in the whole milliseconds it prints. `scripts/s1_block.py` folds the four rounds and judges S1-a to S1-f as D37-4 fixes them.

22 Addendum: four tools of the second October reading, how each is run and read [D38-2]

Written on 11 October 2026, after their pilot and before any of their judged runs.

What each times, from its source

mqtt-benchmark stamps `time.Now()` just before a client publishes and again when the publish's token completes, which at QoS 1 is the broker's acknowledgment, and keeps the difference, in milliseconds as a float, from Go's monotonic readings *(client.go:110, 120, 57)*. It prints each client's minimum, maximum, mean and standard deviation, and their totals. **Sarama's tool** prints Sarama's request-latency metric: every request on every broker connection, each from just before its write to its response, monotonic and truncated to whole milliseconds before it enters a reservoir of 1,028 *(broker.go:1371, 1983-1984)*; produce and metadata requests alike, and per request, not per message. **SBK**'s writer writes its wall clock into the first bytes of each record and its reader subtracts it from its own when the record arrives, the first record of each poll standing for every record of that poll; a value below zero is kept out of its minimum, maximum, mean and percentiles and counted as invalid *(KafkaReader.recordReadTime; LatencyRecorder.java:125-126)*. **kgo-repeater**'s producer writes `time.Now().UnixMicro()` into each message and whichever repeater consumes it subtracts it from its own, with no test of the sign, into a reservoir of 1,024 from which it prints the median, 90th and 99th percentiles *(repeater_worker.go:258-261, 412-415; worker.go:18-30)*.

How a run is read

mqtt-benchmark: its JSON results; the mean of every message from each client's mean weighted by its successes, and the totals' minimum and maximum; its successes, of the 3,000 it was asked to send. Sarama's tool: the last of its metrics lines. SBK: its total line, the latencies it called invalid beside the records it read. kgo-repeater: B's status, its one worker's end-to-end median, 90th and 99th percentiles, and the messages it consumed since its reset. `scripts/tool_readings.py` reads each, and `scripts/s1_block.py` judges S1-g to S1-i as D38-5 fixes them.

23 Addendum: three tools on brokers of their own, how each is run and read [D39-2]

Written on 11 October 2026, after their pilot and before any of their judged runs.

What each times, from its source

Artemis's perf producer writes its clock, in microseconds, into each message's `time` property, and its consumer subtracts it from its own when the message arrives and records the difference in an HdrHistogram *(SkeletalProducerLoadGenerator.java:134; RecordingMessageListener.java:85-88)*; the message is counted as received before that, so a value the histogram refuses is received and not kept, and the JMS client rolls it back for the broker to deliver again *(JMSMessageListenerWrapper.java:111-128)*. **RocketMQ's Consumer** subtracts each message's born time, written by the producer's client, from its own `System.currentTimeMillis()`, adds the difference to a total and raises a maximum that starts at 0 only past it, resetting the maximum at each report *(Consumer.java:112, 167-177, 265)*. **BookKeeper's benchmark** takes `System.nanoTime()` just before each `asyncAddEntry` and again when the add completes, keeps only differences above zero, dumps them and logs the 95th and 99th percentiles *(BenchThroughputLatency.java:187-188, 239, 397-439)*.

How a run is read

Artemis's perf client: the summary its consumer's JSON file ends with, and the AMQ132001 warnings on its console. RocketMQ's benchmark: the Consumer's reports with messages in them, and the Producer's last count. BookKeeper's benchmark: its dump and its log. `scripts/tool_readings.py` reads each, `scripts/block_v39.py` judges T2-a, T2-r and T4-b, and `scripts/s1_block.py` judges S1-k, as D39-4 and D39-5 fix them.

24 Addendum: two tools built from source, how each is run and read [D40-2]

Written on 12 October 2026, after their pilot and before any of their judged runs.

What each times, from its source

Quiver's sending arrow writes `System.currentTimeMillis()` into each message's `SendTime` property just before sending it, and its receiving arrow writes that and its own `System.currentTimeMillis()` at receipt, as one line of its transfers file, for every message *(QuiverArrowJms.java:233, 274)*. When the arrow ends, its wrapper reads the file into unsigned 64-bit integers, subtracts, and writes the average and the percentiles of the differences into its summary *(arrow.py:264-265, 301-308, 310-346)*. **The Pravega benchmark's writer** puts `System.currentTimeMillis()` in each event's first eight bytes, and its reader, in the same JVM, subtracts it from its own; one statistics task counts each difference into an array of an hour's milliseconds, indexed by the difference, and prints a window each second and its total at the end *(WriterWorker.java:207-208; ReaderWorker.java:135-141; PerfStats.java:123-126, 233-234, 334-337, 351-355)*.

How a run is read

Quiver: its receiver's summary, its transfers read again with their sign, and its sender's summary. The Pravega benchmark: its final line, its window lines and the exceptions it prints. `scripts/tool_readings.py` reads each, `scripts/block_v40.py` judges T2-q, and `scripts/s1_block.py` judges S1-q and S1-p, as D40-4 and D40-5 fix them.

25 Addendum: Google's Pub/Sub load test, how it is run and read [D41-2]

Written on 13 October 2026, after its pilot and before any of its judged runs.

What it times, from its source

Its **publisher** takes `System.currentTimeMillis()` once for each message, writes it into the message's `sendTime` attribute, and times its own publish from it to the success of the publish future *(AbstractPublisher.java:83, 93-95; CPSPublisherTask.java:75)*. Its **subscriber** subtracts `sendTime` from its own `System.currentTimeMillis()` on receipt *(CPSSubscriberTask.java:68-70)*. Each client puts each value into bucket $\lfloor \log_{1.5} v \rfloor$, floored at 0 *(MetricsHandler.java:72-73)*, and sends its buckets to the driver, whose controller reads bucket n as $[1.5^{n-1}, 1.5^{n})$, prints the range that holds each side's 50th, 99th and 99.9th percentiles, and writes the midpoints of the 50th, 90th, 99th and 99.9th to `output.csv` *(LatencyTracker.java:28-59; Driver.java:239-246; CsvOutput.java:83-86, 95-98)*.

How a run is read

`scripts/tool_readings.py` reads its console, its exit status, the test's length and its `output.csv`; `scripts/block_v41.py` judges T2-l and T4-l, and `scripts/s1_block.py` S1-l, as D41-4 and D41-5 fix them.

26 Addendum: a full fix, tested [D42-2]

Written on 13 October 2026, after its pilot and before any of its judged runs.

The claim

A benchmark that takes both of a message's timestamps where the message physically leaves and enters the client machine, in the kernel, never in a program thread, reports the latency of the system under test, the network and the broker, with nothing the host's threads add: no late reading, no value at or below zero, nothing to delete, and no wait for a core. The host's own waits are measured beside it, so that the latency an application sees is that latency plus them.

How a run is read

`scripts/full_fix.py` pairs each message's publish time and receipt, from the clients' own files, with its departure and arrival in the two captures, and writes one row per run: D, W, H_{send} and H_{recv}, each run's median, mean and 99th percentile, how many messages each capture missed, and how many wire times came out at or below zero. Where the run captured the card's own stamps, it adds the wire time to the card and the kernel's own delay, described and not judged. F-a to F-e are judged from those rows as D42-3 fixes them.

27 Addendum: the full fix as a tool would read it [D43-2]

Written on 14 October 2026, after its pilot and before any of its judged runs.

The claim

The kernel timestamps a benchmark client reads on its own sockets through `SO_TIMESTAMPING`, the transmit timestamp of the packet that carries each message out and the receive timestamp of the packet that brings it back, give the same kernel-timed latency as the packet captures, message by message, and so keep the clients' waits out of the latency as version 42's captures did.

How a run is read

`scripts/so_fix.py` pairs each message's publish time and receipt, from the clients' own files, with its departure and arrival in the two captures and in the two socket files, and writes one row per run: D, W and W_{so}, each run's median, mean and 99th percentile, $|W_{\mathrm{so}} - W|$'s median and 99th percentile, how many messages either socket missed, and how many socket latencies came out at or below zero. G-a to G-e are judged from those rows as D43-3 fixes them.